

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG

-----○○○-----



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP
XÂY DỰNG HỆ THỐNG CỤM
ĐỒNG HỒ SỐ TRÊN XE HƠI

SVTH:

LÊ TRÍ BÌNH AN

MSSV: 22119160

NGUYỄN HUỲNH

THANH PHÚC

MSSV: 22119212

GVHD:

ThS. LÊ MINH

TP HỒ CHÍ MINH – 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

XÂY DỰNG HỆ THỐNG CỤM ĐỒNG HỒ SỐ TRÊN XE HƠI

SVTH:

LÊ TRÍ BÌNH AN

MSSV: 22119160

NGUYỄN HUỲNH THANH PHÚC

MSSV: 22119212

GVHD:

ThS. LÊ MINH

TP.HCM 06/2026

THÔNG TIN KHÓA LUẬN TỐT NGHIỆP

1. Thông tin sinh viên

Họ và tên sinh viên 1: Lê Trí Bình An

Email: Letribinhan94@gmail.com

Họ và tên sinh viên 2: Nguyễn Huỳnh Thanh Phúc

Email: phucnht3108@gmail.com

MSSV: 22119160

Điện thoại: 0917552302

MSSV: 22119212

Điện thoại: 0585672104

2. Thông tin đề tài

- Tên đề tài: Xây dựng hệ thống cụm đồng hồ số trên xe hơi
- Đơn vị quản lý: Bộ môn Kỹ Thuật máy tính – Viễn Thông, Khoa Điện Điện Tử, Trường Đại Học Công Nghệ Kỹ Thuật Tp. Hồ Chí Minh.
- Thời gian thực hiện: Từ ngày 01 / 03 / 2026 đến ngày 25 / 06 / 2026
- Thời gian bảo vệ trước hội đồng: Ngày / / 20...

3. Lời cam đoan của sinh viên

Nhóm gồm Lê Trí Bình An và Nguyễn Huỳnh Thanh Phúc cam đoan KLTN là công trình nghiên cứu của cả nhóm, dưới sự hướng dẫn của Thạc sĩ Lê Minh. Kết quả công bố trong KLTN là trung thực và không sao chép từ bất kì công trình nào khác.

Tp. HCM, ngày ... tháng ... năm 20...

SV thực hiện đồ án
(Ký và ghi rõ họ tên)

.....

Giảng viên hướng dẫn xác nhận quyền báo cáo đã được chỉnh sửa theo đề nghị được ghi trong biên bản của Hội đồng đánh giá Khóa luận tốt nghiệp.

.....

Tp. HCM, ngày ... tháng ... năm 20..

Xác nhận của Bộ Môn

Giảng viên hướng dẫn
(Ký, ghi rõ họ tên và học hàm – học vị)

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Xây dựng hệ thống cụm đồng hồ số trên xe hơi.

Sinh viên: + Lê Trí Bình An

MSSV: 22119160

+ Nguyễn Huỳnh Thanh Phúc

MSSV: 22119212

Hướng dẫn: ThS. Lê Minh

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

Đặt vấn đề rõ, mục tiêu cụ thể.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

Phương pháp thực hiện hợp lý.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

Kết quả phù hợp với mục tiêu đã đặt ra của đề tài.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

Kết luận phù hợp với đặt vấn đề.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

Bố cục báo cáo rõ ràng.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

Làm việc chuyên cần, nghiêm túc.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

Có trích dẫn tài liệu tham khảo theo chỉ dẫn IEEE.

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

Kết quả kiểm tra trùng lặp với công cụ Turnitin: báo cáo đạt yêu cầu với tỉ lệ trùng lặp dưới 40%... ..

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Tinh thần làm việc tích cực, nghiêm túc.. ..

Đề nghị của giảng viên hướng dẫn

Ghi rõ: “Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp”

Được phép bảo vệ khóa luận tốt nghiệp.

Tp. HCM, ngày 22 tháng 6 năm 2026

Người nhận xét

(Ký và ghi rõ họ tên)

Lê Minh

LỜI CẢM ƠN

Hành trình biến ý tưởng nghiên cứu về **hệ thống cụm đồng hồ số trên xe hơi** thành một sản phẩm và báo cáo hoàn chỉnh là một quá trình học tập và thực hiện đầy thử thách. Nhóm sẽ không thể hoàn thành đồ án này nếu thiếu đi sự hướng dẫn tận tình của quý thầy cô, cũng như sự động viên, hỗ trợ tinh thần từ gia đình và bạn bè.

Nhóm xin bày tỏ lòng biết ơn sâu sắc đến **thầy Lê Minh**, người đã trực tiếp hướng dẫn, định hướng và đồng hành cùng nhóm trong suốt quá trình thực hiện đồ án. Những góp ý chuyên môn của thầy về hệ thống điện – điện tử xe hơi, phương pháp nghiên cứu cũng như cách tiếp cận vấn đề đã giúp nhóm hiểu rõ hơn về cấu trúc, nguyên lý hoạt động và ứng dụng thực tế của đề tài

Nhóm cũng xin chân thành cảm ơn quý thầy cô trong **Khoa Điện – Điện Tử** đã trang bị cho nhóm nền tảng kiến thức chuyên môn vững chắc trong suốt quá trình học tập, giúp nhóm có đủ kiến thức và sự tự tin để thực hiện một đề tài mang tính ứng dụng thực tế trong lĩnh vực công nghệ xe hơi.

Cuối cùng, nhóm xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn quan tâm, động viên và tạo điều kiện thuận lợi để nhóm yên tâm học tập và hoàn thành đồ án này.

Do khuôn khổ thời gian nghiên cứu có hạn và đây là bước đầu tiếp cận hệ thống nhúng xe hơi chuyên sâu, tài liệu khó tránh khỏi một số hạn chế nhất định cần được hoàn thiện thêm trong các giai đoạn phát triển kế tiếp. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ quý thầy cô để đề tài được hoàn thiện hơn trong thời gian tới

Nhóm thực hiện đồ án

Lê Trí Bình An

Nguyễn Huỳnh Thanh Phúc

TÓM TẮT

Sự chuyển dịch từ các cơ cấu cơ khí sang hệ thống điện tử thông minh đang thay đổi cách thức vận hành và điều khiển trên xe hơi hiện đại. Trong đó, cụm đồng hồ táp-lô không còn đơn thuần là các kim đo vật lý mà đã trở thành giao diện giao tiếp người - máy (HMI) cốt lõi, giúp người lái kiểm soát tình trạng xe theo thời gian thực một cách trực quan và chính xác hơn.

Cụm đồng hồ số (IPC) cho phép hiển thị linh hoạt khối lượng lớn dữ liệu vận hành như vận tốc, vòng tua, mức năng lượng và các cảnh báo an toàn trên một màn hình kỹ thuật số duy nhất. Việc nghiên cứu và xây dựng một hệ thống IPC có khả năng xử lý đồ họa mượt mà và truyền tin tin cậy là yêu cầu thực tiễn đối với các kỹ sư máy tính khi tiếp cận lĩnh vực phần mềm nhúng xe hơi hiện nay.

Trong đồ án này, nhóm đã thực hiện thiết kế phần cứng hệ thống trên phần mềm EasyEDA để hoàn thiện sơ đồ nguyên lý và mô phỏng mạch điện. Bộ xử lý trung tâm của hệ thống là máy tính nhúng Raspberry Pi 5, đảm nhiệm vai trò dựng giao diện đồ họa (GUI) trực quan thông qua framework Qt/QML. Dữ liệu đầu vào được thu thập từ các vi điều khiển STM32 đóng vai trò là các ECU giả lập, thực hiện đọc tín hiệu từ cảm biến và thao tác người dùng. Toàn bộ quá trình trao đổi dữ liệu giữa các node STM32 và Raspberry Pi 5 được hiện thực hóa qua chuẩn giao tiếp mạng CAN bus nhằm đảm bảo tính toàn vẹn và ổn định của thông tin.

Thông qua việc triển khai đề tài, nhóm đã xây dựng thành công một mô hình mô phỏng cụm đồng hồ số hoạt động ổn định và có tính tùy biến cao. Kết quả đạt được không chỉ giúp nhóm nắm vững quy trình phát triển phần mềm nhúng thực tế mà còn tạo tiền đề cho các nghiên cứu chuyên sâu về kiến trúc điện - điện tử và buồng lái thông minh trên xe điện trong tương lai.

MỤC LỤC

LỜI CẢM ƠN	i
TÓM TẮT	ii
MỤC LỤC	iii
PHỤ LỤC HÌNH ẢNH.....	v
PHỤ LỤC BẢNG.....	viii
DANH MỤC TỪ VIẾT TẮT	ix
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI.....	1
1.1 Giới thiệu đề tài	1
1.2 Mục tiêu đề tài	1
1.3 Đối tượng và phạm vi nghiên cứu đề tài.....	2
1.4 Giới hạn đề tài.....	4
1.5 Bố cục đề tài	4
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	5
2.1 Chuẩn truyền thông CAN	5
2.1.1 Mạng CAN và cơ chế báo hiệu vật lý	5
2.1.2 Cơ chế định thời bit và đồng bộ hoá	11
2.1.3 Cấu trúc và định dạng khung truyền	12
2.1.4 Cơ chế cô lập lỗi và các bộ đếm sai số (TEC & REC)	17
2.1.5 Kiến trúc phần cứng của một nút mạng CAN.....	22
2.2 Các chuẩn giao tiếp khác	24
2.2.1 Giao thức SPI.....	24
2.2.2 Giao thức I2C	25
2.3 Công cụ hỗ trợ xây dựng và lập trình hệ thống	26
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG	27
3.1 Yêu cầu và sơ đồ khối hệ thống.....	27
3.1.1 Yêu cầu về hệ thống	27

3.1.2	Sơ đồ khối toàn hệ thống.....	28
3.2	Thiết kế hệ thống	30
3.2.1	Khối xử lý dữ liệu Node 1	30
3.2.2	Khối xử lý dữ liệu Node 2	38
3.2.3	Khối xử lý dữ liệu Node 3	47
3.2.4	Khối Giao tiếp CAN tổng.....	57
3.2.5	Khối xử lý trung tâm	61
3.2.6	Khối hiển thị	65
3.2.8	Sơ đồ nguyên lý kết nối toàn hệ thống.....	69
3.2.9	Thi công thiết kế mạch PCB	70
3.3	Thiết kế phần mềm	71
3.3.1	Lưu đồ giải thuật toàn hệ thống.....	71
3.3.2	Lưu đồ xử lý dữ liệu tốc độ vòng tua – node 1	72
3.3.3	Lưu đồ xử lý dữ liệu mức pin, hộp số và chế độ lái - Node 2.....	76
3.3.4	Lưu đồ dữ liệu xi nhan, thời gian, nhiệt độ - Node 3	78
3.3.5	Lưu đồ thu thập tín hiệu và hiển thị trên HMI	81
CHƯƠNG 4: KẾT QUẢ HỆ THỐNG		85
4.1	Giới thiệu	85
4.2	Kết quả phần cứng và màn hình hiển thị	85
4.3	Kết quả mô phỏng và hiển thị Node 1	88
4.4	Kết quả mô phỏng và hiển thị Node 2	90
4.5	Kết quả mô phỏng và hiển thị Node 3	93
4.6	Kết quả hiển thị của trạng thái các Node.....	95
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....		96
5.1	Kết Luận	96
5.2	Hướng phát triển	97
PHỤ LỤC		98
TÀI LIỆU THAM KHẢO.....		99

PHỤ LỤC HÌNH ẢNH

Hình 2.1 Dạng sóng của High-Speed CAN [6]	8
Hình 2.2 Cấu trúc định thời Bit và Lượng tử thời gian (Time Quanta) [2].....	12
Hình 2.3 Cấu trúc các trường dữ liệu trong Khung CAN Tiêu chuẩn (11-bit) [6]	14
Hình 2.4 Bit stuff trong CAN [6]	16
Hình 2.5 Khung lỗi Chủ động [6].....	19
Hình 2.6 Cờ lỗi chủ động được gửi bởi tất cả các node [6]	19
Hình 2.7 Cờ lỗi chủ động được gửi bởi 1 node [6]	20
Hình 2.8 Minh họa cho cờ lỗi thụ động [6]	21
Hình 2.9 Sơ đồ khối CAN Node[6]	24
Hình 3.1 Sơ đồ khối hệ thống.....	28
Hình 3.2 Sơ đồ khối node 1	30
Hình 3.3 Module CANBus TJA 1050 [14]	31
Hình 3.4 Sơ đồ kết nối TJA1050.....	32
Hình 3.5 Rotary Encoder EC11 [15]	33
Hình 3.6 Khối mô phỏng dữ liệu tốc độ, vòng tua.....	34
Hình 3.7 Mạch STM32F103C8T6 BulePill [16].....	35
Hình 3.8 Sơ đồ chân của STM32F103C8T6 [16]	35
Hình 3.9 Sơ đồ kết nối của khối xử lý node 1	36
Hình 3.10 Sơ đồ nguyên lý kết nối tổng thể của Node 1.....	37
Hình 3.11 Sơ đồ khối node 2	38
Hình 3.12 Module MCP2551 [17].....	39
Hình 3.13 Sơ đồ nguyên lý MCP2551	40
Hình 3.14 Biến Trở WH148 [18]	41
Hình 3.15 Sơ đồ kết nối khối mô phỏng dữ liệu mức pin	42
Hình 3.16 Sơ đồ kết nối khối mô phỏng dữ liệu hộp số.....	43
Hình 3.17 sơ đồ kết nối khối mô phỏng dữ liệu chế độ lái	44
Hình 3.18 Sơ đồ kết nối của khối xử lý node 2	45
Hình 3.19 Sơ đồ nguyên lý tổng thể Node 2	46
Hình 3.20 Sơ đồ khối Node 3	47
Hình 3.21 Sơ đồ nguyên lý MCP2551	48
Hình 3.22 nhan Công Tắc Gạt MTS-103 [19].....	49

Hình 3.23 Sơ đồ kết nối khối mô phỏng dữ liệu Xi nhan.....	49
Hình 3.24 Module Cảm Biến BMP180 [20]	52
Hình 3.25 Sơ đồ kết nối khối mô phỏng dữ liệu nhiệt độ	52
Hình 3.26 Module TinyRTC DS1307 [21].....	53
Hình 3.27 Sơ đồ kết nối khối mô phỏng dữ liệu thời gian thực RTC [21]	54
Hình 3.28 Sơ đồ kết nối của khối xử lý node 3	55
Hình 3.29 Sơ đồ nguyên lý kết nối tổng thể của Node 3.....	56
Hình 3.30 Module MCP2515 [22].....	58
Hình 3.31 Sơ đồ kết nối khối giao tiếp can tổng sử dụng module CAN MCP2515	59
Hình 3.32 Module Chuyển Mức Logic 4 Kênh [23]	60
Hình 3.33 Sơ đồ khối Shifter Logic	61
Hình 3.34 Sơ đồ chân của Raspberry Pi 5 [24]	63
Hình 3.35 Sơ đồ kết nối GPIO giao tiếp với Raspberry Pi	64
Hình 3.36 Kích thước màn hình [26]	66
Hình 3.37 Mặt sau của màn hình [26]	66
Hình 3.38 Sơ đồ kết nối vật lý giữa Raspberry Pi 5 và Màn hình LCD [26].....	66
Hình 3.39 Nút 12x12mm [27]	67
Hình 3.40 Sơ đồ nguyên lý nút.....	67
Hình 3.41 Sơ đồ nguyên lý toàn hệ thống.....	69
Hình 3.42 Sơ đồ bố trí linh kiện và đi dây (PCB Layout) của mạch.....	70
Hình 3.43 Thiết kế 3D của mạch PCB tổng thể	70
Hình 3.44 Lưu đồ toàn hệ thống.....	71
Hình 3.45 Lưu đồ xử lý dữ liệu tốc độ vòng tua – node 1	73
Hình 3.46 Lưu đồ xử lý dữ liệu mức pin, hộp số và chế độ lái - Node 2.....	76
Hình 3.47 Lưu đồ dữ liệu xi nhan, thời gian, nhiệt độ - Node 3	78
Hình 3.48 Lưu đồ thu thập tín hiệu và hiển thị trên HMI	81
Hình 4.1 Phần cứng và màn hình hiển thị đồng hồ IPC	85
Hình 4.2 Giao diện HMI - IPC	87
Hình 4.3 Kiểm tra dữ liệu Node 1 bằng lệnh	88
Hình 4.4 Màn hình hiển thị tốc độ - vòng tua.....	89
Hình 4.5 Đo dữ liệu CAN Node 1 bằng logic analyzer.....	90
Hình 4.6 Kiểm tra dữ liệu ID Node 2 bằng lệnh	91

Hình 4.7 Đo dữ liệu CAN Node 2 bằng logic analyzer.....	91
Hình 4.9 Vị trí của các hộp số – Gear (a b c d)	92
Hình 4.8 Màu sắc của mức Pin – Battery (a b c).....	92
Hình 4.10 Màu sắc của chế độ lái – Dmode (a b c)	92
Hình 4.11 Kiểm tra dữ liệu Node 3 bằng lệnh	93
Hình 4.12 a Đo dữ liệu CAN Node 3 - ID 0×103 bằng logic analyzer	94
Hình 4.12 b Đo dữ liệu CAN Node 3 - ID 0×105 và ID 0×1F3 bằng logic analyzer	94
Hình 4.12 c Đo dữ liệu CAN Node 3 - ID 0×107 bằng logic analyzer	94
Hình 4.13 Vị trí của thời gian và nhiệt độ trên giao diện.....	94
Hình 4.14 Vị trí của xi nhan trái – phải trên giao diện.....	94
Hình 4.15 a Trạng thái 3 Node hoạt động tốt.....	95
Hình 4.15 b Trạng thái bị 3 Node ngắt kết nối.....	95
Hình 4.15 c Trạng thái Node 2 bị lỗi.....	95
Hình 4.15 d Trạng thái Node 3 bị lỗi.....	95

PHỤ LỤC BẢNG

Bảng 2.1 Bảng tỉ lệ giữa độ dài đường dây với tốc độ bus	5
Bảng 2.2 Bảng trạng thái bit của CAN.....	6
Bảng 2.3 Bảng trạng thái bit của High-Speed CAN.....	8
Bảng 2.4 Ví dụ về cơ chế phân xử quyền ưu tiên trên Bus CAN.....	13
Bảng 2.5 Bảng cấu trúc các trường dữ liệu trong Khung CAN Tiêu chuẩn (11-bit)	16
Bảng 2.6 Bảng sự thay đổi của REC và TEC theo sự kiện	18
Bảng 3.1 Chức năng từng chân của Module TJA1050 [14].....	32
Bảng 3.2 Chức năng từng chân của Rotary Encoder EC11 [15].....	33
Bảng 3.3 Thông số kết nối của Node 1	37
Bảng 3.4 Chức năng từng chân của Module MCP2551 [17]	39
Bảng 3.6 Thông số kết nối của Node 2	47
Bảng 3.7 Chức năng từng chân của Công Tắc MTS-103 [19].....	50
Bảng 3.8 Chức năng từng chân của Cảm Biến BMP180 [20].....	52
Bảng 3.9 Chức năng từng chân của Module TinyRTC DS1307	54
Bảng 3.10 Thông số kết nối của Node 3	57
Bảng 3.11 Chức năng từng chân của Module MCP2515 [22]	58
Bảng 3.12 Chức năng từng chân của Module Chuyển Mức Logic [23]	60
Bảng 3.13 Bảng kết nối chân giao tiếp giữa Pi, Logic Converter và MCP2515.....	61
Bảng 3.14 Thông số kỹ thuật của Raspberry Pi 5 [24].....	62
Bảng 3.15 Danh sách các chân tín hiệu sử dụng trên Header J1	64
Bảng 3.16 Thống kê dòng điện tiêu thụ trong hệ thống	68
Bảng 4.1 Mô tả thành phần phần cứng hệ thống.....	86
Bảng 4.2 Mô tả thành phần giao diện HMI	88
Bảng 4.3 Dữ liệu ID Node 1	88
Bảng 4.4 Dữ liệu ID Node 1 sau khi xử lý	89
Bảng 4.5 Dữ liệu ID Node 2.....	90
Bảng 4.6 Dữ liệu ID Node 2 sau khi xử lý	91
Bảng 4.7 Dữ liệu ID Node 3	93
Bảng 4.8 Dữ liệu ID Node 3 sau khi xử lý	93

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Từ đầy đủ	Ý nghĩa
ACK	Acknowledge	Xác nhận nhận dữ liệu
CAN	Controller Area Network	Mạng điều khiển cục bộ trên xe
CANH	CAN HIGH	Tín hiệu CAN mức cao
CANL	CAN LOW	Tín hiệu CAN mức thấp
CRC	Cyclic Redundancy Check	Mã kiểm tra lỗi dữ liệu
ECU	Engine Control Unit	Bộ điều khiển điện tử trung tâm
EOF	End Of Frame	Kết thúc khung truyền
HMI	Human Machine Interface	Giao diện người - máy
HV	High Voltage	Về điện áp cao
I2C	Inter Integrated Circuit	Chuẩn giao tiếp 2 dây nối tiếp
ID	Identification	Mã gói tin dữ liệu CAN
IDE	Identifier Extension	Bit xác định chuẩn CAN
IPC	Instrument Panel Cluster	Màn hình đồng hồ số trên ô tô
LCD	Liquid Crystal Display	Màn hình tinh thể lỏng
LV	Low Voltage	Về điện áp thấp
MISO	Master Input Slave Output	Dữ liệu truyền từ Tớ sang Chủ
MOSI	Master Output Slave Input	Dữ liệu truyền từ Chủ sang Tớ
RTR	Remote Transmission Request	Yêu cầu truyền dữ liệu
SCK	Serial Clock	Xung nhịp đồng bộ
SOF	Start Of Frame	Bắt đầu khung truyền
SPI	Serial Peripheral Interface	Chuẩn giao tiếp đồng bộ nối tiếp
SS	Slave Select	Chân chọn thiết bị Tớ

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1 Giới thiệu đề tài

Ngành công nghiệp xe hơi hiện đại đang trải qua một bước ngoặt lớn, đặt ra yêu cầu tái định nghĩa giao diện tương tác giữa người lái và phương tiện (HMI). Để đáp ứng xu thế này, cấu trúc đồng hồ cơ khí truyền thống đang dần nhường chỗ cho cụm đồng hồ kỹ thuật số (Instrument Panel Cluster - IPC). Giải pháp nâng cấp này không chỉ cải thiện thẩm mỹ không gian nội thất mà còn tối ưu hóa phương thức truyền tải thông tin đến người điều khiển.

Để giải quyết bài toán này, Cụm đồng hồ số (Instrument Panel Cluster - IPC) đã trở thành xu hướng công nghệ tất yếu. Việc ứng dụng màn hình kỹ thuật số cho phép hệ thống thu thập và trình bày linh hoạt các thông số như vận tốc, vòng tua hay cảnh báo lỗi dưới dạng đồ họa sinh động. Nhờ đó, người lái có thể dễ dàng nắm bắt thông tin theo thời gian thực và tùy biến giao diện theo từng chế độ lái.

Nắm bắt xu hướng đó, nhóm quyết định thực hiện đề tài **“Xây dựng hệ thống cụm đồng hồ số trên xe hơi”**. Đồ án tập trung ứng dụng nền tảng Linux nhúng và các công cụ đồ họa hiện đại để thiết kế một màn hình HMI hoàn chỉnh. Đồng thời, đề tài cũng mô phỏng thành công quá trình thu thập và truyền nhận dữ liệu vận hành từ vi điều khiển lên màn hình trung tâm, tạo tiền đề vững chắc cho công việc phát triển phần mềm nhúng xe hơi thực tế.

1.2 Mục tiêu đề tài

Mục tiêu cốt lõi của đề tài là nghiên cứu và xây dựng hoàn thiện một mô hình cụm đồng hồ số trên xe, có khả năng mô phỏng chính xác các hành vi vận hành của phương tiện và cung cấp giao diện tương tác trực quan cho người lái. Hệ thống hướng tới việc duy trì luồng dữ liệu thời gian thực ổn định và hiển thị đồ họa hiệu năng cao. Cụ thể, đề tài tập trung hiện thực hóa các mục tiêu sau:

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

- Thiết lập mạng lưới thu thập dữ liệu: Xây dựng hệ thống gồm 03 đơn vị thu thập dữ liệu (Node) độc lập để số hóa các tương tác vật lý và thông số môi trường. Các khối chức năng cần xây dựng bao gồm:
 - o Khối mô phỏng tốc độ vòng tua: Tái tạo sự thay đổi của vận tốc xe, vòng tua máy và tính toán quãng đường hành trình.
 - o Khối mô phỏng năng lượng, hộp số và chế độ lái: Giả lập trạng thái dung lượng pin, vị trí tay số và các chế độ lái thích ứng của phương tiện.
 - o Khối mô phỏng tín hiệu xe và : Cung cấp các lệnh chỉ thị báo rẽ (xi nhan), thu thập dữ liệu nhiệt độ môi trường và đồng bộ hóa thời gian hệ thống.
- Triển khai mạng truyền thông giao tiếp: Sử dụng giao thức CAN Bus làm trục xương sống để liên kết toàn bộ hệ thống. Quá trình trao đổi dữ liệu giữa các node thành phần và bộ điều phối trung tâm phải đảm bảo tính có cấu trúc, tin cậy, chống nhiễu tốt và duy trì độ trễ thấp để đáp ứng yêu cầu thời gian thực.
- Phát triển giao diện người - máy (HMI): Xây dựng lớp giao diện đồ họa hiện đại, trực quan để trình diễn toàn bộ thông số vận hành lên màn hình kỹ thuật số chuyên dụng. Hệ thống hiển thị cần đạt tốc độ làm tươi cao, đảm bảo các hoạt ảnh như kim đồng hồ ảo chuyển động mượt mà và tuân thủ các tiêu chuẩn về an toàn chức năng trong ngành ô tô.
- Đánh giá khả năng vận hành: Thực hiện các kịch bản chạy thử nghiệm để xác nhận dữ liệu từ các khối mô phỏng được truyền tải và hiển thị chuẩn xác, không xảy ra hiện tượng mất gói tin hay trễ thông tin trên giao diện người dùng..

1.3 Đối tượng và phạm vi nghiên cứu đề tài

Đề tài tập trung nghiên cứu thực thể hệ thống Cụm đồng hồ số (IPC) tích hợp Bộ điều khiển xe (VCU) theo kiến trúc phân tán (Zonal Architecture). Các đối tượng nghiên cứu cụ thể bao gồm:

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

- Bộ điều khiển trung tâm (VCU): Sử dụng máy tính nhúng hiệu năng cao (như Raspberry Pi 5) để quản lý máy trạng thái xe và thực thi các tác vụ xử lý đồ họa phức tạp.
- Mạng lưới các nút ECU giả lập: Triển khai các nút vi điều khiển độc lập (dòng STM32) đóng vai trò số hóa tương tác vật lý từ người dùng và dữ liệu cảm biến tại biên.
- Hệ điều hành nhúng: Xây dựng bản phân phối Linux nhúng (Embedded Linux) tùy biến thông qua Yocto Project để tối ưu hóa tài nguyên phần cứng cho bộ xử lý trung tâm.
- Giao diện đồ họa (GUI): Ứng dụng framework Qt/QML cùng kỹ thuật dựng hình phần cứng để thiết kế giao diện HMI trực quan, đảm bảo hoạt ảnh mượt mà ở tốc độ cao.
- Giao thức giao tiếp: Nghiên cứu và triển khai chuẩn mạng CAN Bus để thiết lập luồng trao đổi dữ liệu có cấu trúc, tin cậy và ổn định giữa các node thành phần.

Về phạm vi nghiên cứu, nhóm đã thực hiện hoá đề tài như sau:

- Mô hình thực hiện: Xây dựng mô hình thí nghiệm mô phỏng trên bo mạch PCB chuyên dụng, sử dụng các cơ cấu giả lập (biến trở, encoder, công tắc) thay vì kết nối trực tiếp vào hệ thống xe thật.
- Chức năng mô phỏng: Tập trung tái hiện các nhóm thông số cơ bản gồm động lực học, trạng thái năng lượng và tín hiệu thân xe; các hệ thống hỗ trợ lái nâng cao (ADAS) phức tạp nằm ngoài phạm vi nghiên cứu này.
- Chỉ số kỹ thuật trọng tâm: Chú trọng đo đặc độ trễ phản hồi hệ thống (End-to-End Latency) và đánh giá tính toàn vẹn dữ liệu trên mạng CAN để xác nhận khả năng đáp ứng thời gian thực của mô hình.

1.4 Giới hạn đề tài

Đề tài “**Xây dựng hệ thống cụm đồng hồ số trên xe hơi**” có những giới hạn sau:

- Hạn chế về môi trường thử nghiệm: Hệ thống được triển khai dưới dạng mô hình phần cứng mô phỏng. Dữ liệu vận hành được giả lập thông qua vi điều khiển và chuẩn giao tiếp, chưa được kết nối và đánh giá trực tiếp trên mạng truyền thông của một chiếc xe hơi thực tế.
- Chỉ tập trung vào các thông số cơ bản: Đề tài chủ yếu tập trung thiết kế giao diện đồ họa (HMI) hiển thị các thông số cốt lõi (vận tốc, vòng tua, mức nhiên liệu, nhiệt độ). Hệ thống chưa mô phỏng được các luồng dữ liệu phức tạp từ hệ thống hỗ trợ lái xe nâng cao (ADAS)
- Không có kết nối và cảnh báo từ xa: Do tập trung vào chức năng hiển thị tại chỗ để đảm bảo tính thời gian thực, hệ thống chưa tích hợp tính năng đồng bộ hóa giải trí hay gửi cảnh báo trạng thái xe từ xa qua ứng dụng di động

1.5 Bố cục đề tài

Đề tài được cấu trúc thành 5 chương:

- Chương 1 Tổng quan về đề tài: Nội dung tập trung làm rõ lý do chọn đề tài, giới hạn cùng các mục tiêu cốt lõi cần đạt được.
- Chương 2 Cơ sở lý thuyết: Hệ thống hóa các nền tảng lý thuyết và công nghệ liên quan.
- Chương 3 Thiết kế hệ thống: Quy trình hiện thực hóa sơ đồ nguyên lý và kiến trúc các khối chức năng sẽ được trình bày chi tiết
- Chương 4 Kết quả hệ thống: Đưa ra các số liệu đo đạc và hình thực tế từ sản phẩm
- Chương 5 Kết luận và hướng phát triển: nhìn nhận và nhận định tổng thể cùng định hướng nâng cấp hệ thống trong tương lai

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Chuẩn truyền thông CAN

2.1.1 Mạng CAN và cơ chế báo hiệu vật lý

2.1.1.1 Định nghĩa và Kiến trúc mạng

Mạng Khu vực Bộ điều khiển (Controller Area Network - CAN) được xác lập là một tiêu chuẩn định nghĩa chi tiết về Lớp vật lý (Physical Layer - PHY) và Lớp Liên kết Dữ liệu (Data Link Layer) của một giao diện truyền thông bus vi sai (differential bus communication interface). Trong các ứng dụng xe hơi và công nghiệp, CAN được coi là giao diện truyền thông tiêu chuẩn thực tế (de-facto interface).

Về mặt cấu trúc, mạng CAN vận hành theo sơ đồ hình tuyến (bus topology), cho phép đa thiết bị kết nối trực tiếp vào một đường truyền chung duy nhất. Mặc dù có những điểm tương đồng nhất định với giao thức bus I2C, mạng CAN sở hữu những đặc tính ưu việt hơn nhờ việc ứng dụng các tín hiệu vi sai (differential signals). Cơ chế này không chỉ cho phép đạt được tốc độ truyền dữ liệu cao hơn mà còn đảm bảo khả năng giao tiếp ổn định trên các khoảng cách vật lý lớn. Theo các thông số kỹ thuật, mạng CAN có khả năng mở rộng phạm vi truyền dẫn lên đến 1000 mét nếu sử dụng hệ thống cáp phù hợp, tuy nhiên tốc độ dữ liệu sẽ tỉ lệ nghịch với chiều dài đường dây.

Tốc độ bus	Chiều dài dây bus	Chiều dài nhánh rẽ	Khoảng cách giữa các node
1 Mbps	40m	0.3m	40m
500 kbps	100m	0.3m	100m
100 kbps	500m	0.3m	500m
50 kbps	1000m	0.3m	1000m

Bảng 2.1: Bảng tỉ lệ giữa độ dài đường dây với tốc độ bus [1] [6]

Kiến trúc vật lý của một mạng CAN tiêu chuẩn bao gồm hai đường tín hiệu chính là CAN Cao (CAN High - CAN-H) và CAN Thấp (CAN Low - CAN-L), cùng với một đường

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

dẫn chung (GND) đóng vai trò làm điểm tham chiếu điện thế. Mỗi đơn vị thiết bị tham gia vào mạng lưới được định nghĩa là một nút (node).

Nhằm duy trì tính ổn định của trạng thái mạng và triệt tiêu hiện tượng dội sóng tín hiệu (signal reflections), cả hai đầu cực của đường bus bắt buộc phải được kết thúc bằng các điện trở phối hợp trở kháng có giá trị 120 Ohms (Termination resistors - R_{term}). Đặc tính cốt lõi của hệ thống báo hiệu này nằm ở bản chất vi sai: tại cùng một thời điểm, hai đường dây tín hiệu sẽ truyền tải cùng một thông tin nhưng dưới hai cực tính đối lập nhau.

Trong mô hình thiết kế tiêu chuẩn sử dụng điện thế tham chiếu 0V (GND) và nguồn cấp tối đa 5V, trạng thái nghỉ (idle condition) của mạng được thiết lập khi cả hai đường CAN-H và CAN-L duy trì ở mức điện áp xấp xỉ một nửa nguồn cấp, tương đương 2.5V. Sự kết hợp điện áp giữa hai đường dây này cho phép biểu diễn hai trạng thái logic phân biệt:

Trạng thái Trội (Dominant): Đại diện cho mức logic 0. Trạng thái này được xác lập khi đường CAN-H được điều khiển lên mức điện áp cao hơn và đường CAN-L bị kéo xuống mức điện áp thấp hơn so với ngưỡng tĩnh 2.5V.

Trạng thái Lặn (Recessive): Đại diện cho mức logic 1. Trạng thái này được xác lập khi các đường tín hiệu không bị tác động mạnh, cho phép điện áp trên CAN-H và CAN-L trở về mức cân bằng thông qua hệ thống điện trở đầu cuối.

Trạng thái (State)	CAN-H	CAN-L	Điện áp vi sai (VDiff)	Bit
Nghỉ (Idle)	1/2 VDD	1/2 VDD	0	1
Trội (Dominant)	HIGH	LOW	$CAN-H > CAN-L$	0
Lặn (Recessive)	LOW	HIGH	$CAN-H \leq CAN-L$	1

Bảng 2.2: Bảng trạng thái bit của CAN [1] [6]

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1.1.2 Cơ chế Mã hóa và Phân xử (Arbitration)

Thông tin kỹ thuật số trên mạng CAN được mã hóa thông qua điện áp vi sai, được tính toán dựa trên hiệu số điện thế giữa hai đường dây: CAN_H – CAN_L

Việc quy định trạng thái Trội tương ứng với logic 0 là một đặc tính quan trọng xuất phát từ bản chất chia sẻ và bất đồng bộ (shared and asynchronous) của đường truyền. Do không có sự hiện diện của tín hiệu xung nhịp (clock signals) tập trung hoặc các tín hiệu chọn chip (như Chip-select trong SPI) để điều phối quyền ưu tiên, mọi nút mạng đều có quyền khởi tạo truyền dẫn tại bất kỳ thời điểm nào bus rảnh. Trong tình huống hai hoặc nhiều thiết bị cùng thực hiện truyền dẫn đồng thời, thiết bị nào thiết lập được trạng thái Trội trên đường dây sẽ giành được quyền kiểm soát bus. Quy trình này được gọi là sự phân xử (arbitration), đóng vai trò thiết yếu trong việc thiết lập hệ thống ưu tiên thông điệp trên một hạ tầng truyền dẫn dùng chung. [1] [2] [3] [6]

2.1.1.3 Tiêu chuẩn điện áp theo các phân loại ISO

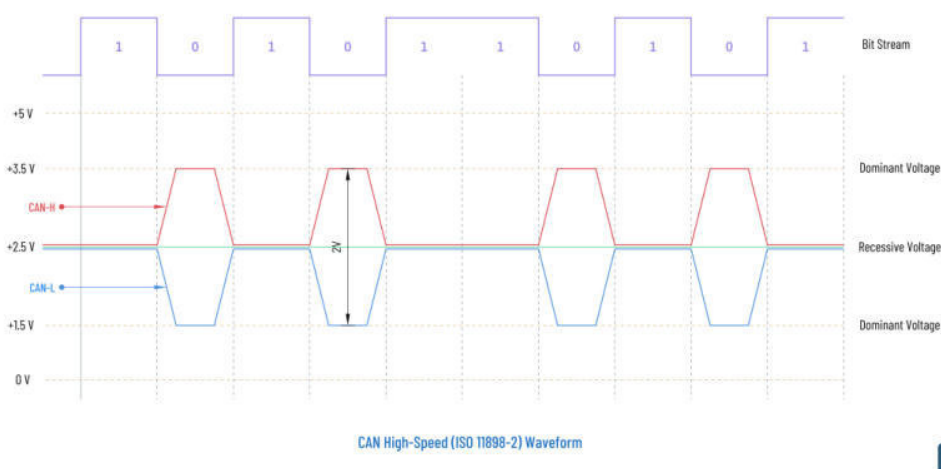
Các mức điện áp định mức cho trạng thái Cao (HIGH) và Thấp (LOW) trên các đường tín hiệu được quy chuẩn hóa dựa trên các biến thể của giao diện:

ISO 11898-2 (High-Speed CAN): Hỗ trợ tốc độ bit lên đến 1 Mbit/s đối với chuẩn CAN truyền thông và lên đến 5 Mbit/s đối với chuẩn CAN-FD. Tiêu chuẩn này sử dụng cấu trúc bus tuyến tính với điện trở đầu cuối 120 Ω . Trạng thái Trội được xác định khi CAN-H đạt 3.5V và CAN-L đạt 1.5V (tạo ra điện áp vi sai là 2V). Ở trạng thái Lặn, hai đường dây không được điều khiển tích cực, các điện trở đầu cuối sẽ kéo điện áp về mức tĩnh 2.5V (điện áp vi sai bằng 0V). Về mặt logic, các nút mạng sẽ diễn giải bất kỳ độ chênh lệch điện áp nào nhỏ hơn hoặc bằng 0.5V là trạng thái Lặn.[2]

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

Trạng thái (State)	CAN-H	CAN-L	Điện áp vi sai (VDiff)	Bit
Trội (Dominant)	3.5V (HIGH)	1.5V (LOW)	2V	0
Lặn (Recessive)	2.5V (LOW)	2.5V (HIGH)	0V	1

Bảng 2.3: Bảng trạng thái bit của High-Speed CAN [1] [6]



Hình 2.1: Dạng sóng của High-Speed CAN [6]

2.1.1.4 Khả năng kháng nhiễu đồng pha (Common Mode Noise)

Nguyên lý giải mã dựa trên điện áp vi sai thay vì điện áp tuyệt đối của từng đường dây cung cấp một lợi thế kỹ thuật quan trọng trong việc triệt tiêu nhiễu đồng pha (Common Mode noise - CM). Nhiễu đồng pha là loại can nhiễu gây ra sự biến đổi điện thế hoặc dòng điện trên các đường tín hiệu theo cùng một phương và cùng một cường độ. Các tiêu chuẩn CAN quy định cụ thể về ngưỡng điện áp đồng pha tối đa cho phép trên các đường truyền để đảm bảo tính toàn vẹn của dữ liệu trong các môi trường có độ nhiễu điện từ cao.

Trong môi trường xe hơi và tự động hóa, hệ thống mạng CAN phải hoạt động dưới điều kiện nhiễu điện từ (EMI) khắc nghiệt sinh ra từ động cơ, hệ thống đánh lửa và các thiết bị công suất lớn. Để đảm bảo tính toàn vẹn của tín hiệu vi sai, việc lựa chọn và thi công cáp

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

truyền dẫn (Bus Line) kết nối giữa các thiết bị trên mạng lưới bắt buộc phải tuân thủ các tiêu chuẩn vật lý nghiêm ngặt.

Theo tiêu chuẩn ISO 11898-2 áp dụng cho mạng CAN tốc độ cao (High-Speed CAN), cáp truyền dữ liệu phải là loại cáp xoắn đôi (Shielded hoặc Unshielded Twisted Pair). Việc sử dụng cáp xoắn đôi nhằm duy trì khoảng cách vật lý đồng đều giữa hai dây CAN_H và CAN_L, từ đó giữ cho trở kháng đặc tính đường dây luôn ổn định ở mức danh định là xấp xỉ 120 Ohm. Yêu cầu kỹ thuật cốt lõi trong tuyến cáp là mật độ xoắn (Twist Pitch) bắt buộc phải đạt từ 33 đến 50 vòng xoắn trên 1 mét chiều dài (tương đương với bước xoắn từ 2.0 cm đến 3.0 cm cho mỗi vòng). Tiết diện lõi cáp thường được sử dụng là chuẩn AWG 22 hoặc AWG 24 tùy thuộc vào tổng chiều dài của toàn mạng. Theo tiêu chuẩn GB/T 36048 và SAE J2284 thì bước xoắn càng nhỏ (xoắn càng chặt) thì khả năng chống nhiễu từ trường càng tốt. Tuy nhiên, không thể xoắn dây quá chặt vì lực căng có thể làm hỏng, nứt lớp vỏ nhựa cách điện. Các kỹ sư phải cân bằng giữa khả năng chống nhiễu, đường kính dây và độ uốn dẻo của vật liệu. [2]

Khả năng chống nhiễu của cáp xoắn đôi khi kết hợp với tín hiệu vi sai được thiết lập dựa trên hai nguyên lý vật lý:

- Triệt tiêu nhiễu điện từ (EMI Cancellation): Khi cáp dẫn đi qua vùng có từ trường biến thiên, dòng điện cảm ứng sẽ sinh ra trên dây dẫn. Việc xoắn đan xen liên tục làm cho vị trí tương đối của hai dây so với nguồn nhiễu thay đổi luân phiên. Hệ quả là chiều của dòng điện nhiễu cảm ứng trên các đoạn xoắn liên tiếp sẽ ngược nhau và tự triệt tiêu lẫn nhau.
- Tối ưu hóa khả năng loại bỏ nhiễu đồng pha (Common-Mode Rejection Ratio - CMRR): Bất kỳ xung nhiễu nào từ môi trường bên ngoài sẽ tác động một điện áp có biên độ gần như bằng nhau V_{noise} lên cả hai dây CAN_H và CAN_L. Vì dữ liệu được giải mã dựa trên chênh lệch điện áp giữa hai dây, thành phần nhiễu đồng pha này sẽ bị triệt tiêu hoàn toàn qua phép trừ vi sai tại IC thu phát. Sự triệt tiêu này được biểu diễn qua phương trình toán học tính toán điện áp vi sai:

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

$$V_{Diff} = (V_{CAN_H} + V_{noise}) + (V_{CAN_L} - V_{noise}) = V_{CAN_H} - V_{CAN_L} \quad (2.1)$$

Trong giai đoạn đầu phát triển, khi kết nối các thiết bị như bộ điều khiển hiển thị trung tâm (IPC) với các Node ECU mô phỏng trên bàn làm việc, việc sử dụng các cáp nối thẳng (như cáp cắm testboard thông thường) ít khi bộc lộ lỗi vật lý.

Sự ổn định tạm thời này xuất phát từ các yếu tố lý tưởng của môi trường thử nghiệm:

- Chiều dài đường truyền ngắn: Khoảng cách giữa các Node thường chỉ dưới 1 mét. Tại khoảng cách này, hiệu ứng đường truyền (Transmission line effects) chưa đủ lớn để gây ra sự dội tín hiệu (Signal reflection) đáng kể.
- Môi trường điện từ (EMI) lý tưởng: Nguồn cấp cho hệ thống thường là nguồn phẳng, không chịu sự tác động của các xung nhiễu lớn từ động cơ điện, biến tần hay hệ thống đánh lửa.
- Biên độ vi sai an toàn: Với mức chênh lệch điện áp vi sai là 2.0V giữa Bit Trội và Bit Lặn, mạng CAN có dải dự trữ nhiễu (Noise Margin) rất rộng. Khi không có nhiễu ngoại vi, IC thu phát (Transceiver) vẫn lấy mẫu và giải mã thành công 100% khung truyền dữ liệu ngay cả khi cáp không được xoắn.

Khi hệ thống được triển khai lên khung xe hoặc trong dây chuyền sản xuất, môi trường nhiễu điện từ (EMI) cường độ cao sẽ phá vỡ hoàn toàn tính toàn vẹn tín hiệu của các tuyến cáp không xoắn thông qua các cơ chế vật lý sau:

- Suy giảm khả năng loại bỏ nhiễu đồng pha (Common-Mode Rejection Failure): Đối với cáp không xoắn, khi một luồng nhiễu từ trường cắt ngang, khoảng cách từ nguồn nhiễu đến dây CAN_H và CAN_L là khác nhau. Hệ quả là điện áp cảm ứng sinh ra trên hai dây dẫn không đồng đều. Phép trừ vi sai $V_{CAN_H} - V_{CAN_L}$ tại bộ thu không thể triệt tiêu được phần điện áp chênh lệch này, dẫn đến việc giải mã sai trạng thái logic của Bit.
- Phá vỡ trở kháng đặc tính cục bộ: Khoảng cách vật lý giữa hai lõi dây không xoắn thay đổi ngẫu nhiên dọc theo tuyến cáp làm cho điện dung ký sinh C và độ

tự cảm L biến thiên liên tục. Điều này khiến trở kháng đặc tính Z_0 không duy trì được ở mức danh định 120Ω gây ra hiện tượng méo dạng sóng và dội tín hiệu.

- Tăng bức xạ điện từ (Radiated Emissions): Do không có hiệu ứng triệt tiêu từ trường như cáp xoắn đôi, đường cáp CAN thẳng sẽ hoạt động như một ăng-ten, liên tục phát tán nhiều điện từ ra môi trường xung quanh, gây ảnh hưởng đến các cảm biến Analog và mạch tín hiệu yếu trong hệ thống.

2.1.2 Cơ chế định thời bit và đồng bộ hoá

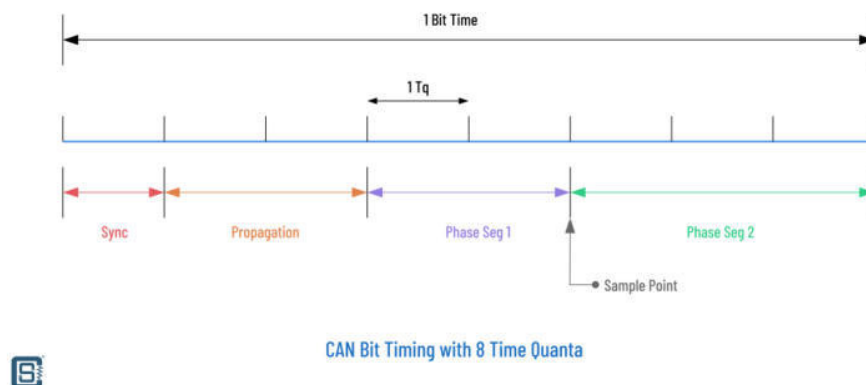
Do mạng CAN không sử dụng đường dây xung nhịp (clock line) chuyên dụng, tất cả các nút mạng phải thiết lập thỏa thuận chung về việc sử dụng cùng một tốc độ bit danh định (nominal bit rate) để đảm bảo khả năng giao tiếp trên cùng một đường truyền. Tuy nhiên, do mỗi nút được trang bị các bộ dao động (oscillators) độc lập với mức dung sai khác nhau để tạo ra xung nhịp cơ sở, hệ thống không thể kỳ vọng tất cả các tần số đều đồng nhất tuyệt đối. Đây chính là yếu tố đòi hỏi sự can thiệp của cơ chế đồng bộ hóa (synchronization). [2]

Các nút mạng có khả năng tự khôi phục xung nhịp từ chính các tín hiệu truyền tải và tiến hành điều chỉnh động (adjust dynamically). Mọi nút CAN đều bắt buộc phải sở hữu năng lực này. Quá trình đồng bộ hóa cứng (hard synchronization) được kích hoạt tại lần chuyển đổi trạng thái đầu tiên từ Lặn sang Trội ($1 \rightarrow 0$) sau một khoảng thời gian bus nghỉ (chính là bit bắt đầu - start bit). Quá trình tái đồng bộ (resynchronization) diễn ra tại mọi điểm chuyển đổi từ Lặn sang Trội trong suốt quá trình truyền khung dữ liệu. Bộ điều khiển CAN (CAN controller) kỳ vọng sự chuyển đổi sẽ xảy ra tại một thời điểm là bội số của thời gian bit danh định. Nếu sự chuyển đổi không diễn ra chính xác tại thời điểm mà bộ điều khiển dự kiến, nó sẽ tự động điều chỉnh lại thời gian bit danh định cho phù hợp.

Nhằm tối ưu hóa tiến trình này, một chu kỳ thời gian bit danh định đơn lẻ được lượng tử hóa thành nhiều lát cắt thời gian có độ dài bằng nhau, được gọi là Lượng tử (Quanta - Tq). Các lượng tử này sau đó được phân bổ vào bốn pha bao gồm: pha đồng bộ (synchronization), pha lan truyền (propagation), phân đoạn pha 1 (phase segment 1) và phân đoạn pha 2 (phase segment 2). Số lượng lượng tử thường là một tham số có thể lập

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

trình được, và số lượng lượng tử được gán cho mỗi phân đoạn có thể biến thiên tùy thuộc vào tốc độ bit và các điều kiện vật lý của mạng.



Hình 2.2: Cấu trúc định thời Bit và Lượng tử thời gian (Time Quanta) [2]

Khi một sự chuyển đổi tín hiệu diễn ra sớm hơn hoặc trễ hơn so với dự kiến, bộ điều khiển sẽ tính toán độ lệch thời gian (time difference) và tiến hành kéo dài phân đoạn pha 1 hoặc rút ngắn phân đoạn pha 2 một lượng thời gian tương ứng. Cơ chế này thực hiện việc hiệu chỉnh thời gian của bộ thu theo bộ phát nhằm duy trì sự đồng bộ giữa chúng. Quá trình tái đồng bộ này được thực thi liên tục tại mọi điểm chuyển đổi từ Lặn sang Trội để đảm bảo bộ phát và bộ thu luôn đồng bộ pha (stay in sync).

Nằm giữa phân đoạn pha 1 và phân đoạn pha 2 là Điểm lấy mẫu (Sample Point), thời điểm mang tính tới hạn mà bộ thu phát CAN thực sự tiến hành đọc trạng thái logic của bit. Việc liên tục tái đồng bộ giúp giảm thiểu đáng kể các sai số do nhiễu gây ra, đồng thời cho phép một nút nhận (từng được đồng bộ với một nút vừa bị mất quyền phân xử) có thể tái đồng bộ mượt mà với nút mạng đã giành chiến thắng trong quá trình phân xử.

Thông thường, khi thiết lập tốc độ bit của mạng CAN, kỹ sư hệ thống có thể chọn phương án chỉ định tốc độ bit danh định và ủy thác cho bộ điều khiển CAN tự quản lý tiến trình đồng bộ hóa, hoặc tiến hành chỉ định các giá trị chính xác cho toàn bộ bốn phân đoạn của chu kỳ đồng bộ.

2.1.3 Cấu trúc và định dạng khung truyền

2.1.3.1 Khái niệm Giao tiếp Hướng Thông điệp và Cơ chế Phân xử

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

Khác với các giao thức định tuyến dựa trên địa chỉ nút mạng (node addressing) như I2C, mạng CAN sử dụng phương pháp giao tiếp hướng thông điệp (message-centric communication). Trong kiến trúc này, bản thân thông điệp mang một Bộ định danh (Identifier - ID) duy nhất, có chức năng xác định nội dung dữ liệu và mức độ ưu tiên của gói tin trên toàn mạng. Bộ định danh này được đánh giá thông qua một cơ chế gọi là Phân xử theo bit không phá hủy (Non-destructive bit-wise arbitration).[3] [6]

Khi đường truyền ở trạng thái rảnh (bus idle), nhiều nút mạng có thể đồng thời bắt đầu truyền dữ liệu. Trong quá trình truyền Bộ định danh, mỗi nút sẽ liên tục so sánh trạng thái logic mà nó phát ra với trạng thái điện áp thực tế đo được trên đường truyền. Theo nguyên lý vật lý của mạng CAN, trạng thái Trội (0) luôn áp đảo trạng thái Lặn (1). Do đó, nếu một nút đang phát bit Lặn nhưng lại phát hiện đường truyền bị kéo xuống mức Trội, nút đó ngay lập tức nhận ra có một nút khác với mức độ ưu tiên cao hơn đang truyền dữ liệu. Nút thua cuộc sẽ tự động dừng việc phát tín hiệu, chuyển sang chế độ nhận (receive mode) và chờ đến chu kỳ rảnh tiếp theo của bus. Khung truyền mang ID có giá trị số học nhỏ hơn (chứa nhiều bit 0 - mức Trội hơn ở các vị trí đầu tiên) sẽ luôn giành chiến thắng trong quá trình phân xử.

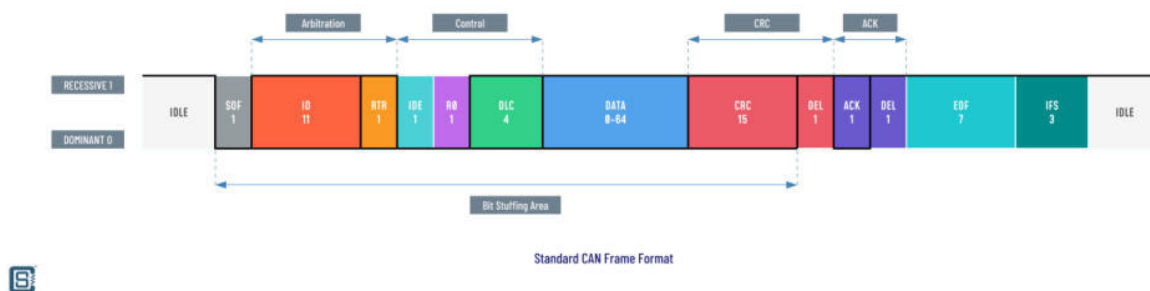
	Start Bit	ID Bits												Phần còn lại của Frame
		10	9	8	7	6	5	4	3	2	1	0		
NODE 15	0	0	0	0	0	0	0	0	1	1	1	1		
NODE 16	0	0	0	0	0	0	0	1	Dừng truyền					
DATA	0	0	0	0	0	0	0	0	1	1	1	1		

Bảng 2.4 : Ví dụ về cơ chế phân xử quyền ưu tiên trên Bus CAN [3] [6]

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1.3.2 Khung CAN Tiêu chuẩn (11-bit)

Khung CAN Tiêu chuẩn, hay còn gọi là Base Frame Format (theo chuẩn CAN 2.0A), sử dụng Bộ định danh có độ dài 11 bit. Một khung truyền dữ liệu hoàn chỉnh được cấu thành từ các trường (fields) cơ bản sau, truyền theo thứ tự nối tiếp



Hình 2.3: Cấu trúc các trường dữ liệu trong Khung CAN Tiêu chuẩn (11-bit) [6]

Tên Trường (Field name)	Độ dài	Mục đích
SOF (Start of Frame)	1 Bit	Bit bắt đầu khung truyền, luôn mang giá trị Trội (0). Bit này phá vỡ trạng thái nghỉ của mạng, báo hiệu cho tất cả các nút bắt đầu quá trình đồng bộ hóa cứng (hard synchronization).
Identifier	11 Bits	Bộ định danh tiêu chuẩn, xác định nội dung bản tin và mức độ ưu tiên phân xử. ID được truyền theo định dạng MSB
RTR (Remote Transmission Request)	1 Bit	Cờ yêu cầu truyền từ xa. Bit này mang giá trị Trội (0) đối với Khung dữ liệu (Data Frame) và giá trị Lặn (1) đối với Khung yêu cầu từ xa (Remote Frame - dùng

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

		để yêu cầu một nút khác gửi dữ liệu).
IDE (Identifier Extension)	1 Bit	Cờ mở rộng định danh. Trong khung tiêu chuẩn, bit này mang giá trị Trội (0) để khẳng định đây là khung 11-bit.
DLC	4 Bits	Mã chiều dài dữ liệu, chỉ định số lượng byte (từ 0 đến 8 byte) có trong trường dữ liệu tiếp theo.
Data Field	0 - 8 Bytes	Trường dữ liệu mang tải trọng hữu ích (payload), độ dài phụ thuộc vào giá trị của DLC.
CRC (Cyclic Redundancy Check)	15 Bits	Chuỗi kiểm tra vòng dự phòng. Bộ phát tính toán mã đa thức này dựa trên toàn bộ luồng bit trước đó để đảm bảo tính toàn vẹn của dữ liệu. Đi kèm sau đó là CRC Delimiter (1 bit Lặn) tạo khoảng đệm.
ACK (Acknowledgement)	1 Bit	Khe xác nhận. Bộ phát gửi một bit Lặn (1) ở vị trí này. Bất kỳ bộ thu nào trên mạng nhận và xác minh mã CRC hợp lệ sẽ tự động ghi đè khe ACK này bằng một bit Trội (0) để phản hồi lại cho bộ phát. Đi kèm sau đó là ACK Delimiter (1 bit Lặn).

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

EOF (End of Frame)	7 Bits	Trường kết thúc khung truyền, bao gồm 7 bit Lặn (1) liên tiếp, báo hiệu việc truyền dữ liệu đã hoàn tất.
IFS (Inter-Frame Space)	Tối thiểu 3 Bits	Khoảng trống liên khung, gồm ít nhất 3 bit Lặn để tạo thời gian cho các bộ điều khiển nội bộ xử lý thông điệp trước khi sẵn sàng cho khung truyền mới.

Bảng 2.5: Bảng cấu trúc các trường dữ liệu trong Khung CAN Tiêu chuẩn (11-bit) [3] [6]

Nhằm duy trì đủ số lượng các cạnh chuyển đổi tín hiệu (signal transitions) phục vụ cho quá trình tái đồng bộ nội bộ, giao thức CAN áp dụng quy tắc Nhồi bit ở Lớp Liên kết dữ liệu. Quy tắc quy định: Bất cứ khi nào có 5 bit liên tiếp mang cùng một mức logic (dù là Trội hay Lặn) được truyền đi, bộ điều khiển phát sẽ tự động chèn (nhồi) thêm một bit có mức logic ngược lại vào luồng dữ liệu.

Tại phía nhận, bộ điều khiển thu sẽ tiến hành giải nhồi (de-stuffing) bằng cách loại bỏ bit nghịch đảo này để khôi phục luồng dữ liệu gốc. Cần lưu ý, cơ chế nhồi bit chỉ áp dụng từ trường SOF cho đến hết trường CRC; các trường cố định định dạng (Fixed-form fields) như CRC Delimiter, ACK, và EOF không chịu sự chi phối của luật nhồi bit.



Hình 2.4 : Bit stuff trong CAN [6]

2.1.4 Cơ chế cô lập lỗi và các bộ đếm sai số (TEC & REC)

Trong giao thức CAN, độ tin cậy của hệ thống không chỉ nằm ở việc phát hiện lỗi mà còn ở khả năng tự chẩn đoán để xác định một nút mạng đang gặp nhiều nhất thời hay đã bị hỏng hóc phân cứng vĩnh viễn. Cơ chế này được gọi là Cô lập lỗi (Fault Confinement), vận hành dựa trên hai bộ đếm nội bộ: Bộ đếm lỗi truyền (Transmit Error Counter - TEC) và Bộ đếm lỗi nhận (Receive Error Counter - REC).[3]

2.1.4.1 Các trạng thái hoạt động dựa trên chỉ số lỗi

Tùy thuộc vào giá trị tích lũy trong TEC và REC, một nút mạng sẽ tự động chuyển đổi giữa ba trạng thái với các đặc quyền truyền dẫn khác nhau:

- Lỗi Chủ động (Error Active): Trạng thái mặc định ($TEC \leq 127$ và $REC \leq 127$). Nút được quyền tham gia giao tiếp bình thường và phát ra Cờ lỗi Chủ động (6 bit Trội) để thông báo lỗi cho toàn mạng.
- Lỗi Thụ động (Error Passive): Kích hoạt khi một trong hai bộ đếm vượt ngưỡng 127. Nút bị hạn chế quyền hạn, chỉ được phép phát Cờ lỗi Thụ động (6 bit Lặn), không làm gián đoạn các thông điệp khác. Nút này cũng phải chờ thêm khoảng thời gian Suspend Transmission (8 bit Lặn) trước khi thử truyền lại.

Ngắt kết nối (Bus-Off): Kích hoạt khi TEC vượt ngưỡng 255. Nút bị đánh giá là hỏng nặng và bị cưỡng bức ngắt kết nối vật lý khỏi bus để tránh gây nhiễu cho hệ thống.

2.1.4.2 Nguyên nhân gây tăng bộ đếm

Bộ đếm sẽ tăng điểm dựa trên việc phát hiện một trong năm lỗi sau đây:

- Lỗi Bit (Bit Error): Nút đọc lại mức logic khác với mức nó đang phát.
- Lỗi Nhồi (Stuff Error): Phát hiện 6 bit cùng mức logic liên tiếp trong vùng nhồi bit.
- Lỗi CRC (CRC Error): Mã kiểm tra tính toán lại không khớp với mã nhận được.
- Lỗi Định dạng (Form Error): Vi phạm cấu trúc các trường cố định (EOF, ACK Delimiter).

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

- Lỗi Xác nhận (ACK Error): Nút phát không nhận được bit Trội từ bất kỳ nút thu nào trong khe ACK.

Sự kiện	TEC	REC
Bộ phát kích hoạt Cờ lỗi Chủ động sơ cấp (Primary Active Error Flag) và thứ cấp (Secondary Active Error Flag)	+8	Không thay đổi
Bộ thu kích hoạt Cờ lỗi Chủ động sơ cấp	Không thay đổi	+8
Bộ thu kích hoạt Cờ lỗi Chủ động thứ cấp	Không thay đổi	+1
Bộ phát truyền thành công một thông điệp	+1	Không thay đổi
Bộ thu nhận thành công một thông điệp	Không thay đổi	-1

Bảng 2.6 : Bảng sự thay đổi của REC và TEC theo sự kiện [1]

2.1.4.3 Khung lỗi Chủ động (Active Error Frame)

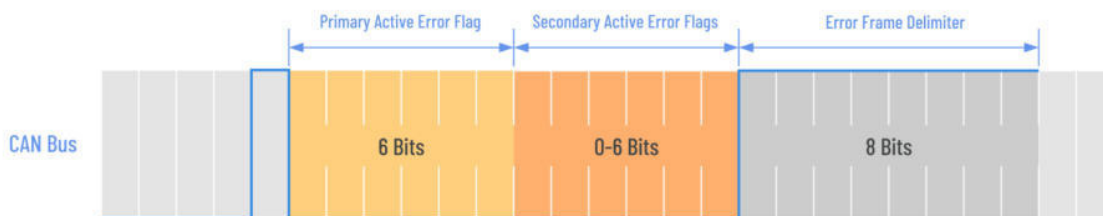
Khi một thiết bị đang ở trạng thái Lỗi Chủ động phát hiện ra sự cố (ví dụ: Lỗi Bit), nó sẽ lập tức phát ra một Khung lỗi Chủ động.

Khung này bắt đầu bằng một Cờ lỗi (Error Flag) gồm 6 bit Trội (0) liên tiếp, theo sau là Khoảng đệm lỗi (Error Delimiter) gồm 8 bit Lặn (1). Hành động phát ra 6 bit Trội là một sự cố tình vi phạm quy tắc nhồi bit. Khối tín hiệu Trội này mạnh đến mức nó lập tức ghi đè lên bất kỳ dữ liệu nào đang tồn tại trên bus.

Hệ quả là toàn bộ các thiết bị còn lại trên mạng cũng sẽ đọc được dải 6 bit Trội này, đánh giá đó là một Lỗi Nhồi bit (Stuff Error) và tự động phát ra Cờ lỗi Chủ động của riêng chúng. Cơ chế lây lan này tạo ra hiện tượng chồng lấp tín hiệu (Error Flag Overlap), kéo dài dải bit Trội trên bus từ 6 đến tối đa 12 bit. Quá trình này đảm bảo 100% các nút trên

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

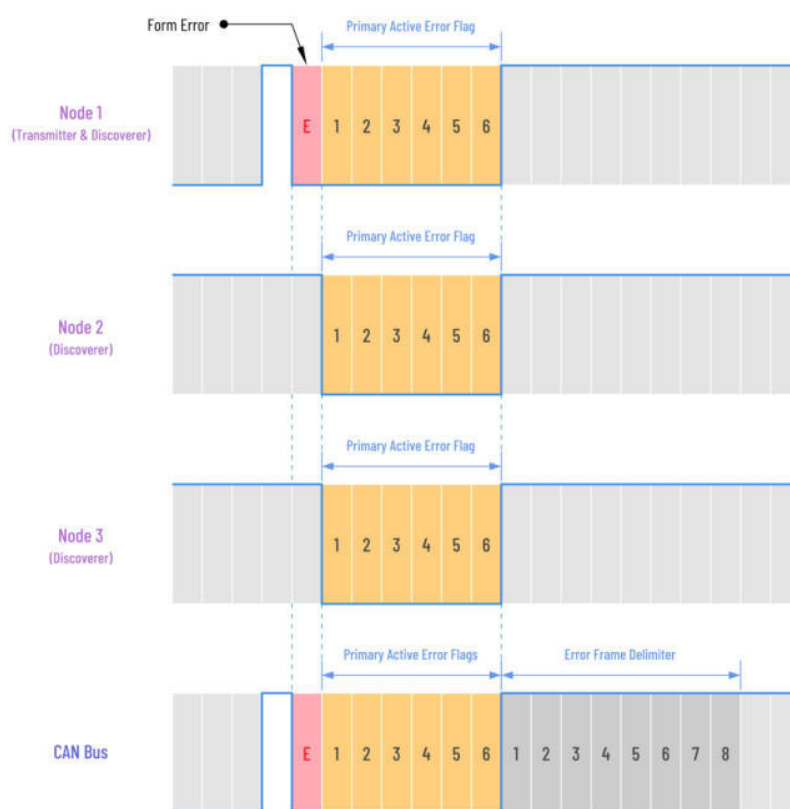
mạng đều nhận thức được gói tin đã hỏng và tiến hành hủy bỏ nó một cách đồng bộ, trước khi đường truyền bước vào giai đoạn 8 bit Lặn để xả bus.



CAN Error Frame



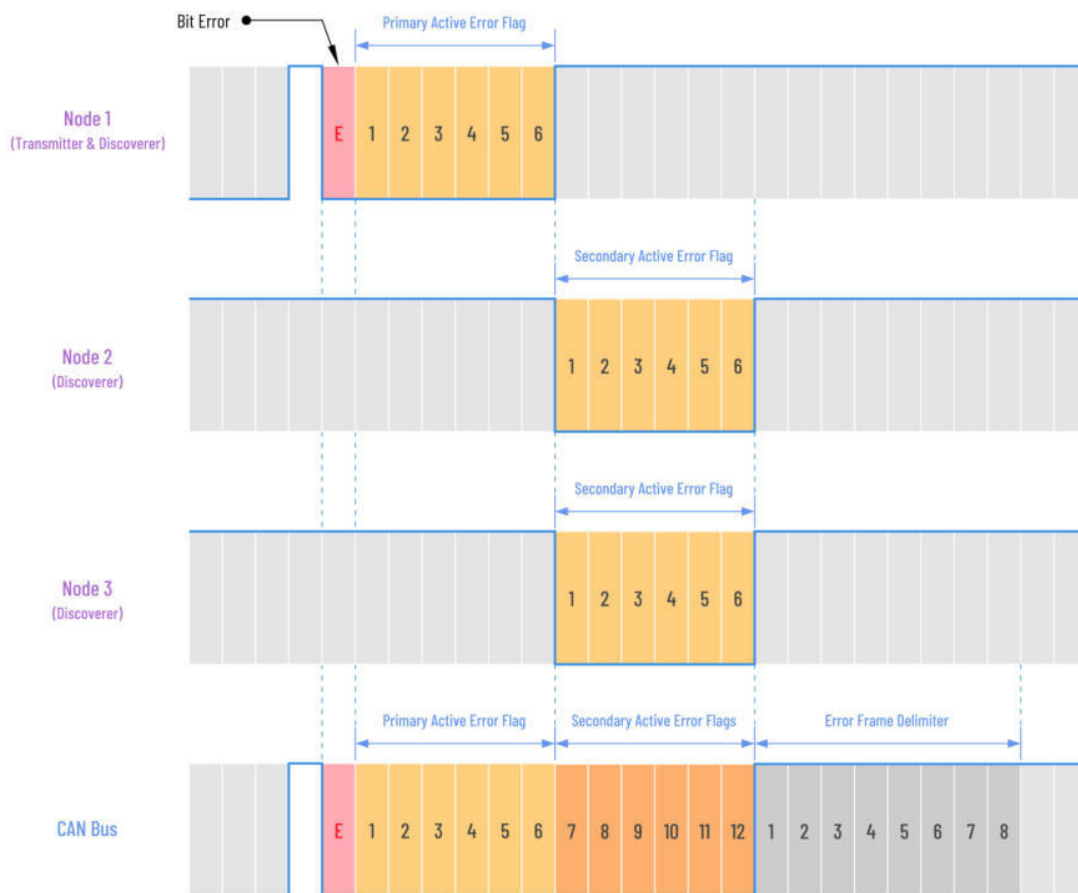
Hình 2.5: Khung lỗi Chủ động [6]



Form Error and Primary Active Error Frames on a CAN bus



Hình 2.6: Cờ lỗi chủ động được gửi bởi tất cả các node [6]



Primary and Secondary Active Error Frames on a CAN bus after a Bit Error



Hình 2.7 : Cờ lỗi chủ động được gửi bởi 1 node [6]

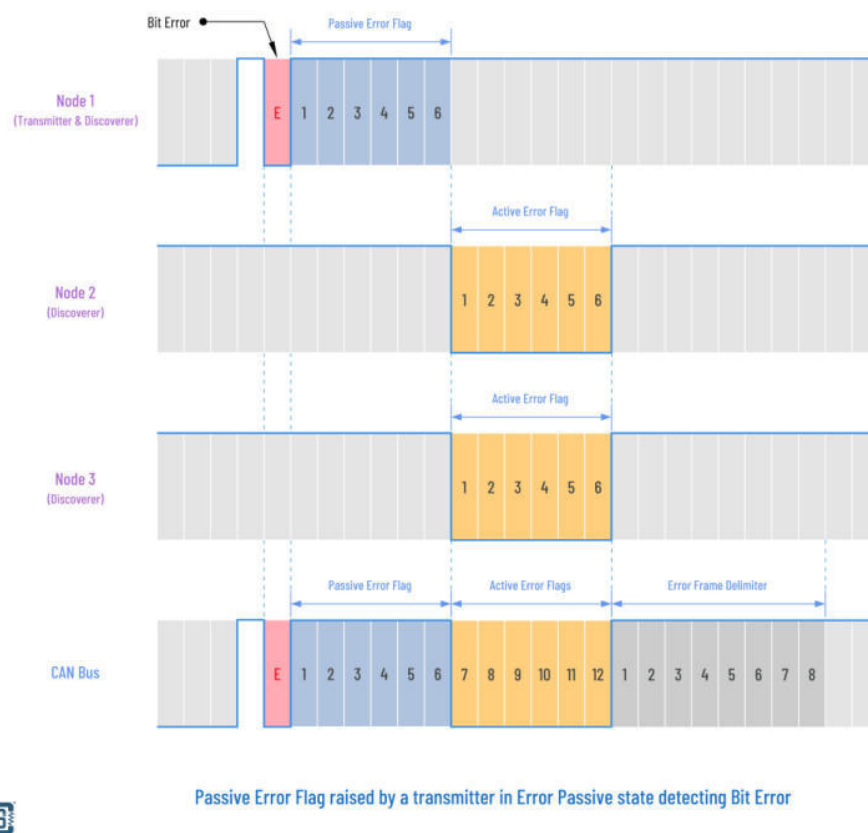
2.1.4.4 Khung lỗi Thụ động (Passive Error Frame)

Nếu một nút mạng đã bị đánh tụt xuống trạng thái Lỗi Thụ động (do TEC hoặc REC > 127), nó vẫn có thể truyền và nhận dữ liệu. Tuy nhiên, nếu phát hiện sự cố, nó chỉ được phép phát ra một Khung lỗi Thụ động.

Khung lỗi này được cấu thành từ một Cờ lỗi gồm 6 bit Lặn (1) liên tiếp, theo sau là 8 bit Lặn của Khoảng đệm lỗi. Vì mang mức điện áp vi sai bằng 0 (đặc tính Lặn), các bit này hoàn toàn "vô hình" trên đường truyền nếu các nút mạng khỏe mạnh khác đang phát bit Trội. Cơ chế này cho phép thiết bị lỗi tự ghi nhận sự cố nội bộ mà không làm xáo trộn đường truyền hay phá hủy các thông điệp hợp lệ.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

Ngoài ra, giao thức còn áp dụng một hình phạt bổ sung về thời gian: Một nút ở trạng thái Lỗi Thụ động, sau khi truyền xong một khung dữ liệu, bắt buộc phải chờ thêm một khoảng thời gian gọi là Đình chỉ phát (Suspend Transmission - 8 bit Lặn) trước khoảng trống liên khung (IFS). Sự trì hoãn này khiến thiết bị lỗi luôn chậm nhịp hơn các thiết bị Active, làm giảm đáng kể khả năng giành chiến thắng của nó trong quá trình phân xử quyền ưu tiên ở các gói tin tiếp theo.



Hình 2.8: Minh họa cho cờ lỗi thụ động [6]

2.1.4.5 Hiện tượng lây lan và Giới hạn an toàn

Giống như cơ chế của Cờ lỗi, việc một nút tung ra 6 bit Trội của Cờ Quá tải sẽ làm thay đổi trạng thái đường truyền. Khi các nút mạng khác đang đọc phân đoạn Nghỉ (mong đợi bit Lặn) nhưng lại thấy bit Trội, chúng sẽ phản ứng bằng cách đồng loạt phát ra Cờ Quá tải của riêng mình.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

Điều này tạo ra hiện tượng Chồng lấp Cờ Quá tải (Overload Flag Overlap), kéo dài dải bit Trội trên bus lên tối đa 12 bit. Sau khi tất cả các thiết bị đã phát xong cờ của mình, chúng sẽ chuyển sang trạng thái giám sát. Khi phát hiện đường truyền chuyển đổi từ Trội sang Lặn ($0 \rightarrow 1$), toàn bộ mạng lưới sẽ cùng lúc xuất ra 7 bit Lặn của phần Khoảng đệm (Delimiter) để kết thúc Khung Quá tải, và sau đó thiết lập lại một chu kỳ IFS mới.

Nhằm ngăn chặn kịch bản một nút bị lỗi phần cứng liên tục phát Khung Quá tải làm "đóng băng" toàn bộ băng thông của mạng, tiêu chuẩn CAN giới hạn rất nghiêm ngặt: Mỗi nút mạng chỉ được phép khởi tạo tối đa 2 Khung Quá tải liên tiếp để trì hoãn một khung dữ liệu. Vượt quá giới hạn này, thiết bị buộc phải giải phóng đường truyền.

2.1.5 Kiến trúc phần cứng của một nút mạng CAN

Một nút mạng CAN (CAN Node) không chỉ đơn thuần là một cảm biến hay cơ cấu chấp hành, mà là một thực thể xử lý thông tin hoàn chỉnh. Để có thể tham gia vào giao thức truyền thông phức tạp này, mỗi nút mạng bắt buộc phải được cấu thành từ ba khối chức năng cơ bản, hoạt động nối tiếp nhau từ Lớp Ứng dụng (Application Layer) xuống Lớp Vật lý (Physical Layer): Vi xử lý chủ (Host Processor/MCU), Bộ điều khiển CAN (CAN Controller) và Bộ thu phát CAN (CAN Transceiver). [4] [5]

2.1.5.1 Vi xử lý (Microcontroller Unit - MCU)

Đây là "bộ não" của nút mạng, chịu trách nhiệm thực thi các thuật toán của ứng dụng cụ thể (ví dụ: bộ điều khiển động cơ ECU, cảm biến nhiệt độ, hoặc bảng điều khiển trung tâm).

- Vai trò khi Truyền: MCU quyết định thông tin nào cần được gửi đi (Payload/Data) và mức độ ưu tiên của nó (Identifier). Nó sẽ truyền các thông số này xuống Bộ điều khiển CAN.
- Vai trò khi Nhận: MCU nhận dữ liệu đã được giải mã từ Bộ điều khiển CAN, sau đó phân tích và thực thi các lệnh điều khiển phần cứng tương ứng. MCU hoàn toàn

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

không "hiểu" về cấu trúc khung CAN hay cơ chế nhồi bit; nó chỉ quan tâm đến dữ liệu tinh gọn mang ý nghĩa thực tế. [4] [5]

2.1.5.2 Bộ điều khiển CAN

Thường được tích hợp sẵn dưới dạng một khối ngoại vi (peripheral) bên trong các dòng vi điều khiển hiện đại, Bộ điều khiển CAN đóng vai trò là "biên dịch viên" chuyên trách Lớp Liên kết dữ liệu (Data Link Layer).

Quá trình Đóng gói (Framing): Khi nhận dữ liệu từ MCU, Bộ điều khiển CAN sẽ gắn thêm các trường SOF, ID, RTR, IDE, DLC, tự động tính toán mã kiểm tra CRC và thêm trường EOF để tạo thành một Khung truyền (Frame) hoàn chỉnh.

Quản lý Giao thức: Đây là nơi thực thi toàn bộ các cơ chế phức tạp mà chúng ta đã phân tích ở các phần trước: Nhồi bit (Bit Stuffing), Phân xử quyền ưu tiên (Arbitration), Phát hiện lỗi đa tầng, và quản lý các bộ đếm lỗi TEC/REC.

Giao tiếp: Giao tiếp giữa CAN Controller và CAN Transceiver được thực hiện qua hai đường tín hiệu số (Digital Logic): Tx (Transmit - Truyền) và Rx (Receive - Nhận) với các mức logic chuẩn (ví dụ: 0V và 3.3V/5V). [4] [6]

2.1.5.3 Bộ thu phát CAN (CAN Transceiver)

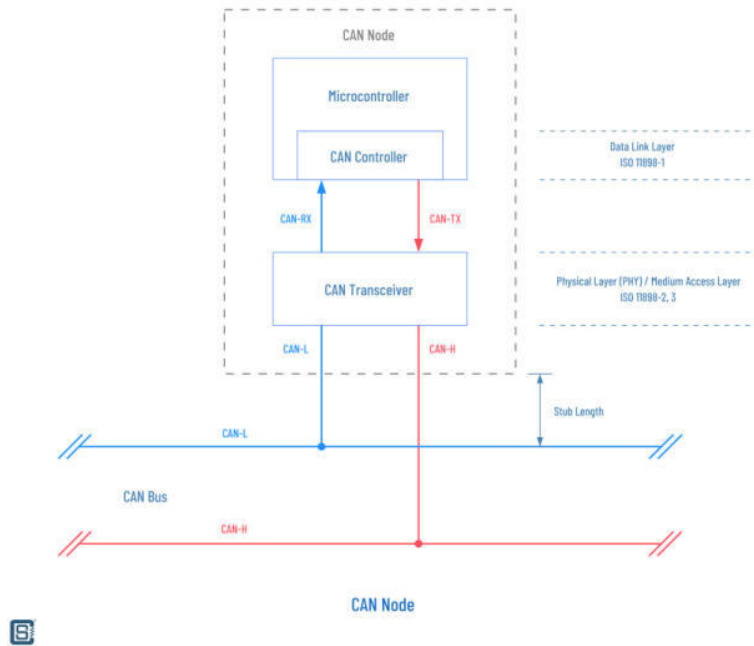
Đây là cầu nối cuối cùng giữa mạch điện tử logic và môi trường dây dẫn bên ngoài, đảm nhiệm chức năng của Lớp Vật lý (Physical Layer). Các dòng IC tiêu biểu có thể kể đến như TJA1050, MCP2551.

Chuyển đổi tín hiệu vi sai (Differential Signaling): Mạng CAN không truyền tín hiệu số (0-5V) trực tiếp qua dây dẫn vì chúng rất dễ bị nhiễu. Thay vào đó, CAN Transceiver nhận tín hiệu logic từ chân Tx của CAN Controller và chuyển đổi chúng thành mức điện áp vi sai trên hai sợi cáp CAN_H (CAN High) và CAN_L (CAN Low).

Khả năng tự đọc (Self-monitoring): Bộ thu phát liên tục đo điện áp vi sai trên cáp CAN_H/CAN_L, dịch ngược lại thành tín hiệu logic số và gửi về chân Rx của CAN Controller. Nhờ vậy, bộ điều khiển mới có thể phát hiện Lỗi Bit hoặc thực hiện phân xử.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

Bảo vệ phần cứng: Transceiver cung cấp khả năng cách ly điện học (tùy IC) và bảo vệ các linh kiện logic mỏng manh bên trong khỏi các xung điện áp cao, quá dòng hoặc hiện tượng chập mạch trên cáp mạng vật lý.



Hình 2.9 : Sơ đồ khối CAN Node[6]

2.2 Các chuẩn giao tiếp khác

2.2.1 Giao thức SPI

Giao tiếp SPI (Serial Peripheral Interface) phân loại thuộc nhóm giao thức truyền thông đồng bộ, vận hành theo phương thức toàn song công (full-duplex). Đặc tính này cho phép quá trình phát và nhận dữ liệu giữa các thực thể trong mạng được thực hiện một cách đồng thời tại cùng một thời điểm kích hoạt. Cấu trúc phần cứng tiêu chuẩn của giao thức này yêu cầu thiết lập 4 đường tín hiệu độc lập, đảm nhiệm các vai trò phối hợp chuyên biệt:

- SCK (Serial Clock): Tuyến phát tín hiệu xung định thời toàn hệ thống, được khởi tạo và điều phối duy nhất bởi thiết bị Master. Do SPI tuân thủ nguyên lý truyền thông đồng bộ, sự dịch chuyển của mỗi bit dữ liệu (đến hoặc đi) đều được ràng buộc và chuẩn hóa dựa trên từng chu kỳ nhịp trên chân SCK này.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

- SS (Slave Select): Đường tín hiệu tích cực mức thấp dùng để định tuyến và lựa chọn thiết bị Slave mục tiêu. Ở trạng thái tĩnh (không làm việc), đường SS được duy trì ở mức điện áp cao. Tiến trình thiết lập giao tiếp chỉ được cấu hình thành công khi Master chủ động kéo điện áp của chân SS tương ứng xuống mức thấp.
- MOSI (Master Output / Slave Input): Kênh truyền tải dữ liệu đơn hướng có sơ đồ kết nối trực tiếp giữa các nút mạng. Đường truyền này đóng vai trò là ngõ ra đối với bộ điều khiển Master và chuyển đổi thành ngõ vào khi tiếp cận các nút cấu hình Slave.
- MISO (Master Input / Slave Output): Tuyến phản hồi dữ liệu có cấu trúc định tuyến ngược lại với MOSI. Tín hiệu dịch chuyển trên đường này xuất phát từ ngõ ra của thiết bị Slave và được tiếp nhận trực tiếp tại ngõ vào của thiết bị Master.[7]

2.2.2 Giao thức I2C

I2C (Inter-Integrated Circuit) đóng vai trò là cấu trúc bus liên kết dữ liệu giữa các vi mạch tích hợp (IC) khác nhau trong hệ thống, các đặc điểm chính của giao thức I2C:

- Mạng I2C tối giản chỉ yêu cầu hai đường truyền tín hiệu là SDA (Serial Data - đường truyền dữ liệu hai chiều) và SCL (Serial Clock - tuyến cấp xung định thời đơn hướng).
- Tuyến cấp xung định thời SCL đóng vai trò quan trọng trong việc giữ cho toàn bộ hệ thống luôn hoạt động đồng bộ với nhau.
- Mọi thiết bị (nút mạng) khi tham gia vào bus I2C đều phải thiết lập liên kết vật lý bằng cách nối các chân chức năng SDA và SCL của mình vào hai đường trục chính này.

Do các chân giao tiếp I2C có cấu hình ngõ ra dạng cực máng hở (open-drain) hoặc cực thu hở (open-collector), việc sử dụng các điện trở kéo lên (pull-up resistors) kết nối với nguồn điện áp dương là bắt buộc để duy trì trạng thái logic cho đường truyền, giá trị của các điện trở kéo này thường dao động trong khoảng từ $1k\Omega$ đến $4.7k\Omega$. [8]

Hệ thống truyền tải dữ liệu qua bus I2C vận hành linh hoạt dưới ba phân cấp tốc độ: Standard mode, Fast mode và High-Speed mode. Về mặt định thời, mỗi chu kỳ xung SCL sẽ truyền tải một bit dữ liệu, mức logic trên đường SDA chỉ được phép thay đổi khi SCL ở mức thấp và phải giữ ổn định tuyệt đối khi SCL ở mức cao.[9]

2.3 Công cụ hỗ trợ xây dựng và lập trình hệ thống

Visual Studio Code: Visual Studio Code (VS Code) là một trình chỉnh sửa mã nguồn miễn phí, hỗ trợ đa hệ điều hành, do Microsoft phát triển. Nhờ vào giao diện trực quan, nhiều tính năng lập trình mạnh mẽ, cùng với khả năng mở rộng linh hoạt thông qua các tiện ích, VS Code đã nhanh chóng trở thành một trong những công cụ được lập trình viên ưa chuộng và sử dụng rộng rãi trên toàn thế giới. [10]

EasyEDA: EasyEDA là một công cụ thiết kế mạch điện tử (EDA) hoàn toàn miễn phí, hoạt động trực tiếp trên nền tảng đám mây mà không yêu cầu người dùng phải cài đặt phần mềm trên máy tính cá nhân. Ứng dụng này được phát triển nhằm hỗ trợ kỹ sư điện, giảng viên, sinh viên ngành kỹ thuật, cũng như những người có niềm đam mê với điện tử, giúp họ thực hiện quá trình thiết kế mạch một cách nhanh chóng và tiện lợi hơn. [11]

STM32CubeIDE: STM32CubeIDE là một môi trường phát triển tích hợp (IDE) C/C++ nâng cao hoàn toàn miễn phí do hãng STMicroelectronics phát hành, được thiết kế chuyên biệt cho các dòng vi điều khiển và vi xử lý STM32. Phần mềm này được phát triển nhằm hỗ trợ các kỹ sư hệ thống nhúng, sinh viên ngành kỹ thuật cũng như các nhà nghiên cứu thực hiện toàn bộ quy trình phát triển phần mềm từ lập trình, biên dịch cho đến gỡ lỗi (debug) một cách đồng bộ và chuyên nghiệp. [12]

Figma: Figma là một công cụ thiết kế giao diện (UI/UX) nâng cao hoạt động chủ yếu trên nền tảng đám mây, được thiết kế chuyên biệt cho việc xây dựng giao diện và trải nghiệm người dùng. Phần mềm này được phát triển nhằm hỗ trợ các nhà thiết kế (designer), kỹ sư front-end cũng như các quản lý sản phẩm thực hiện toàn bộ quy trình từ lên ý tưởng, thiết kế đồ họa vector cho đến tạo nguyên mẫu tương tác (prototype) một cách đồng bộ và chuyên nghiệp.[13]

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1 Yêu cầu và sơ đồ khối hệ thống

3.1.1 Yêu cầu về hệ thống

Để bảo đảm tính ổn định và chính xác trong vận hành của mô hình Cụm động hồ số (IPC), cấu trúc hệ thống được chuẩn hóa dựa trên các nhóm chức năng cốt lõi sau đây:

Cơ chế mô phỏng động lực học (Node 1): Tác vụ tái tạo tuyến tính các thông số vận tốc xe (Speed), tốc độ vòng tua động cơ (RPM), đồng thời tích lũy và tính toán hành trình (Trip/Odometer) do Node 1 đảm nhiệm. Về hạ tầng phần cứng, bộ mã hóa vòng quay gia số Rotary Encoder EC11 được tích hợp cùng vi điều khiển STM32F103 nhằm chuyển đổi và số hóa các tương tác cơ khí từ người lái.

Cơ chế mô phỏng hệ thống năng lượng và điều khiển vận hành (Node 2): Trạng thái dung lượng Pin (SoC), các dải vị trí cấp số và chế độ lái thích ứng được giả lập độc lập bởi Node 2. Tín hiệu analog biến thiên đầu vào của khối này được tạo lập thông qua mạng lưới các biến trở xoay (Potentiometers) cấu hình theo nguyên lý bộ chia áp, kết nối trực tiếp với mạch ADC của STM32F103.

Cơ chế giả lập tín hiệu thân xe và điều kiện ngoại cảnh (Node 3): Node 3 chịu trách nhiệm cung cấp các lệnh chỉ thị báo rẽ (Turn Signals), thu thập dữ liệu nhiệt độ môi trường và chuẩn hóa thời gian thực. Khối chức năng này vận hành dựa trên sự phối hợp giữa vi điều khiển STM32F103, hệ thống công tắc gạt cơ học, cảm biến áp suất/nhiệt độ BMP180 và module thời gian thực RTC DS1307.

Giao tiếp truyền thông liên nút: Quá trình trao đổi dữ liệu định kỳ giữa các Node thành phần và bộ điều khiển trung tâm được thực hiện thông qua giao thức mạng CAN Bus. Để chuyển đổi mức logic và bảo vệ điện áp cho các nút mạng vật lý, module điều khiển độc lập MCP2515 được lựa chọn cấu hình.

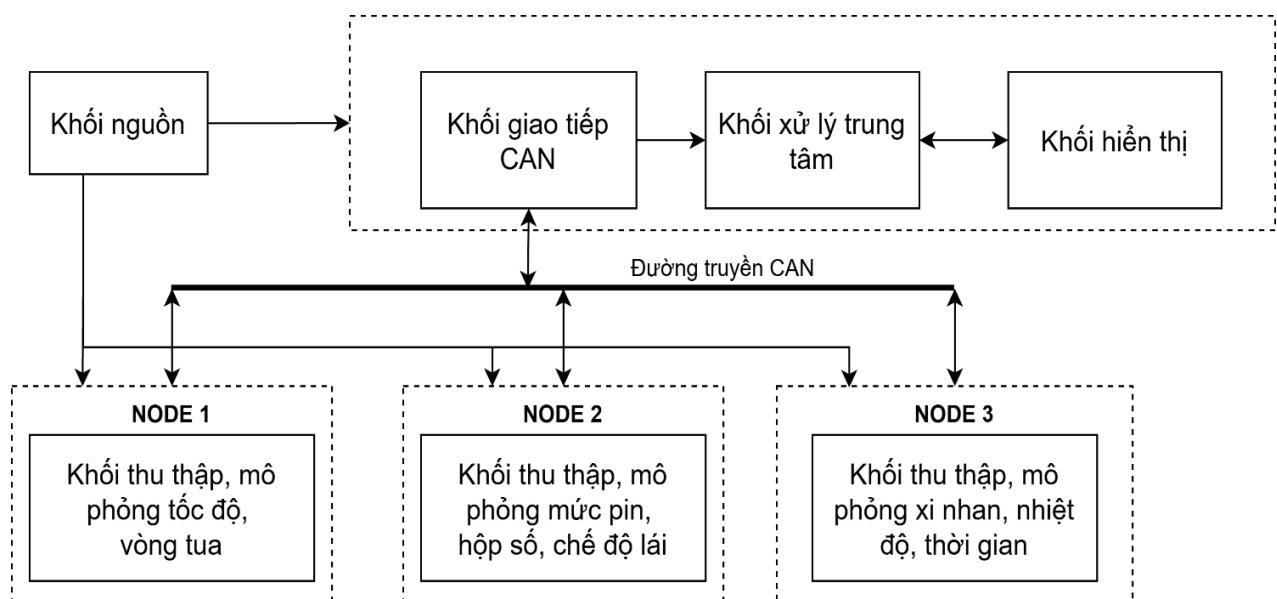
Giao diện tương tác người-máy và điều phối trung tâm (HMI Block): Bộ xử lý trung tâm (VCU) phối hợp cùng Khối hiển thị (Output Block) thực thi các tác vụ quản lý máy

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

trạng thái xe, giải mã các khung tin mạng (CAN frames) và kết xuất đồ họa 2D/3D. Máy tính nhúng Raspberry Pi 5 và màn hình LCD Waveshare 7 inch độ phân giải cao được sử dụng để trình diễn thông tin trực quan theo thời gian thực.

3.1.2 Sơ đồ khối toàn hệ thống

Để minh họa cấu trúc tổng thể của hệ thống và mối liên hệ giữa các khối chức năng, sơ đồ khối hệ thống được xây dựng như hình 3.1.



Hình 3.1 Sơ đồ khối hệ thống

Mô hình điều khiển phân tán được áp dụng xuyên suốt thiết kế nhằm phân tách nhiệm vụ thành các phân vùng độc lập, từ đó tối ưu hóa mật độ lưu thông của luồng dữ liệu bus mạng và nâng cao khả năng mở rộng phần cứng. Vai trò chi tiết của từng phân khối được quy định như sau:

- Bộ điều khiển trung tâm (VCU): Tiến trình quản lý máy trạng thái của phương tiện, thực thi thuật toán định lượng thông số vận hành và kiểm soát lớp giao diện đồ họa được xử lý tập trung tại đây. Khối này trực tiếp tiếp nhận và biên dịch dữ liệu từ bus mạng để cập nhật trạng thái HMI tức thời.

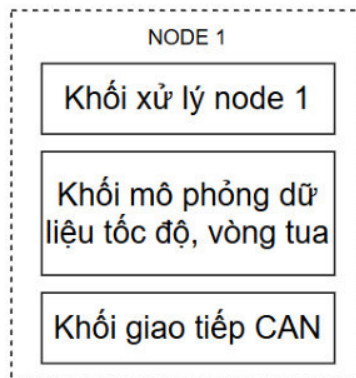
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

- Khối liên lạc CAN bus: Đóng vai trò là trục truyền thông chuyên dụng liên kết toàn mạch. Khối thực hiện nhiệm vụ đóng gói các biến số thành khung tin CAN tiêu chuẩn, đồng thời cách ly và chuyển đổi mức điện áp logic nhằm bảo toàn an toàn phần cứng cho bộ vi xử lý trung tâm.
- Khối kết xuất hiển thị: Thiết bị này tiếp nhận luồng dữ liệu đồ họa đã qua xử lý từ GPU để chỉ thị các thông số vận hành động (tốc độ, dung lượng năng lượng, hệ thống đèn cảnh báo an toàn) lên màn hình cho người điều khiển.
- Khối quản lý năng lượng (Khối Nguồn): Nhằm triệt tiêu rủi ro sụt áp cục bộ gây ảnh hưởng đến năng lực thực thi đồ họa của nhân VideoCore VII, Khối quản lý năng lượng được cấu hình để duy trì tính liên tục của dòng phụ tải. Giải pháp này không chỉ bảo toàn tính ổn định cho các nút chức năng mà còn đóng vai trò như một bộ đệm lọc, giảm thiểu xung nhiễu điện từ ký sinh trên đường truyền bus mạng chung
- Mạng lưới các Node giả lập dữ liệu: Hệ thống chia tách việc số hóa các hành vi lái xe và biến động môi trường thành 03 phân vùng chức năng biệt lập:
 - *Node 1*: Tính toán và phát các thông số động lực học cốt lõi liên quan đến hệ thống truyền động phương tiện.
 - *Node 2*: Giả lập dữ liệu dung lượng pin, vị trí tay số vật lý và các tùy biến chế độ vận hành từ người dùng.
 - *Node 3*: Thu thập thông tin nhiệt độ môi trường, duy trì nhịp thời gian thực và đồng bộ hóa tín hiệu đèn chỉ thị báo rẽ.

3.2 Thiết kế hệ thống

3.2.1 Khối xử lý dữ liệu Node 1

Trong cấu trúc tổng thể của đề tài, Node 1 đảm nhiệm vai trò cốt lõi là tái tạo chính xác sự thay đổi các thông số động lực học cơ bản của phương tiện bao gồm tốc độ xe (Speed), vòng tua máy (RPM) và tính toán quãng đường hành trình (Trip/Odometer). Khối này thực hiện nhiệm vụ chuyển hóa các tương tác cơ khí trực tiếp từ người lái thành các khung dữ liệu số có cấu trúc, sau đó truyền tải liên tục với độ trễ thấp lên mạng truyền thông để cụm đồng hồ trung tâm (IPC) xử lý và hiển thị trực quan.



Hình 3.2 Sơ đồ khối node 1

Dựa trên hình 3.2, cấu trúc phân cứng của Node 1 được phân tách thành ba khối chức năng chính bao gồm:

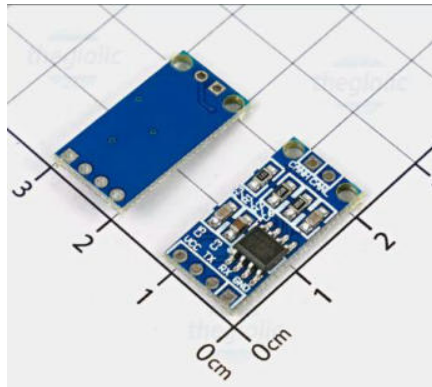
- Khối xử lý Node 1: Sử dụng vi điều khiển STM32F103 để tiếp nhận tín hiệu ngoại vi, tính toán thuật toán số hóa dữ liệu và điều phối chu kỳ đóng gói tin của hệ thống.
- Khối mô phỏng dữ liệu tốc độ, vòng tua: Sử dụng bộ mã hóa vòng quay (Rotary Encoder EC11) để chuyển đổi các thao tác xoay/nhấn cơ học từ người lái thành tín hiệu xung số gửi về vi điều khiển.
- Khối giao tiếp CAN: Sử dụng module TJA1050 để tiếp nhận dữ liệu từ STM32F103 qua giao thức SPI, đóng gói thành khung tin CAN tiêu chuẩn và truyền lên trục mạng Bus.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Khối giao tiếp CAN (TJA1050):

Vai trò của module TJA1050 như là bộ thu phát mạng CAN tốc độ cao, làm giao diện vật lý chuyển đổi tín hiệu logic (TX, RX) từ vi điều khiển (như STM32) thành tín hiệu vi sai (CAN_H, CAN_L) để giao tiếp trên CAN Bus.

Trong hệ thống IPC, module này giúp kết nối vi điều khiển trung tâm với mạng CAN (hoặc các node giả lập) để thu thập chính xác các thông số vận hành thời gian thực (vận tốc, vòng tua, nhiệt độ...), từ đó xử lý và hiển thị lên màn hình táp-lô



Hình 3.3 Module CANBus TJA 1050 [14]

Thông số kỹ thuật:

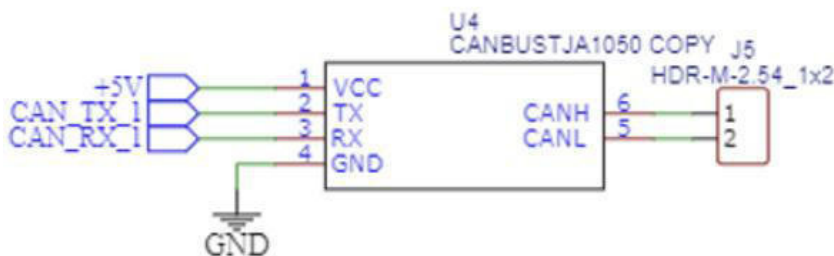
- IC chính: TJA1050 (NXP Semiconductors)
- Chuẩn giao tiếp: CAN Bus (Tương thích chuẩn ISO 11898)
- Tốc độ truyền tối đa: Lên đến 1 Mbps
- Điện áp hoạt động: 4.75V – 5.25V (Tiêu chuẩn là 5V)
- Dòng tối đa: 50mA (0.05A)
- Số lượng node tối đa trên bus: Hỗ trợ lên đến 110 node
- Tính năng bảo vệ: Tích hợp bảo vệ ngắn mạch trên đường dây CAN_H, CAN_L và bảo vệ quá tải nhiệt.
- Khả năng chống nhiễu: Bức xạ điện từ (EME) cực thấp và khả năng tương thích
- Có trở đầu cuối: 120Ω tích hợp sẵn trên module, phối hợp trở kháng đường truyền.[14]

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Chân	Chức năng
VCC	Nguồn cấp cho module (5V)
TX	Chân nhận tín hiệu truyền từ vi điều khiển (kết nối với chân CAN_TX của STM32/MCU)
RX	Chân xuất tín hiệu nhận về vi điều khiển (kết nối với chân CAN_RX của STM32/MCU)
GND	Mass của hệ thống
CANH	Đường tín hiệu CAN High (Kết nối vào mạng CAN Bus)
CANL	Đường tín hiệu CAN Low (Kết nối vào mạng CAN Bus)

Bảng 3.1 Chức năng từng chân của Module TJA1050 [14]

Để dữ liệu từ vi điều khiển có thể truyền đi xa và chống lại các tác động nhiễu trong môi trường điện từ, IC TJA1050 được sử dụng như một cầu nối vật lý.



Hình 3.4 Sơ đồ kết nối TJA1050

Giao tiếp vật lý lớp truyền dẫn mạng CAN trong phân khối IPC sử dụng IC TJA1050 do NXP Semiconductors sản xuất làm cầu nối vật lý. Thiết bị này đảm nhận vai trò chuyển dịch các mức logic TTL (0V - 3.3V) từ các đường tín hiệu TX, RX của vi điều khiển thành dạng điện áp vi sai chuẩn hóa (CAN_H, CAN_L) truyền tải trên trục bus mạng

Khối mô phỏng dữ liệu tốc độ, vòng tua:

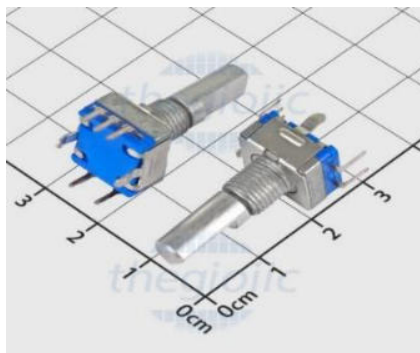
Khối sử dụng Rotary Encoder EC11 là thiết bị mã hóa vòng quay gia số, chuyển đổi chuyển động cơ học thành các xung điện kỹ thuật số nhằm xác định chiều quay và số bước quay.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Trong hệ thống bảng điều khiển IPC, núm xoay EC11 đóng vai trò là thiết bị tương tác người dùng (HMI) chính. Thiết bị này cho phép người lái dễ dàng cuộn để điều hướng qua lại giữa các menu thông số trên màn hình và nhấn trực tiếp vào trục để xác nhận lựa chọn một cách trực quan.

Thông số kỹ thuật:

- Loại linh kiện: Mã hóa vòng quay (Incremental Encoder) tích hợp nút nhấn.
- Số vị trí (Detents): 20 vị trí trên 1 vòng quay (20 xung/vòng).
- Chiều dài trục: 20mm (kiểu trục khuyết nửa chữ D).
- Phạm vi nguồn cấp duy trì ổn định tại ngưỡng: 3.3V – 5V.
- Tuổi thọ: Khoảng 30.000 chu kỳ xoay. [15]

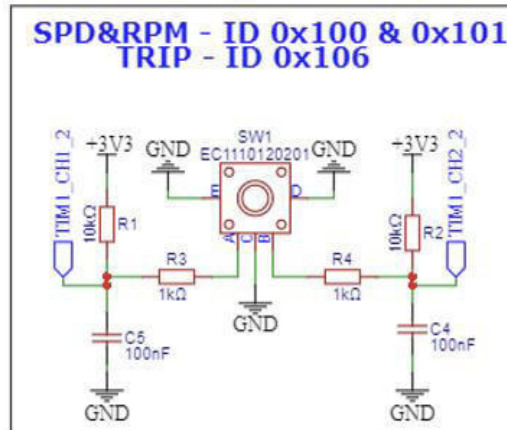


Hình 3.5 Rotary Encoder EC11 [15]

Chân	Tên	Chức năng
1	A (CLK)	Tín hiệu xung kênh A (Dùng để xác định chiều quay và bước)
2	C (GND)	Chân chung của khối mã hóa xoay (thường nối Mass)
3	B (DT)	Tín hiệu xung kênh B (Dùng kết hợp kênh A để định chiều)
4	SW	Chân tín hiệu của nút nhấn tích hợp (Kích hoạt khi nhấn trục)
5	SW	Chân GND của nút nhấn tích hợp

Bảng 3.2 Chức năng từng chân của Rotary Encoder EC11 [15]

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG



Hình 3.6 Khối mô phỏng dữ liệu tốc độ, vòng tua

Hình 3.6 cho thấy sơ đồ kết nối khối mô phỏng dữ liệu tốc độ, vòng tua nhằm tạo tín hiệu giả lập. Trong hệ thống, Encoder EC11 được dùng để thao tác mô phỏng sự thay đổi liên tục của tốc độ xe (SPD) và vòng tua động cơ (RPM), sau đó dữ liệu được truyền lên mạng CAN với ID lần lượt là 0x100 và 0x101 và hành trình (TRIP) với CAN ID 0x106.

Do tín hiệu từ của Encoder EC11 bị nhiễu dội phím (bouncing), một tầng lọc thông thấp RC gồm điện trở hạn dòng 1kΩ, điện trở kéo lên 10kΩ (nối nguồn 3.3V) và tụ điện 100nF được triển khai để dập tắt gai nhiễu tần số cao và chuẩn hóa sóng vuông trước khi đưa vào bộ định thời phân cứng (Timer). Với khả năng tạo ra 20 xung/vòng xoay không giới hạn hành trình, EC11 được sử dụng để giả lập chính xác dải vận tốc và vòng tua máy cho hệ thống. Điện trở nối tiếp $R_{nối\ tiếp}$: Để bảo vệ tiếp điểm không bị đánh lửa, dòng xả cực đại qua tụ phải dưới 10mA. [28]

$$R_{nối\ tiếp} = \frac{U}{I_{max}} = \frac{3.3V}{0.01} = 330\Omega \quad (3.1)$$

⇒ Chọn giá trị tiêu chuẩn 1kΩ (dòng xả thực tế đạt mức an toàn 3.3mA).

- Điện trở kéo lên R_{pullup} : Để dòng tiêu thụ tĩnh cực nhỏ (dưới 1mA), chọn giá trị tiêu chuẩn 10kΩ
- Tụ điện : Để dập nhiễu dội phím, ta chọn hằng số thời gian nạp mục tiêu $\tau = 1.1ms$.

Từ công $\tau = (R_{nối\ tiếp} + R_{pullup}) \cdot C$ (3.2) suy ngược lại điện dung: [29]

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

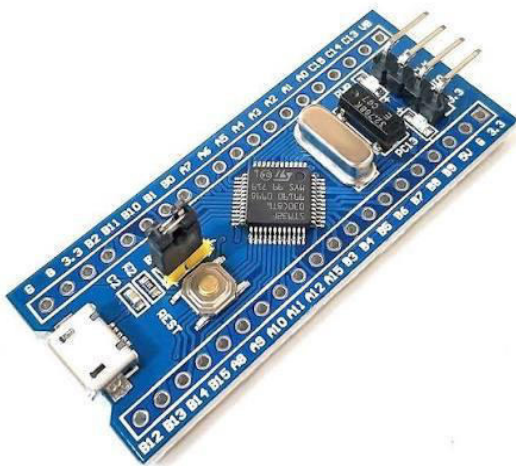
$$C = \frac{\tau}{R_{nối tiếp} + R_{pullup}} = \frac{0.0011}{10000 + 1000} = 100 \times 10^{-9} \text{ F} = 100\text{nF} \quad (3.3)$$

⇒ Kết quả khớp hoàn toàn với giá trị linh kiện tiêu chuẩn trên thị trường: 100nF

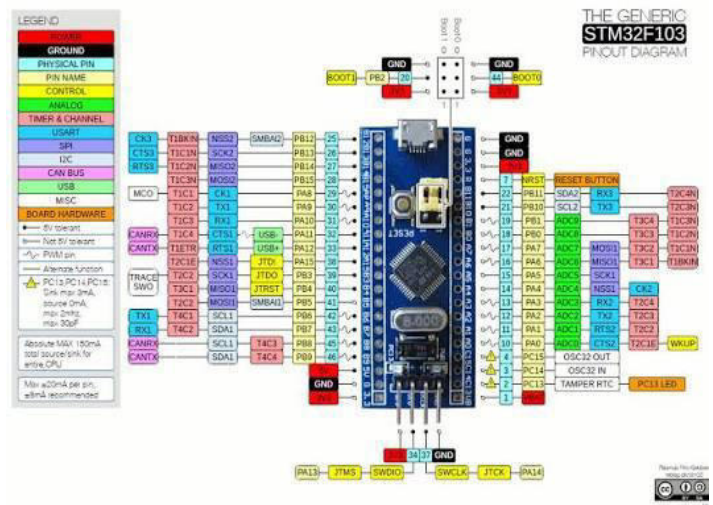
Khối xử lý node 1 (STM32F103C8T6):

Khối xử lý Node 1 sử dụng vi điều khiển STM32F103 để tiếp nhận tín hiệu ngoại vi, tính toán thuật toán số hóa dữ liệu và điều phối chu kỳ đóng gói tin của hệ thống.

Cấu trúc phần cứng của các nút thành phần (Node 1, 2, 3) ưu tiên tích hợp dòng vi điều khiển STM32F103C8T6. Sự phân vùng này dựa trên các ràng buộc về tính định thời hệ thống và yêu cầu mật độ ngoại vi lớn, giúp quá trình số hóa hành vi từ người lái và trích xuất dữ liệu cảm biến diễn ra độc lập, không gây nghẽn luồng xử lý thời gian thực:



Hình 3.7 Mach STM32F103C8T6 BulePill [16]



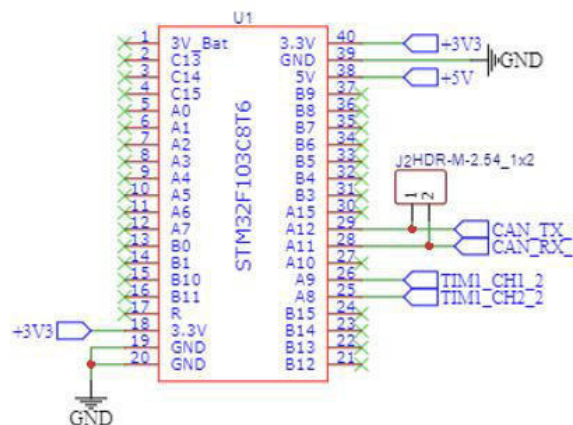
Hình 3.8 Sơ đồ chân của STM32F103C8T6 [16]

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Yêu cầu về tính toán và điều phối dữ liệu thời gian thực của hệ thống IPC được đáp ứng trọn vẹn nhờ cấu trúc lõi ARM Cortex-M3 mạnh mẽ tích hợp trong vi điều khiển STM32F103C8T6. Việc đồng bộ dòng dữ liệu trên mạng nội bộ xe hơi cũng như chuẩn OBD-II được thực thi dựa trên nền tảng cốt lõi là giao tiếp CAN bus 2.0B nguyên bản sẵn có của chip. Thêm vào đó, hiệu suất vận hành được tối ưu hóa đáng kể khi các tác vụ phần cứng được phân tải cho khối DMA và bộ định thời cao cấp xử lý độc lập, cho phép CPU tập trung hoàn toàn năng lực xử lý vào các thuật toán hiển thị chính.

Thông số của STM32F103C8T6:

Cập nhật dữ liệu thời gian thực: Đảm bảo luồng thông số (vận tốc, vòng tua, nhiệt độ) từ CAN Bus được giải mã và hiển thị lên giao diện HMI liên tục, không có độ trễ Cập nhật



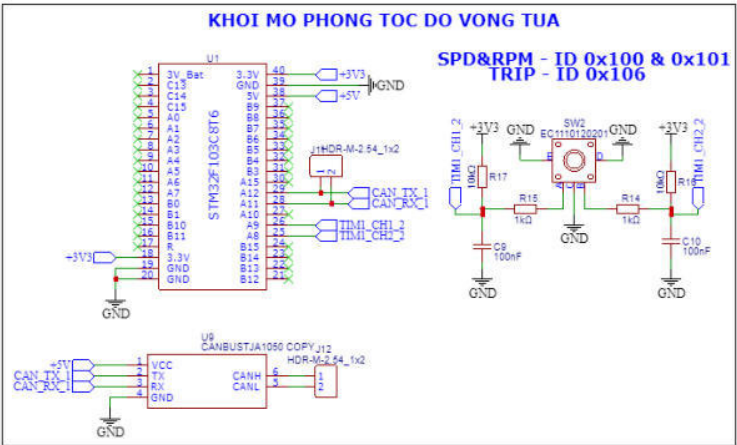
Hình 3.9 Sơ đồ kết nối của khối xử lý node 1

dữ liệu thời gian thực: Đảm bảo luồng thông số (vận tốc, vòng tua, nhiệt độ) từ CAN Bus được giải mã và hiển thị lên giao diện HMI liên tục, không có độ trễ Dựa trên sơ đồ kết nối, hệ thống tận dụng các chân PA8 và PA9 để tiếp nhận xung nhịp. Nhờ tích hợp cơ chế giải mã Encoder bằng phần cứng trực tiếp trên bộ Timer (TIM1), tiến trình đếm xung duy trì độ chính xác tuyệt đối trong mọi dải tốc độ cao mà không gây tổn hao đến tài nguyên xử lý của CPU.

Sơ đồ nguyên lý kết nối tổng thể của Node 1

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Khi tích hợp các thành phần trên, ta có một sơ đồ nguyên lý tổng thể cho khối mô phỏng tốc độ, vòng tua. Sự đồng bộ giữa phần cứng xử lý và mạch bảo vệ tín hiệu giúp khối vận hành ổn định trong suốt quá trình chạy thử nghiệm.



Hình 3.10 Sơ đồ nguyên lý kết nối tổng thể của Node 1

Khởi thực hiện phát định kỳ các gói tin lên đường CANBus với các ID cụ thể để bộ xử lý trung tâm nhận diện:

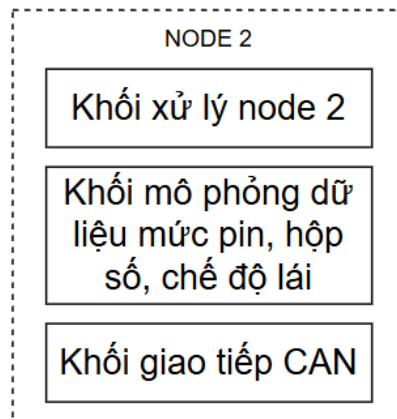
- CAN ID 0x100: Cập nhật Tốc độ xe (Speed).
- CAN ID 0x101: Cập nhật Vòng tua máy (RPM).
- CAN ID 0x106: Cập nhật Quãng đường hành trình (Trip).

Chân	Ký hiệu	Kết nối STM32	Chức năng
1	+3V3	3.3V	Cấp nguồn hoạt động Encoder EC11
2	VCC	5V	Cấp nguồn hoạt động TJA1050
3	GND	GND	Mass chung
4	TIM1_CH_2	A8	Dữ liệu TIM1
5	TIM1_CH1_2	A9	Dữ liệu TIM1
6	CAN_RX_1	A11	Dữ liệu CAN RX
7	CAN_TX_1	A12	Dữ liệu CAN TX

Bảng 3.3 Thông số kết nối của Node 1

3.2.2 Khối xử lý dữ liệu Node 2

Trong cấu trúc tổng thể của đề tài, Node 2 đảm nhiệm vai trò tái hiện các trạng thái liên quan đến hệ thống quản lý năng lượng và các tùy chọn vận hành của xe. Chức năng cốt lõi của khối là số hóa các tương tác vật lý từ người dùng để giả lập trạng thái dung lượng Pin (Battery), các giá trị vị trí cấp số (Gear) và các chế độ lái thích ứng (Drive Mode), sau đó truyền tải liên tục lên bus mạng để cụm đồng hồ xử lý và hiển thị trực quan.



Hình 3.11 Sơ đồ khối node 2

Dựa trên Hình 3.11, cấu trúc phần cứng của Node 2 được phân tách thành ba khối chức năng chính bao gồm:

- Khối xử lý Node 2: Sử dụng vi điều khiển STM32F10C8T6 để vận hành các bộ ADC đọc điện áp biến thiên từ ngoại vi, xử lý logic trạng thái và điều phối đóng gói gói tin.
- Khối giao tiếp CAN: Sử dụng module chuyên dụng MCP2551 để tiếp nhận dữ liệu từ khối xử lý, đóng gói thành khung tin CAN tiêu chuẩn và truyền lên trục mạng Bus.
- Khối mô phỏng dữ liệu mức pin, hộp số, chế độ lái: Sử dụng hệ thống các Biến trở xoay (Potentiometers) đóng vai trò bộ chia áp để chuyển đổi các thao tác cơ học của người dùng thành tín hiệu analog gửi về vi điều khiển.

Khối giao tiếp CAN (MCP2551)

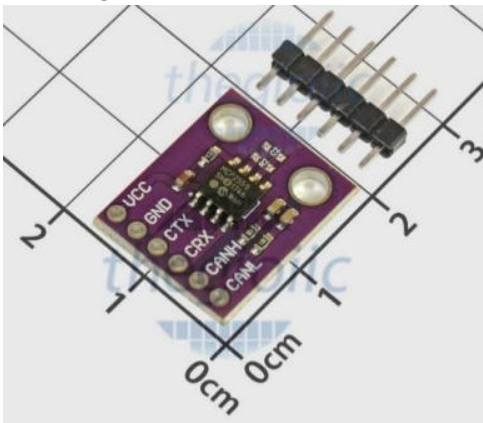
Khác biệt hoàn toàn với TJA1050, cấu trúc phần cứng của module MCP2551 do Microchip sản xuất không tích hợp sẵn điện trở đầu cuối 120Ω . Đặc tính này tối ưu hóa thiết bị khi cấu hình làm một nút trung gian mạng Bus, ngăn ngừa triệt để hiện tượng suy

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

giảm biên độ tín hiệu do mắc song song quá nhiều điện trở đầu cuối, duy trì đúng kiến trúc mạng CAN chuẩn xe hơi.. [17]

Thông số kỹ thuật:

- IC chính: MCP2551 (Microchip)
- Chuẩn giao tiếp: CAN Bus tốc độ cao (ISO-11898)
- Tốc độ truyền tối đa: Lên đến 1 Mbps
- Điện áp hoạt động: 4.5V – 5.5V (Tiêu chuẩn là 5V)
- Dòng tối đa: ~ 75mA (0.075A) ở trạng thái trội (Dominant)
- Số lượng node tối đa: Lên đến 112 node.
- Tính năng: Tích hợp bảo vệ ngắn mạch và miễn nhiễm nhiễu vô tuyến (RFI) cao



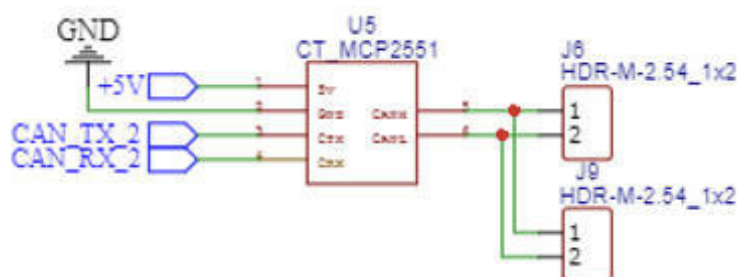
Hình 3.12 Module MCP2551 [17]

Chân	Chức năng
VCC	Nguồn cấp cho module (5V)
GND	Mass của hệ thống
CTX	Chân nhận dữ liệu truyền từ vi điều khiển
CRX	Chân xuất dữ liệu nhận về vi điều khiển
CANH	Đường tín hiệu CAN High (Kết nối mạng CAN)
CANL	Đường tín hiệu CAN Low (Kết nối mạng CAN)

Bảng 3.4 Chức năng từng chân của Module MCP2551 [17]

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Để dữ liệu có thể truyền dẫn trên bus mạng chung, hệ thống tích hợp IC thu phát MCP2551 hoạt động ở mức nguồn 5V. Linh kiện này đóng vai trò là giao diện vật lý, chuyển đổi tín hiệu mức logic chuẩn từ vi điều khiển sang tín hiệu điện áp vi sai trên đường truyền.



Hình 3.13 Sơ đồ nguyên lý MCP2551

Các đường phát CAN_TX_2 và đường nhận CAN_RX_2 được kết nối trực tiếp với IC thu phát, giúp dữ liệu luôn được toàn vẹn và có khả năng chống nhiễu điện từ vượt trội khi đi qua các đoạn dây dẫn trong mô hình.

Linh kiện được lựa chọn vì khác với TJA1050, MCP2551 có cấu trúc phần cứng không tích hợp sẵn điện trở đầu cuối 120Ω giúp nó tối ưu khi cấu hình làm một Node trung gian trên mạng Bus. Điều này ngăn ngừa hiện tượng suy giảm biên độ tín hiệu do mắc song song quá nhiều điện trở, từ đó đảm bảo tính toàn vẹn dữ liệu và duy trì đúng kiến trúc mạng CAN chuẩn xe hơi.

Khối mô phỏng dữ liệu mức pin

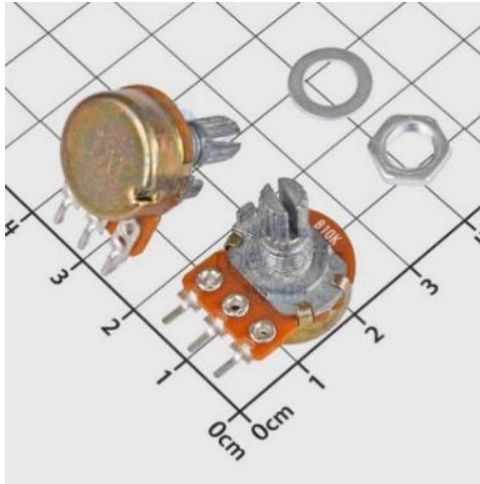
Khối sử dụng biến trở WH148 (loại 10 kΩ) là linh kiện có khả năng thay đổi giá trị điện trở thông qua việc xoay trục cơ học, từ đó hoạt động như một bộ chia áp để tạo ra mức điện áp đầu ra (analog) thay đổi tuyến tính theo góc xoay.

Trong đồ án IPC, các biến trở này được sử dụng như những thiết bị giả lập hoạt động trên xe (mức pin, hộp số và chế độ lái). STM32 sẽ chuyển đổi ADC để đọc các giá trị điện áp, sau đó xử lý và hiển thị thông số tương ứng lên màn hình hiển thị. [18]

Thông số kỹ thuật:

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

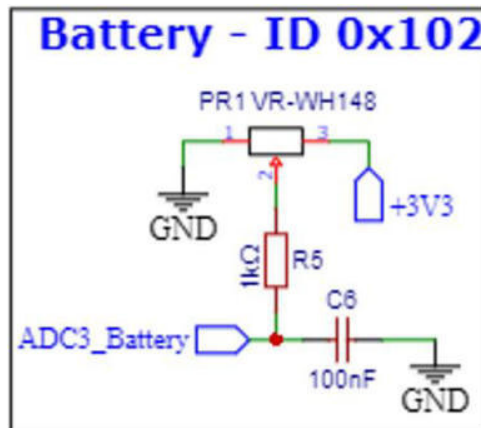
- Mã sản phẩm: WH148-3P-103 (B10K), 3 chân
- Loại linh kiện: Biến trở xoay đơn (Rotary Potentiometer)
- Giá trị điện trở tối đa: 10 kOhm (Sai số $\pm 20\%$)
- Công suất định mức: 0.125W



Hình 3.14 Biến Trở WH148 [18]

Chân	Tên	Chức năng
1	VCC (hoặc GND)	Chân cấp nguồn dương (Ví dụ: 3.3V)
2	OUT (Wiper)	Chân chạy ở giữa, xuất tín hiệu điện áp analog (Kết nối vào chân đọc ADC của vi điều khiển)
3	GND (hoặc VCC)	Chân nối Mass của hệ thống

Bảng 3.5 Chức năng từng chân của Biến Trở [18]



Hình 3.15 Sơ đồ kết nối khối mô phỏng dữ liệu mức pin

Hình 3.15 cho thấy sơ đồ kết nối biến trở dùng để tạo tín hiệu giả lập dung lượng pin (Battery). Dữ liệu này sau khi được vi điều khiển số hóa sẽ được truyền lên mạng CAN với mã định danh (CAN ID) là 0x102.

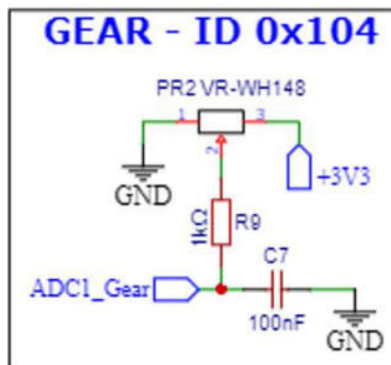
Lý do lựa chọn biến trở: Linh kiện cho phép thay đổi giá trị từ 0Ω đến $10k\Omega$ một cách tuyến tính, giúp tạo ra dải điện áp giả lập mịn từ 0V đến 3.3V ứng với mức pin 0 đến 100%. Đây là giải pháp phần cứng có sai số thấp (dưới 5%), giá thành rẻ và đáp ứng tần số lấy mẫu 12-bit ADC của STM32 một cách ổn định.

Về mặt nguyên lý:

- Tạo tín hiệu: Bằng cách liên kết hai đầu cố định của biến trở vào mạng nguồn +3V3 và điểm tham chiếu GND, việc tác động lực xoay cơ học lên trục sẽ trực tiếp điều khiển điện áp ngõ ra tại chân giữa biến đổi một cách tuyến tính
- Lọc tín hiệu: Trước khi đưa vào vi điều khiển, tín hiệu này đi qua mạch lọc RC bao gồm điện trở R5 ($1k\Omega$) mắc nối tiếp và tụ điện C6 ($100nF$) mắc xuống mass. Giúp triệt tiêu các nhiễu cao tần và làm phẳng tín hiệu, đảm bảo mức điện áp đưa vào chân ADC3_Battery luôn ổn định

Khối mô phỏng dữ liệu hộp số

Khối sử dụng biến trở WH148 (loại 10 kOhm) là linh kiện có khả năng thay đổi giá trị điện trở thông qua việc xoay trục cơ học, từ đó hoạt động như một bộ chia áp để tạo ra mức điện áp đầu ra (analog) thay đổi tuyến tính theo góc xoay.



Hình 3.16 Sơ đồ kết nối khối mô phỏng dữ liệu hộp số

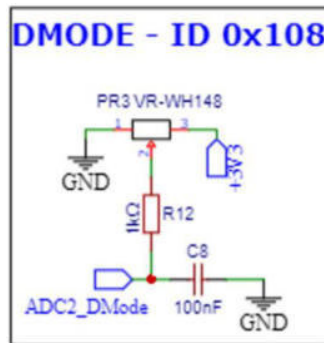
Hình 3.16 cho thấy sơ đồ kết nối biến trở dùng để tạo tín hiệu giả lập vị trí cấp số của xe (Gear). Dữ liệu này sau khi được vi điều khiển số hóa sẽ được truyền lên mạng CAN với mã định danh (CAN ID) là 0x104.

Lý do lựa chọn biến trở: Linh kiện cho phép thay đổi giá trị từ 0Ω đến $10k\Omega$ một cách tuyến tính, giúp tạo ra dải điện áp giả lập mịn từ 0V đến 3.3V ứng với mức pin 0 đến 100%. Đây là giải pháp phần cứng có sai số thấp (dưới 5%), giá thành rẻ và đáp ứng tần số lấy mẫu 12-bit ADC của STM32 một cách ổn định.

Về mặt nguyên lý:

- Tạo tín hiệu: Tương tự như khối giả lập pin, biến trở PR2 được mắc giữa nguồn +3V3 và mass (GND). Việc thao tác xoay biến trở sẽ đại diện cho các trạng thái vị trí số khác nhau của xe như: P, R, N, D.
- Lọc tín hiệu: Trước khi đưa vào vi điều khiển, tín hiệu đi qua mạch lọc RC bao gồm điện trở R9 (1kΩ) mắc nối tiếp và tụ điện C7 (100nF) mắc xuống mass. Giúp triệt tiêu nhiễu cao tần, giúp làm phẳng tín hiệu đầu vào. Điều này đảm bảo mức điện áp đưa tới chân ADC1_Gear luôn ổn định,

Khối mô phỏng dữ liệu chế độ lái



Hình 3.17 sơ đồ kết nối khối mô phỏng dữ liệu chế độ lái

Hình 3.17 cho thấy sơ đồ kết nối biến trở dùng để tạo tín hiệu giả lập chế độ lái của xe (DMode - Drive Mode). Dữ liệu này sau khi được vi điều khiển số hóa sẽ được truyền lên mạng CAN với mã định danh (CAN ID) là 0x108.

Về mặt nguyên lý:

- Tạo tín hiệu: Tương tự như các khối giả lập trước, biến trở PR3 được mắc giữa nguồn +3V3 và mass (GND). Việc thao tác xoay biến trở sẽ tạo ra sự thay đổi điện áp tuyến tính tại chân giữa, đại diện cho các chế độ lái khác nhau của xe (Eco, Normal, Sport).
- Lọc tín hiệu: Trước khi đưa vào vi điều khiển, tín hiệu đi qua mạch lọc RC bao gồm điện trở R12 (1kΩ) mắc nối tiếp và tụ điện C8 (100nF) mắc xuống mass. Giúp triệt tiêu nhiễu cao tần, làm phẳng tín hiệu đầu vào. Điều này giúp mức điện áp đưa tới chân ADC2_DMode luôn ổn định

Điện trở (R): Theo Datasheet của STM32, trở kháng đầu vào ADC phải nhỏ hơn 50kΩ để nạp đủ tụ lấy mẫu nhanh chóng. Ta chọn giá trị tiêu chuẩn nhỏ để mạch đáp ứng tốt: 1kΩ

Tụ điện(C): Thao tác vận núp có tần số rất thấp. Để tín hiệu tay người đi qua và chặn nhiễu điện từ, ta đặt mục tiêu tần số cắt $f_c \approx 1.5\text{kHz}$. Từ công thức $f_c = \frac{1}{2\pi RC}$ (3.4), ta có:

$$C = \frac{1}{2\pi \cdot R \cdot f_c} = \frac{1}{2\pi \cdot 1000 \cdot 1500} \approx 106 \times 10^{-9} \text{ F} = 106\text{nF} \quad (3.5)$$

Để dễ dàng thiết kế và lắp ráp, ta chọn giá trị tiêu chuẩn gần nhất: 100nF [27]

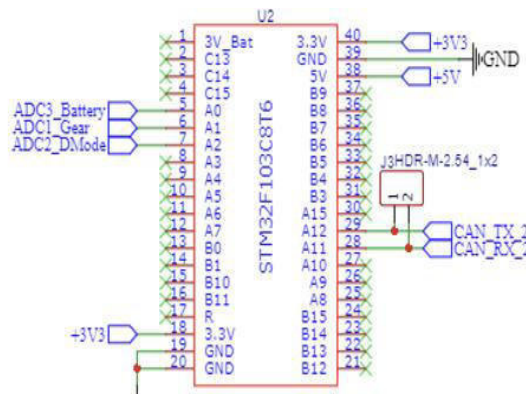
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Khởi xử lý Node 2 (STM32F103C8T6)

Để đảm bảo tính đồng bộ và nhất quán về mặt kiến trúc phần cứng trong toàn hệ thống IPC, Node 2 tiếp tục sử dụng vi điều khiển STM32F103C8T6 làm thiết bị xử lý. Tuy nhiên, thay vì tập trung vào các bộ định thời đo xung như Node 1, cấu hình phần cứng của Node 2 được tối ưu hóa chuyên biệt cho nhiệm vụ giám sát năng lượng và đo đặc điện áp thời gian thực của hệ thống pin/ắc quy phương tiện.

Lý do lựa chọn STM32F103C8T6 là thiết bị xử lý cho Node 2:

- Khai thác khối ngoại vi ADC 12-bit độ phân giải cao: Node 2 tận dụng tối đa bộ chuyển đổi tương tự - số (ADC) tích hợp sẵn với độ phân giải 12-bit (cho ra giá trị số hóa từ 0 đến 4095). Điều này cho phép hệ thống chia nhỏ dải đo điện áp, nhận diện chính xác những sai số hoặc sụt áp nhỏ nhất từ mạch phân áp giám sát pin (BAT_SEN).
- Chế độ quét liên tục (Continuous Scan Mode): Tích hợp cấu hình ADC quét liên tục kết hợp bộ lọc trung bình số (Digital Filtering) ngay trong firmware, giúp loại bỏ nhiễu tần số cao từ hệ thống nguồn trước khi đóng gói dữ liệu.



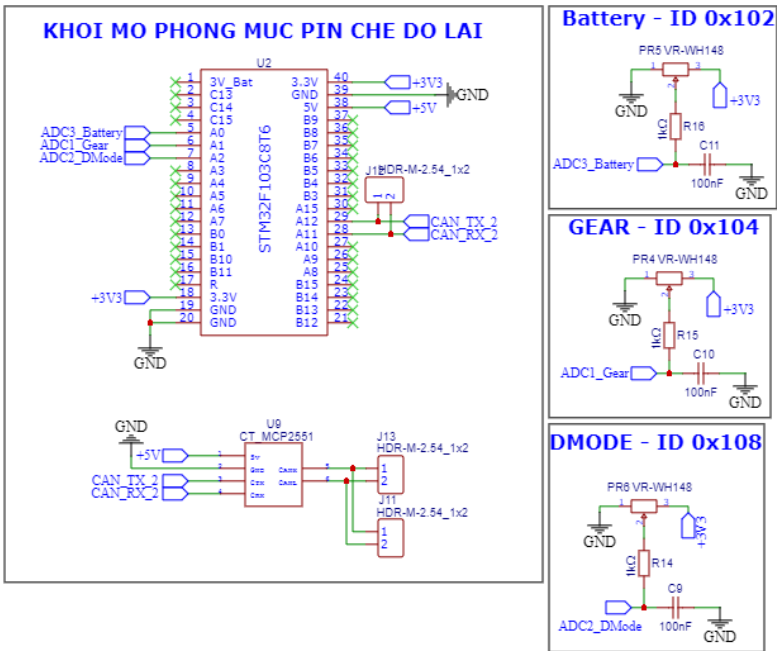
Hình 3.18 Sơ đồ kết nối của khối xử lý node 2

Hình 3.18 cho thấy khối sử dụng vi điều khiển STM32F103C8T6. Nhiệm vụ chính của nó là vận hành các bộ chuyển đổi tương tự - số (ADC) tích hợp sẵn để đọc dải điện áp biến thiên từ các thiết bị ngoại vi, từ đó tính toán chính xác trạng thái mô phỏng của xe.

Dựa trên sơ đồ thiết kế, các chân PA0 (ADC3_Battery), PA1 (ADC1_Gear) và PA2 (ADC2_DMode) được quy hoạch làm ngõ vào tín hiệu analog. Tốc độ lấy mẫu cao giúp vi

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

điều khiển ghi nhận tức thời mọi thao tác điều chỉnh, đảm bảo tính thời gian thực cho dữ liệu.



Hình 3.19 Sơ đồ nguyên lý tổng thể Node 2

Khi tích hợp các thành phần trên, ta có một sơ đồ nguyên lý tổng thể cho Node 2. Sự đồng bộ giữa phần cứng xử lý và mạch bảo vệ tín hiệu giúp khối vận hành ổn định trong suốt quá trình chạy thử nghiệm.

Khởi thực hiện phát định kỳ các gói tin lên đường CANBus với các ID cụ thể để bộ xử lý trung tâm nhận diện:

- CAN ID 0x102: Cập nhật Mức pin (Batter).
- CAN ID 0x104: Cập nhật hộp số (Gear).
- CAN ID 0x108: Cập nhật chế độ lái (Dmode).

Chân	Ký hiệu	Kết nối STM32	Chức năng
1	+3V3	3.3V	Cấp nguồn hoạt động Biến trở 10K
2	VCC	5V	Cấp nguồn hoạt động MCP2551
3	GND	GND	Mass chung

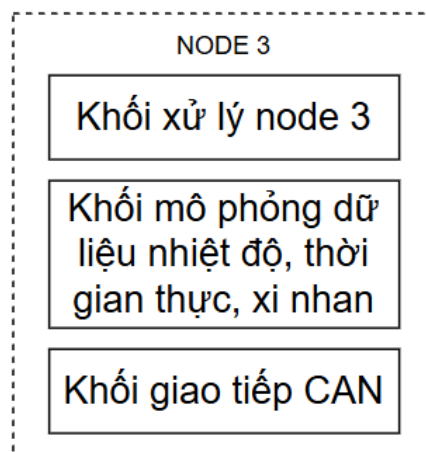
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

4	ADC3_Battery	A0	Dữ liệu ADC
5	ADC1_Gear	A1	Dữ liệu ADC
6	ADC2_Dmode	A2	Dữ liệu ADC
7	CAN_RX_2	A11	Dữ liệu CAN RX
8	CAN_TX_2	A12	Dữ liệu CAN TX

Bảng 3.6 Thông số kết nối của Node 2

3.2.3 Khối xử lý dữ liệu Node 3

Trong cấu trúc tổng thể của đề tài, Node 3 đảm nhiệm vai trò tái hiện các tín hiệu điều khiển thân xe và cập nhật các thông số môi trường cùng thời gian thực. Chức năng cốt lõi của khối là số hóa các tương tác logic từ người lái (tín hiệu báo rẽ) và thu thập dữ liệu từ các cảm biến số, sau đó đóng gói và truyền tải các thông số này lên bus mạng để cụm đồng hồ xử lý hiển thị trực quan



Hình 3.20 Sơ đồ khối Node 3

Dựa trên hình 3.20, cấu trúc phân cứng của Node 3 được phân tách thành ba khối chức năng chính bao gồm:

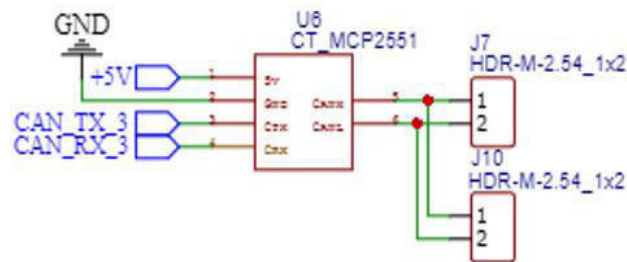
- Khối xử lý Node 3: Sử dụng vi điều khiển STM32F103C8T6 để khai thác dữ liệu qua giao tiếp I2C, quét trạng thái nút nhấn ngoại vi và điều phối chu kỳ đóng gói khung tin CAN.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

- Khối mô phỏng dữ liệu nhiệt độ, thời gian thực, xi nhan: Tích hợp cảm biến BMP180, module thời gian thực DS1307 và công tắc gạt để thu thập, giả lập toàn bộ thông số môi trường lẫn tín hiệu thân xe.
- Khối giao tiếp CAN: Sử dụng IC chuyên dụng MCP2551 để tiếp nhận dữ liệu, đóng gói thành khung tin CAN tiêu chuẩn (CAN ID 0x106) và truyền lên trực mạng Bus.

Khối giao tiếp giao tiếp CAN (MCP2551)

Khối sử dụng module MCP2551 là bộ thu phát mạng CAN (CAN Transceiver) tốc độ cao chuẩn công nghiệp, làm giao diện trung gian chuyển đổi tín hiệu logic từ vi điều khiển thành tín hiệu vi sai (CAN_H, CAN_L) truyền trên CAN Bus.



Hình 3.21 Sơ đồ nguyên lý MCP2551

Trong hệ thống IPC, module này giúp STM32 kết nối với các mạch giả lập tín hiệu, đảm bảo việc truyền nhận dữ liệu CAN diễn ra ổn định, chính xác và chống nhiễu tốt ở môi trường điện áp 5V. Các đường phát CAN_TX_3 và đường nhận CAN_RX_3 được kết nối trực tiếp với IC thu phát, giúp dữ liệu luôn được toàn vẹn và có khả năng chống nhiễu điện từ vượt trội khi đi qua các đoạn dây dẫn trong mô hình.

Linh kiện được lựa chọn giống như ở Node 2 là nhờ cấu trúc phần cứng không tích hợp sẵn điện trở đầu cuối 120Ω giúp nó tối ưu khi cấu hình làm một Node trung gian trên mạng Bus. Điều này ngăn ngừa hiện tượng suy giảm biên độ tín hiệu do mắc song song quá nhiều điện trở, từ đó đảm bảo tính toàn vẹn dữ liệu và duy trì đúng kiến trúc mạng CAN chuẩn xe hơi.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

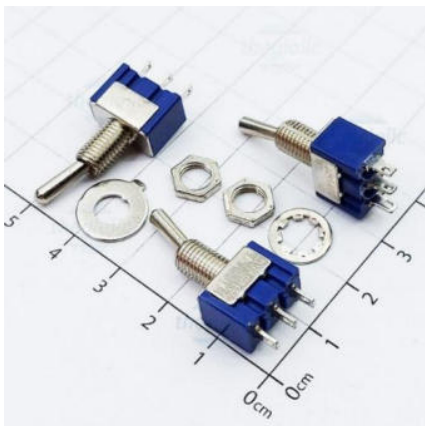
Khối mô phỏng dữ liệu xi nhan (MTS-103)

Khối sử dụng công tắc gạt MTS-103 là loại công tắc cơ học 3 vị trí (ON-OFF-ON) có tính năng giữ trạng thái, đóng vai trò chuyển hướng dòng điện hoặc thay đổi mức logic của mạch.

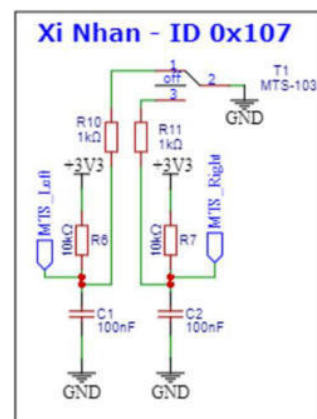
Trong hệ thống IPC, linh kiện này được sử dụng để giả lập các tín hiệu điều khiển vật lý từ người lái (công tắc gạt xi-nhan trái/phải, chế độ đèn chiếu sáng). Các trạng thái này sẽ được vi điều khiển (như STM32) đọc mức logic để xử lý và cập nhật trạng thái hiển thị đồ họa tương ứng lên màn hình đồng hồ. [19]

Thông số kỹ thuật:

- Mã sản phẩm: MTS-103
- Kiểu hoạt động: Gạt giữ trạng thái, 3 vị trí (ON - OFF - ON)
- Số cực/Số ngả: SPDT (Single Pole Double Throw)
- Dòng tải/Điện áp tối đa: 6A - 125VAC (Hoạt động tốt và an toàn với các mức điện áp DC thấp 3.3V/5V trong mạch)
- Số chân: 3 chân
- Tuổi thọ cơ học: Hơn 10.000 chu kỳ gạt.



Hình 3.22 nhan Công Tắc Gạt MTS-103 [19]



Hình 3.23 Sơ đồ kết nối khối mô phỏng dữ liệu Xi nhan

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Chân	Tên	Chức năng
1	Cực L1 (OUT 1)	Chân xuất tín hiệu 1 (Kết nối với mạch hoặc chân GPIO thứ nhất)
2	COM (Chân chung)	Chân trung tâm, thường được cấp nguồn (VCC) hoặc nối Mass (GND)
3	Cực L2 (OUT 2)	Chân xuất tín hiệu 2 (Kết nối với mạch hoặc chân GPIO thứ hai)

Bảng 3.7 Chức năng từng chân của Công Tắc MTS-103 [19]

Hình 3.23 cho thấy sơ đồ kết nối công tắc gạt 3 vị trí (MTS-103) để giả lập tín hiệu đèn báo rẽ (Xi nhan), dữ liệu trạng thái được truyền lên mạng CAN với ID 0x107.

Về mặt nguyên lý:

- Tạo tín hiệu: Chân chung của công tắc nối mass (GND). Khi gạt sang trái hoặc phải, đường tín hiệu tương ứng (MTS_Left hoặc MTS_Right) sẽ được kéo xuống mức thấp (0V). Ở vị trí giữa (OFF), mạch hở.
- Xử lý tín hiệu: Mỗi kênh sử dụng điện trở kéo lên 10k Ω (nối +3V3) để duy trì mức logic cao ổn định khi ở trạng thái OFF. Đồng thời, tín hiệu đi qua mạch lọc RC (điện trở 1k Ω và tụ 100nF) giúp chống dội phím (debounce), loại bỏ nhiễu tiếp điểm cơ học để vi điều khiển đọc trạng thái dứt khoát và chính xác.
- Lý do lựa chọn: Linh kiện cơ học 3 vị trí (ON-OFF-ON) cấu hình chính xác cho 3 trạng thái: Xi-nhan Trái, Tắt và Xi-nhan Phải.
- Điện trở nối tiếp $R_{nối\ tiếp}$: Để bảo vệ tiếp điểm không bị đánh lửa, dòng xả cực đại qua tụ phải dưới 10mA. [28]

$$R_{nối\ tiếp} = \frac{U}{I_{max}} = \frac{3.3V}{0.01} = 330\Omega \quad (3.6)$$

⇒ Chọn giá trị tiêu chuẩn 1k Ω (ứng với R10, R11 trong sơ đồ) để dòng xả thực tế ở mức an toàn 3.3mA.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

- Điện trở kéo lên R_{pullup} : Để duy trì mức logic CAO ổn định và không chế dòng tiêu thụ tĩnh cực nhỏ (dưới 1mA), chọn giá trị tiêu chuẩn 10k Ω (ứng với R6, R7).
- Tụ điện : Để dập nhiễu dội phím, ta chọn hằng số thời gian nạp mục tiêu $\tau = 1.1\text{ms}$. Từ công thức $\tau = (R_{nối\ tiếp} + R_{pullup}) \cdot C$ (3.7) suy ngược lại điện dung: [29]

$$C = \frac{\tau}{R_{nối\ tiếp} + R_{pullup}} = \frac{0.0011}{10000 + 1000} = 100 \times 10^{-9} \text{ F} = 100\text{nF} \quad (3.8)$$

⇒ Kết quả tính toán khớp hoàn toàn với giá trị linh kiện tiêu chuẩn trên thị trường: 100nF (ứng với C1, C2). Mức thông số này giúp vi điều khiển nhận diện tín hiệu gạt xi nhan trái/phải dứt khoát, không bị chập chờn.

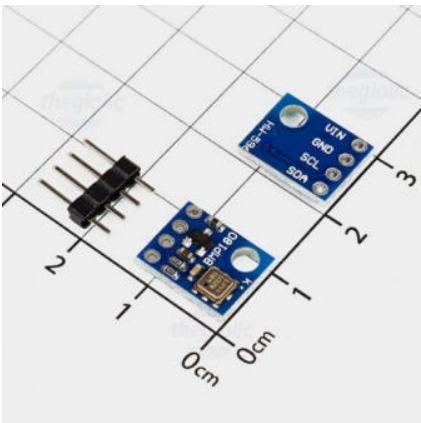
Khối mô phỏng dữ liệu nhiệt độ (BMP180)

Tác vụ thu thập thông số ngoại cảnh (như giá lập nhiệt độ nước làm mát hoặc áp suất không khí động cơ) sử dụng cảm biến kỹ thuật số BMP180 của hãng Bosch làm linh kiện cốt lõi. Điểm vượt trội của linh kiện này là việc tích hợp sẵn bộ xử lý tín hiệu và mạch chuyển đổi ADC nội tại, giúp loại bỏ hoàn toàn các sai số và nhiễu của đường truyền analog truyền thống.[20]

Thông số kỹ thuật:

- IC chính: BMP180 (Bosch)
- Chuẩn giao tiếp: I2C
- Dải đo áp suất: 300 - 1100 hPa (tương đương độ cao từ -500m đến 9000m)
- Dải đo nhiệt độ: 0 đến 65°C
- Điện áp hoạt động: 1.8V - 3.6V (Module trên có tích hợp IC hạ áp nên có thể cấp VIN 3.3V - 5V)
- Độ phân giải: 0.01 hPa (áp suất) / 0.1°C (nhiệt độ)

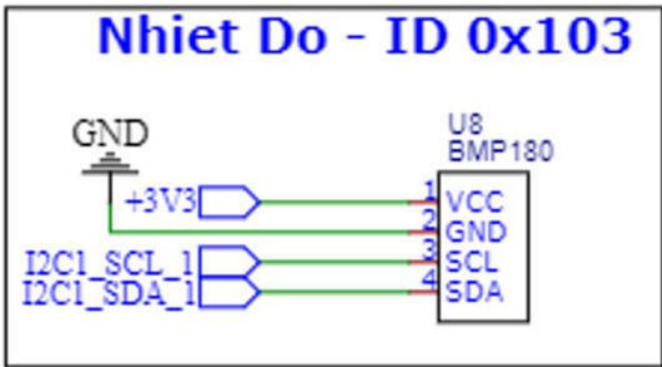
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG



Hình 3.24 Module Cảm Biến BMP180 [20]

Chân	Tên	Chức năng
1	VIN	Chân cấp nguồn dương cho module (3.3V hoặc 5V)
2	GND	Chân nối Mass của hệ thống
3	SCL	Đường xung clock đồng bộ giao tiếp I2C
4	SDA	Đường truyền nhận dữ liệu giao tiếp I2C

Bảng 3.8 Chức năng từng chân của Cảm Biến BMP180 [20]



Hình 3.25 Sơ đồ kết nối khối mô phỏng dữ liệu nhiệt độ

Hình 3.25 cho thấy sơ đồ kết nối cảm biến BMP180 để thu thập dữ liệu nhiệt độ (mô phỏng nhiệt độ nước làm mát hoặc động cơ), dữ liệu sau đó được truyền lên mạng CAN với ID **0x103**.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Về mặt nguyên lý hoạt động:

- Giao tiếp: Cảm biến được sử dụng nguồn cấp 3V3 và giao tiếp với vi điều khiển thông qua chuẩn truyền thông I2C. Kết nối bao gồm hai đường tín hiệu: xung nhịp đồng bộ (I2C1_SCL_1) và đường dữ liệu (I2C1_SDA_1).
- Xử lý tín hiệu: Khác với các biến trở analog, BMP180 là cảm biến kỹ thuật số đã tích hợp sẵn bộ xử lý và ADC bên trong. Vi điều khiển chỉ cần gửi lệnh và đọc trực tiếp giá trị nhiệt độ đã được số hóa qua bus I2C, đảm bảo độ chính xác cao và miễn nhiễm với nhiễu điện từ. BMP180 tích hợp sẵn ADC và giao tiếp qua bus I2C với độ phân giải nhiệt độ 0.1°C (sai số $\pm 1.0^{\circ}\text{C}$). Cấu trúc này giúp đơn giản hóa thiết kế mạch, loại bỏ nhiễu đường truyền analog và đảm bảo dữ liệu đo đạc chính xác.

Khối mô phỏng dữ liệu thời gian (RTC DS1307)

Module TinyRTC DS1307 sử dụng thạch anh 32.768 kHz để cung cấp thời gian thực liên tục cho cụm đồng hồ táp-lô. Khi hệ thống chính bị ngắt nguồn tổng, module sẽ tự động chuyển sang viên pin dự phòng CR2032 (3V) tích hợp sẵn nhằm duy trì mốc thời gian hoạt động chính xác và độc lập. [21]

Thông số kỹ thuật:

- IC chính: DS1307 (Maxim Integrated)
- Chuẩn giao tiếp: I2C
- Dòng tiêu thụ: 5mA
- Phạm vi nguồn cấp duy trì ổn định tại ngưỡng: 5V

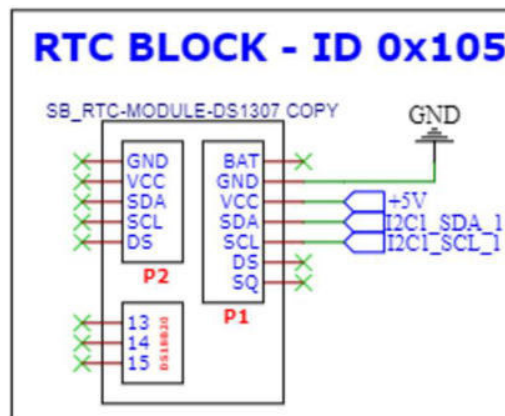


Hình 3.26 Module TinyRTC DS1307 [21]

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Chân	Tên	Chức năng
1	BAT	Chân cấp nguồn dự phòng ngoài (thường không dùng vì đã gắn sẵn pin CR2032)
2	GND	Chân nối Mass của hệ thống
3	VCC	Chân cấp nguồn dương cho module (5V)
4	SDA	Đường truyền nhận dữ liệu giao tiếp I2C
5	SCL	Đường xung clock đồng bộ giao tiếp I2C
6	DS	Chân chờ để hàn thêm cảm biến nhiệt độ 1-Wire (như DS18B20)
7	SQ	Ngõ ra tạo xung vuông (Square Wave Output)

Bảng 3.9 Chức năng từng chân của Module TinyRTC DS1307



Hình 3.27 Sơ đồ kết nối khối mô phỏng dữ liệu thời gian thực RTC [21]

Hình 3.27 cho thấy sơ đồ kết nối module thời gian thực DS1307 nhằm cung cấp thông tin ngày giờ chính xác cho hệ thống. Dữ liệu thời gian này sẽ được đóng gói và truyền lên mạng CAN với mã định danh (CAN ID) là 0x105.

Về mặt nguyên lý hoạt động:

Thiết bị vận hành ở mức điện áp 5V DC với dòng tiêu thụ tĩnh cực thấp chỉ 5mA. DS1307 sử dụng chung trục bus giao tiếp I2C với cảm biến BMP180 (đường xung nhịp I2C1_SCL_1 và đường dữ liệu I2C1_SDA_1), giúp tiết kiệm tối đa tài nguyên chân I/O

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

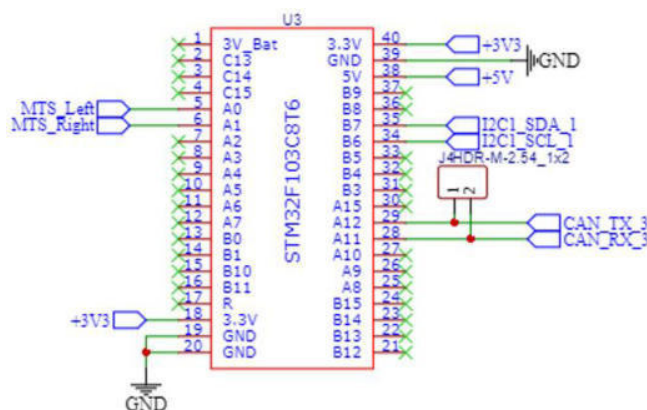
của vi điều khiển. Ngoài ra, board module còn tích hợp thêm chip bộ nhớ EEPROM 24C32 phục vụ lưu trữ các thông số cấu hình và ghi log dữ liệu vận hành khi cần thiết.

Khối xử lý Node 3 (STM32F103C8T6)

Node 3 đóng vai trò là bộ quản lý và giám sát toàn bộ các tín hiệu điều khiển vật lý từ người lái (gồm vị trí tay số P-R-N-D, chế độ lái Drive Mode, và hệ thống đèn báo rẽ/xi-nhan). Việc lựa chọn STM32F103C8T6 cho khối này tập trung vào khả năng đáp ứng số lượng lớn các tín hiệu logic số cấu hình ngắt ngõ vào.

Lý do lựa chọn STM32F103C8T6 là thiết bị xử lý cho Node 3:

- Số lượng chân GPIO dồi dào và linh hoạt: Với số lượng chân I/O lớn, Node 3 dễ dàng tiếp nhận đồng thời nhiều đường tín hiệu độc lập từ ma trận nút nhấn hoặc công tắc hành trình mà không cần sử dụng IC mở rộng chân (như các dòng chip thiếu chân).
- Hệ thống trở kéo lên/xuống nội tại (Internal Pull-up/Pull-down): Giúp đơn giản hóa sơ đồ nguyên lý mạch PCB, không cần thiết kế cụm điện trở treo cổng kênh bên ngoài cho các nút bấm chỉ thị số và đèn tín hiệu.
- Ngắt ngoại vi (EXTI) phản hồi tức thời: Khối xử lý cấu hình ngắt cạnh cho các chân nhận biết tín hiệu xi-nhan hoặc chuyển trạng thái số, đảm bảo phát hiện lệnh điều khiển của người dùng ngay lập tức và chống rung phím hiệu quả.

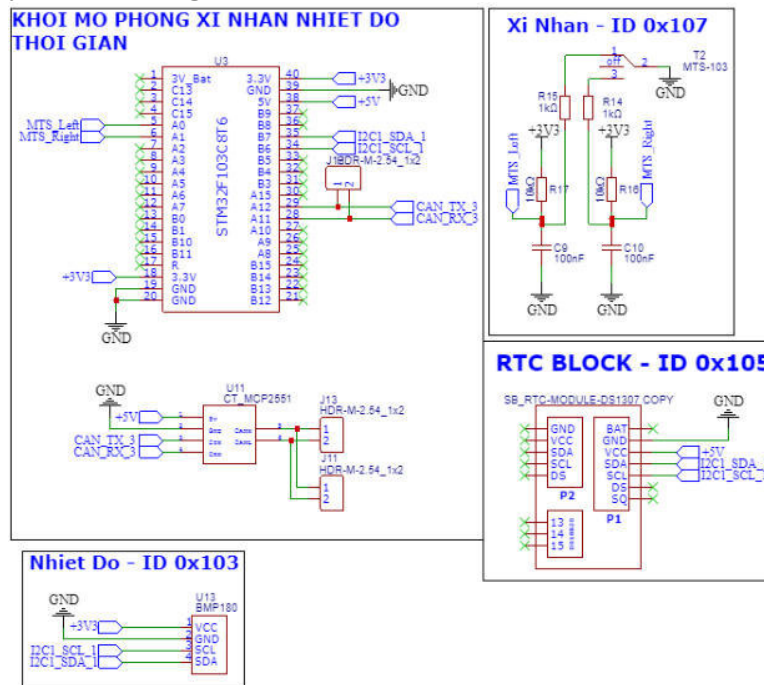


CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Nhiệm vụ chính là giám sát trạng thái các chân ngắt kỹ thuật số và vận hành giao thức I2C để giao tiếp với các module cảm biến, từ đó tính toán chính xác các thông điệp trạng thái của xe.

Dựa trên sơ đồ thiết kế hình 3.28, các chân PA0 (MTS_Left) và PA1 (MTS_Right) được quy hoạch để tiếp nhận các tín hiệu logic từ công tắc gạt. Đồng thời, cặp chân PB6 (SCL) và PB7 (SDA) được cấu hình để quản lý đường bus I2C dùng chung cho cảm biến nhiệt độ và mạch thời gian thực.

Sơ đồ nguyên lý kết nối tổng thể của Node 3



Hình 3.29 Sơ đồ nguyên lý kết nối tổng thể của Node 3

Khi tích hợp các thành phần trên như hình 3.29, ta có một sơ đồ nguyên lý tổng thể cho Node 3. Sự đồng bộ giữa phần cứng xử lý và mạch bảo vệ tín hiệu giúp khối vận hành ổn định trong suốt quá trình chạy thử nghiệm.

Khởi thực hiện phát định kỳ các gói tin lên đường CANBus với các ID cụ thể để bộ xử lý trung tâm nhận diện:

- CAN ID 0x103: mô phỏng nhiệt (Temp).

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

- CAN ID 0x105: Cập nhật thời gian (RTC).
- CAN ID 0x107: Mô phỏng xi nhan (Signal).

Chân	Ký hiệu	Kết nối STM32	Chức năng
1	+3V3	3.3V	Cấp nguồn hoạt động Công tắc, BMP180
2	VCC	5V	Cấp nguồn hoạt động MCP2551, DS1307
3	GND	GND	Mass chung
4	MTS_Left	A0	Dữ liệu ADC
5	MTS_Right	A1	Dữ liệu ADC
6	I2C1_SCL_1	B6	Dữ liệu I2C
7	I2C1_SDA_1	B7	Dữ liệu I2C
8	CAN_RX_3	A11	Dữ liệu CAN RX
9	CAN_TX_3	A12	Dữ liệu CAN TX

Bảng 3.10 Thông số kết nối của Node 3

3.2.4 Khối Giao tiếp CAN tổng

Do Raspberry Pi 5 không được tích hợp sẵn bộ điều khiển CAN phần cứng, hệ thống sử dụng module MCP2515 làm cầu nối giao tiếp. Module này kết nối với Raspberry Pi qua chuẩn SPI và sử dụng IC thu phát để dịch tín hiệu logic sang tín hiệu điện áp vi sai trên CAN Bus (CANH, CANL). Nhờ đó, cụm đồng hồ có thể tiếp nhận và giải mã liên tục dữ liệu từ khối thu thập hành vi.

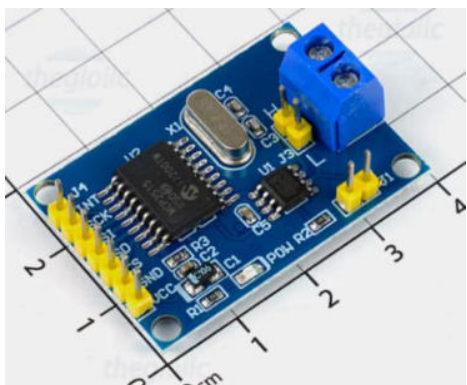
Module MCP2515 là mạch điều khiển mạng CAN độc lập tích hợp sẵn IC thu phát (thường là TJA1050), giao tiếp với hệ thống xử lý trung tâm thông qua chuẩn SPI.

Trong hệ thống IPC, module này đóng vai trò quan trọng trong việc bổ sung cổng CAN cho các bo mạch không tích hợp sẵn bộ điều khiển CAN, giúp hệ thống dễ dàng kết nối, đọc và xử lý các thông số vận hành của xe trên CAN Bus với độ ổn định cao.[22]

Thông số kỹ thuật:

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

- IC điều khiển CAN: MCP2515 (Microchip)
- IC thu phát CAN (Transceiver): TJA1050
- Chuẩn CAN hỗ trợ: CAN V2.0B (tốc độ lên đến 1 Mbps)
- Giao tiếp với vi điều khiển: Chuẩn SPI (tốc độ tối đa 10 MHz)
- Phạm vi nguồn cấp duy trì ổn định tại ngưỡng: 5V
- Dòng tối đa: 5mA
- Tính năng: Tích hợp bộ đệm nhận/truyền và chân ngắt (INT) phản cứng giúp giảm tải xử lý



Hình 3.30 Module MCP2515 [22]

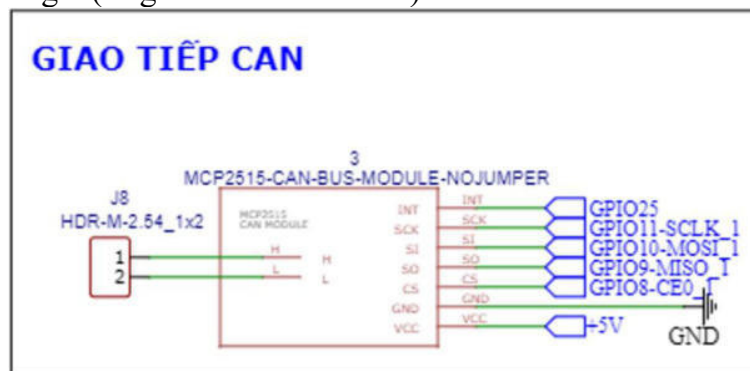
Chân	Tên	Chức năng
1	VCC	Nguồn cấp cho module (5V)
2	GND	Mass của hệ thống
3	CS	Chân chọn chip (Chip Select - SPI)
4	SO	Chân xuất dữ liệu SPI (MISO - Master In Slave Out)
5	SI	Chân nhận dữ liệu SPI (MOSI - Master Out Slave In)
6	SCK	Xung clock đồng bộ giao tiếp SPI
7	INT	Chân ngắt (Interrupt) báo hiệu khi nhận được dữ liệu CAN
Domino	CANH/CANL	Cặp tín hiệu vi sai kết nối vào mạng CAN bus

Bảng 3.11 Chức năng từng chân của Module MCP2515 [22]

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Việc sử dụng module MCP2515 (tích hợp TJA1050) là giải pháp tối ưu để bổ sung cổng CAN cho Raspberry Pi 5. Do hệ điều hành nhúng (Yocto Linux) phải xử lý các tác vụ đồ họa HMI nặng, MCP2515 đóng vai trò như một bộ đồng xử lý mạng giúp tự động phân loại, kiểm tra lỗi dữ liệu nhằm giảm tải CPU và tránh tràn bộ đệm.

Tuy nhiên, để hoàn thiện giao tiếp SPI tốc độ cao giữa bộ điều khiển mạng này và Raspberry Pi 5, hệ thống bắt buộc phải giải quyết bài toán xung đột điện áp thông qua khối chuyển đổi mức logic (Logic Level Converter).



Hình 3.31 Sơ đồ kết nối khối giao tiếp can tổng sử dụng module CAN MCP2515

Sự cần thiết của khối chuyển đổi mức logic (Logic Level Converter), thách thức lớn nhất khi ghép nối là sự chênh lệch mức điện áp: các chân GPIO của SoC Broadcom BCM2712 trên Raspberry Pi 5 chỉ chịu được mức 3.3V, trong khi module MCP2515 sử dụng mức logic 5V. Để tránh nguy cơ phá hủy cấu trúc phần cứng của máy tính nhúng, việc tích hợp mạch chuyển đổi mức logic hai chiều (Logic Level Converter) làm tầng đệm là bắt buộc.

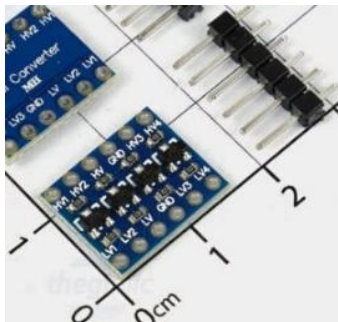
⇒ Vì vậy cần sử dụng mạch bảo vệ dòng hay khối chuyển đổi mức logic (Logic Level Converter)

Trong hệ thống IPC, Logic level Converter được sử dụng để đồng bộ tín hiệu giao tiếp (như UART, I2C, SPI) giữa Raspberry Pi 5 hoặc STM32 (hoạt động ở mức 3.3V) với các linh kiện, mạch giả lập hoạt động ở mức 5V. Việc này giúp dữ liệu truyền nhận luôn ổn định và bảo vệ các chân I/O của vi điều khiển khỏi hư hỏng do quá điện áp.[23]

Thông số kỹ thuật:

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

- Số kênh: 4 kênh chuyển đổi tín hiệu 2 chiều độc lập.
- Điện áp mức thấp (LV): 1.8V – 3.3V.
- Điện áp mức cao (HV): Lên đến 5V (HV phải lớn hơn LV).
- Giao tiếp hỗ trợ: UART, I2C, SPI, One-Wire và các tín hiệu logic rời rạc.
- Thành phần: Sử dụng các Mosfet N-Channel chuyển mạch tốc độ cao



Hình 3.32 Module Chuyển Mức Logic 4 Kênh [23]

Chân	Tên	Chức năng
1	LV (Low Voltage)	Chân cấp nguồn mức điện áp thấp (Ví dụ: 3.3V từ vi điều khiển)
2	HV (High Voltage)	Chân cấp nguồn mức điện áp cao (Ví dụ: 5V từ mạch ngoài)
3	GND	Chân nối Mass chung cho cả hai bên LV và HV
4	LV1 - LV4	Các chân tín hiệu đầu vào/ra ở mức điện áp thấp (3.3V)
5	HV1 - HV4	Các chân tín hiệu đầu vào/ra tương ứng ở mức điện áp cao (5V)

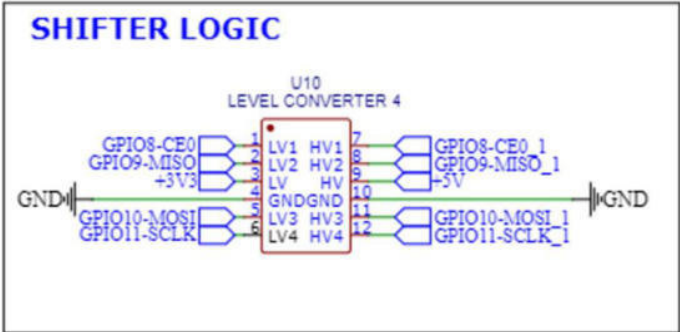
Bảng 3.12 Chức năng từng chân của Module Chuyển Mức Logic [23]

Để giải quyết xung đột này, mạch chuyển đổi mức điện áp Logic Level Converter được tích hợp để giao tiếp hai chiều an toàn:

- Dịch lên (Step-up): Các tín hiệu điều khiển SPI (SCLK, MOSI, CE0) từ Pi xuất ra ở 3.3V được nâng lên 5V để MCP2515 nhận diện chính xác.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

- Dịch xuống (Step-down): Dữ liệu trả về từ MCP2515 (SO) ở mức 5V được ghim xuống 3.3V tại chân MISO, đảm bảo an toàn tuyệt đối cho Raspberry Pi



Hình 3.33 Sơ đồ khối Shifter Logic

Raspberry Pi 5	Về điện áp thấp	Về điện áp cao	Module MCP2515	Chức năng giao tiếp (SPI)
Nguồn 3.3V	LV	HV	Nguồn 5V (VCC)	Cấp nguồn tham chiếu cho mạch
GND	GND	GND	GND	Nối mass chung toàn hệ thống
GPIO8 (CE0)	LV1	HV1	CS (Chip Select)	Chọn thiết bị giao tiếp
GPIO9 (MISO)	LV2	HV2	SO (Serial Out)	Nhận dữ liệu từ MCP2515 về Pi
GPIO10 (MOSI)	LV3	HV3	SI (Serial In)	Gửi dữ liệu từ Pi đến MCP2515
GPIO11 (SCLK)	LV4	HV4	SCK (Serial Clock)	Xung nhịp đồng bộ hệ thống

Bảng 3.13 Bảng kết nối chân giao tiếp giữa Pi, Logic Converter và MCP2515

3.2.5 Khối xử lý trung tâm

Khối sử dụng Raspberry Pi 5 là máy tính nhúng đơn bo, mang lại hiệu suất vượt trội gấp 2 đến 3 lần so với phiên bản tiền nhiệm nhờ được trang bị vi xử lý Broadcom BCM2712 (bốn nhân Cortex-A76, tốc độ 2.4GHz). Bo mạch sử dụng bộ nhớ RAM tốc độ cao với

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

dung lượng 8GB, kết hợp cùng chip xử lý I/O độc quyền RP1 giúp tối ưu hóa đáng kể băng thông truyền tải ngoại vi.

Về khả năng xử lý đồ họa, Raspberry Pi 5 hỗ trợ xuất hình ảnh chuẩn 4K với tần số quét 60Hz đồng thời trên hai màn hình thông qua cổng micro-HDMI kép. Đi kèm với đó là nhân đồ họa VideoCore VII mạnh mẽ, hỗ trợ cực kỳ tốt cho các ứng dụng có giao diện đồ họa phức tạp và đòi hỏi sự mượt mà.

Bên cạnh các kết nối tiêu chuẩn như USB 3.0, Gigabit Ethernet, Wi-Fi và Bluetooth 5.0, bo mạch còn được tích hợp giao tiếp PCIe 2.0 giúp mở rộng bộ lưu trữ tốc độ cao. Ngoài ra, sơ đồ 40 chân GPIO tiêu chuẩn vẫn được giữ nguyên, đảm bảo tính tương thích tối đa với các module phần cứng nhúng sẵn có.

Bảng 3.14 Thông số kỹ thuật của Raspberry Pi 5 [24]

Thông số	Chi tiết, chức năng
Điện áp	DC 5V – 5A qua đầu nối USB-C (Hỗ trợ USB PD) hoặc qua đầu cắm GPIO
CPU	Broadcom BCM2712, 4 nhân Cortex-A76 (ARM v8) 64-bit, tốc độ 2.4GHz
GPU	VideoCore VII hỗ trợ OpenGL ES 3.1, Vulkan 1.2
RAM	2GB, 4GB hoặc 8GB LPDDR4X-4267 SDRAM
Kết nối mạng	Gigabit Ethernet (hỗ trợ PoE+ qua HAT phụ), Wi-Fi băng tần kép 2.4GHz/5GHz 802.11ac, Bluetooth 5.0 / BLE
Cổng USB	2x USB 3.0 (băng thông 5Gbps đồng thời), 2x USB 2.0
Cổng Video	2x micro-HDMI (hỗ trợ xuất song song 2 màn hình 4K@60Hz)
Giao tiếp MIPI	2x cổng MIPI 4 làn kết hợp (Hỗ trợ linh hoạt cả Camera CSI và Màn hình DSI)
Lưu trữ	Khe cắm thẻ microSD hỗ trợ chế độ SDR104 tốc độ cao, giao tiếp PCIe 2.0 x1 cho ổ cứng NVMe

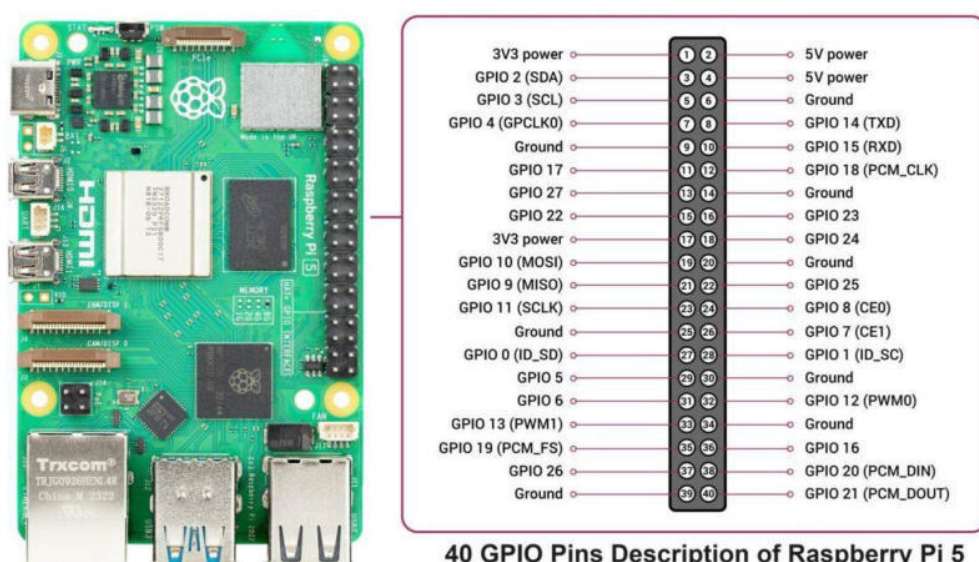
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Trong kiến trúc phần cứng của hệ thống Cụm đồng hồ kỹ thuật số (IPC), Raspberry Pi 5 là bộ não xử lý trung tâm, đảm nhiệm chạy hệ điều hành và giao diện đồ họa (HMI).

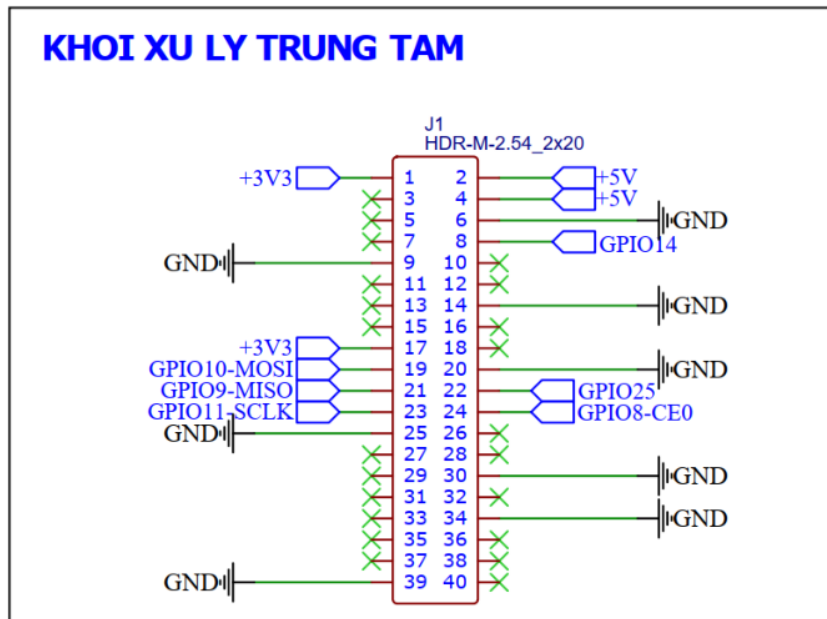
Hình 3.35 minh họa sơ đồ nguyên lý của khối giao tiếp trung tâm (ký hiệu J1) – một header 40 chân (chuẩn HDR-M-2.54_2x20) đồng nhất với sơ đồ GPIO của Raspberry Pi 5, đóng vai trò là trạm trung chuyển phân bổ nguồn điện và bus dữ liệu đến các ngoại vi.

Cấu hình các chân kết nối được quy hoạch cụ thể như sau:

- Cấp nguồn hệ thống: Sử dụng các chân 1, 17 (+3.3V) và chân 2, 4 (+5V) để trích xuất điện áp vận hành trực tiếp từ Raspberry Pi 5.
- Đồng bộ Mass và chống nhiễu: Các chân GND (6, 9, 14, 20, 25, 30, 34, 39) được phân bổ đều nhằm ổn định điện áp tham chiếu và triệt tiêu nhiễu xuyên âm trong môi trường xe hơi.
- Giao tiếp bus SPI: Cụm chân 19 (MOSI), 21 (MISO), 23 (SCLK) và 24 (CE0) thiết lập kết nối trực tiếp với module điều khiển CAN MCP2515.
- Cập nhật dữ liệu thời gian thực: Đảm bảo luồng thông số (vận tốc, vòng tua, nhiệt độ) từ CAN Bus được giải mã và hiển thị lên giao diện HMI liên tục, không có độ trễ



Hình 3.34 Sơ đồ chân của Raspberry Pi 5 [24]



Hình 3.35 Sơ đồ kết nối GPIO giao tiếp với Raspberry Pi

Số chân	Tên chân trên mạch	Chức năng trong hệ thống IPC
8	GPIO14	Xuất tín hiệu TX / Điều khiển phụ trợ
19	GPIO10-MOSI	Đường truyền dữ liệu Master Out Slave In (Bus SPI)
21	GPIO09-MISO	Đường nhận dữ liệu Master In Slave Out (Bus SPI)
22	GPIO25	Tín hiệu ngắt ngoài (Interrupt) nhận từ bộ điều khiển CAN
23	GPIO11-SCLK	Đường cấp xung nhịp đồng bộ Serial Clock (Bus SPI)
24	GPIO08-CE0	Chân chọn chip Chip Select (Bus SPI)

Bảng 3.15 Danh sách các chân tín hiệu sử dụng trên Header J1

- Cơ chế ngắt phần cứng: Chân 22 (GPIO25) tiếp nhận tín hiệu ngắt từ module CAN, giúp bộ xử lý trung tâm cập nhật dữ liệu tức thời thay vì tiêu tốn tài nguyên cho việc thăm dò (polling), đảm bảo đồ họa hiển thị luôn mượt mà.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

- Điều khiển phụ trợ: Chân 8 (GPIO14) được quy hoạch để xuất tín hiệu UART giả lập hoặc kích hoạt các module cảnh báo ngoại vi khi hệ thống phát hiện trạng thái vận hành bất thường.
- Tối ưu hóa thiết kế PCB: Các chân không kết nối (NC) được xác định rõ để lược bỏ đi dây dư thừa, tạo không gian cho việc đổ phủ đồng (copper pour) nhằm tăng cường khả năng chống nhiễu điện từ cho toàn mạch

3.2.6 Khối hiển thị

Khối sử dụng màn hình LCD 7 inch HDMI IPS cảm ứng điện dung - một thiết bị hiển thị chất lượng cao, được thiết kế đặc biệt để tương thích tốt với các hệ thống máy tính nhúng như Raspberry Pi, mini PC hoặc các bo mạch vi điều khiển hỗ trợ xuất hình ảnh.

Việc sử dụng cổng HDMI tiêu chuẩn giúp truyền tải hình ảnh độ phân giải cao mà không có độ trễ (zero latency), đáp ứng tốt các yêu cầu hiển thị đồ họa động, hoạt ảnh mượt mà ở tốc độ khung hình cao (thường là 60fps).

Thông số kỹ thuật

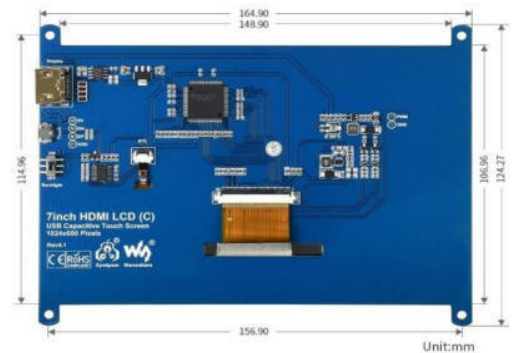
- Kích thước hiển thị: 7 inch.
- Độ phân giải: 1024 x 600 pixel (Hỗ trợ cấu hình nội suy lên các độ phân giải cao hơn qua phần mềm).
- Công nghệ tấm nền: IPS (Góc nhìn rộng, chống chói).
- Công nghệ cảm ứng: Điện dung đa điểm (Capacitive Touch - tối đa 5 điểm).
- Giao tiếp hiển thị: Chuẩn HDMI.
- Giao tiếp cảm ứng & Cấp nguồn: Chuẩn USB (Micro-USB hoặc Type-C tùy phiên bản board).
- Phạm vi nguồn cấp duy trì ổn định tại ngưỡng 3.3V – 5V: 5V DC (Cấp qua cổng USB).
- Hỗ trợ hệ điều hành: Raspbian (Raspberry Pi OS), Ubuntu, Windows 10/11, Yocto Linux,... không cần cài đặt thêm driver.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Màn hình LCD 7 inch IPS HDMI được lựa chọn nhằm đáp ứng yêu cầu hiển thị trực quan, sắc nét với góc nhìn rộng 178° trên cụm đồng hồ số IPC. Kích thước 7 inch vừa vặn để bố trí giao diện đồ họa HMI mà không chiếm quá nhiều diện tích taplo. Kết nối chuẩn HDMI và cảm ứng USB giúp tối ưu hóa phần cứng, đồng bộ trực tiếp với driver của Yocto Linux trên Raspberry Pi 5 mà không cần cài đặt thêm trình điều khiển phức tạp



Hình 3.36 Kích thước màn hình [26]



Hình 3.37 Mặt sau của màn hình [26]



Hình 3.38 Sơ đồ kết nối vật lý giữa Raspberry Pi 5 và Màn hình LCD [26]

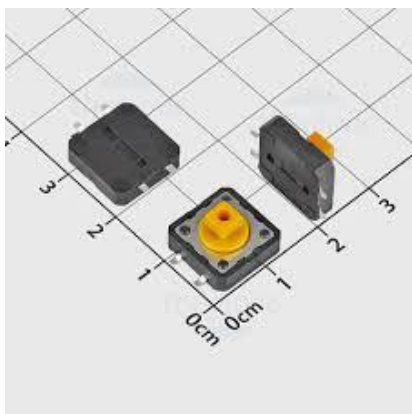
Trong cấu trúc phần cứng, màn hình LCD 7 inch đóng vai trò là giao diện tương tác trực tiếp (HMI) giữa hệ thống và người lái. Việc thiết kế khối này tập trung vào sự ổn định của luồng dữ liệu và tối giản hóa hạ tầng cấp nguồn: [26]

Màn hình LCD 7 inch đóng vai trò là giao diện hiển thị (HMI). Cơ chế truyền tải dữ liệu đồ họa qua cáp micro-HDMI từ Raspberry Pi 5 giúp triệt tiêu hiện tượng xé hình của các phần tử đồ họa động. Mạch điều khiển tận dụng trực tiếp dòng điện phân bổ qua cổng USB từ bo mạch Raspberry Pi 5 (vốn sử dụng nguồn chuẩn 5V-5A qua cổng Type-C).

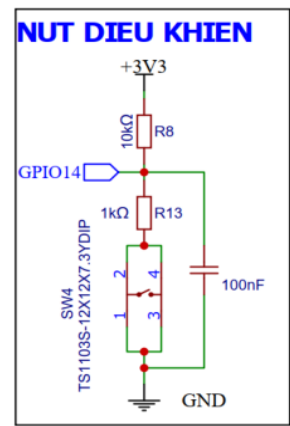
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Phương án cấp nguồn hợp nhất này giúp tối giản không gian kết cấu, loại bỏ các mạch hạ áp phụ trợ và giảm thiểu nhiễu điện từ cho hệ thống IPC.

Nút điều khiển



Hình 3.39 Nút 12x12mm [27]



Hình 3.40 Sơ đồ nguyên lý nút

Hình 3.39 và 3.40 cho thấy hình ảnh và sơ đồ mạch của khối nút nhấn điều khiển được kết nối trực tiếp với chân GPIO14. Khối này được thiết kế đóng vai trò như một ngoại vi, cho phép người dùng có thể tùy chỉnh linh hoạt chức năng trên giao diện hoặc cấu hình lại các cài đặt hệ thống sau này tùy theo nhu cầu thực tế trong quá trình vận hành [27]

3.2.7 Thông số tính toán khối nguồn cung cấp

Hệ thống sử dụng một nguồn cấp tổng có điện áp 5V dòng điện tối đa 5A. Toàn bộ các khối bao gồm Cụm đồng hồ số (Raspberry Pi 5, màn hình LCD, module MCP2515) và khối thu thập dữ liệu (3 Node STM32) đều sử dụng nguồn này.

Dựa vào tài liệu kỹ thuật (Datasheet) của nhà sản xuất, dòng điện tiêu thụ ở trạng thái hoạt động tối đa (Peak Current) của từng thành phần được thống kê như bảng 3.16:

STT	Linh kiện	SL	Điện áp	Dòng tiêu thụ	Tổng dòng tiêu thụ
1	Raspberry Pi 5	1	5 V	3000 mA	3000 mA
2	Màn hình LCD	1	5 V	≈ 700 mA	700 mA
3	STM32 (3 Node)	3	5 V	≈ 50 mA	250 mA

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

4	MCP2551(Node)	2	5 V	$\approx 70 \text{ mA}$	140 mA
5	TJA1050	1	5V	$\approx 50\text{mA}$	50mA
6	MCP2515	1	5 V	$\approx 60 \text{ mA}$	60 mA
7	BMP180	1	5 V	$<5 \text{ mA}$	5 mA
8	DS1307	1	3.3V	$< 5\text{mA}$	5mA
9	Mạch lọc, Biến trở, Công tắc.	Cụm	3.3 V	$\approx 15 \text{ mA}$	15 mA

Bảng 3.16 Thống kê dòng điện tiêu thụ trong hệ thống

Tổng dòng điện tiêu thụ của toàn hệ thống được tính bằng tổng dòng điện của các khối thành phần khi hoạt động ở công suất tối đa:

$$I_{\text{Tổng}} = I_{\text{Pi5}} + I_{\text{LCD}} + (3 \times I_{\text{STM32}}) + (2 \times I_{\text{MCP2551}}) + I_{\text{TJA1050}} + I_{\text{MCP2515}} + I_{\text{Cảm biến}} + I_{\text{ngoại vi}}$$

$$I_{\text{Tổng}} = 3000 + 700 + 250 + 140 + 50 + 60 + 5 + 15 = 4120(\text{mA}) = 4.12(\text{A}) \quad (3.9)$$

Đánh giá khả năng đáp ứng của nguồn điện 5V - 5A

Để một hệ thống phần cứng hoạt động an toàn và bền bỉ, nguồn cấp điện cần phải được thiết kế với một hệ số dự phòng an toàn (Safety Factor) tối thiểu là 15%-20% nhằm tránh hiện tượng nguồn bị quá tải, sinh nhiệt cao dẫn đến sụt áp dòng, được tính như sau:

$$\Delta I = I_{\text{Max}} - I_{\text{Tổng}} = 5 - 4.12 = 0.88(\text{A}) = 880(\text{mA}) \quad (3.10)$$

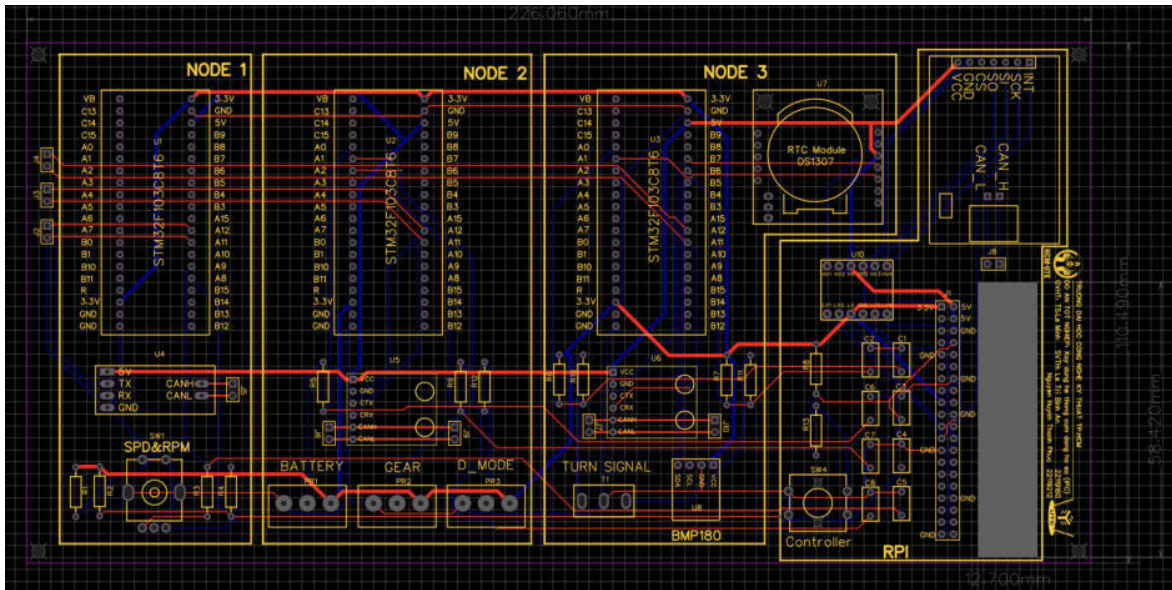
Tỷ lệ phần trăm mức dự phòng an toàn của bộ nguồn đạt:

$$\% \text{dự phòng} = \frac{0.88}{5} \times 100\% = 17.6\% \quad (3.11)$$

⇒ Kết luận: Với tổng dòng điện tiêu thụ cực đại của hệ thống là 4.12A, bộ nguồn 5V - 5A hoàn toàn đủ khả năng đáp ứng công suất cho toàn mạch hoạt động ổn định. Mức dự phòng 17.6% (880mA) đảm bảo an toàn cho toàn bộ hệ thống.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

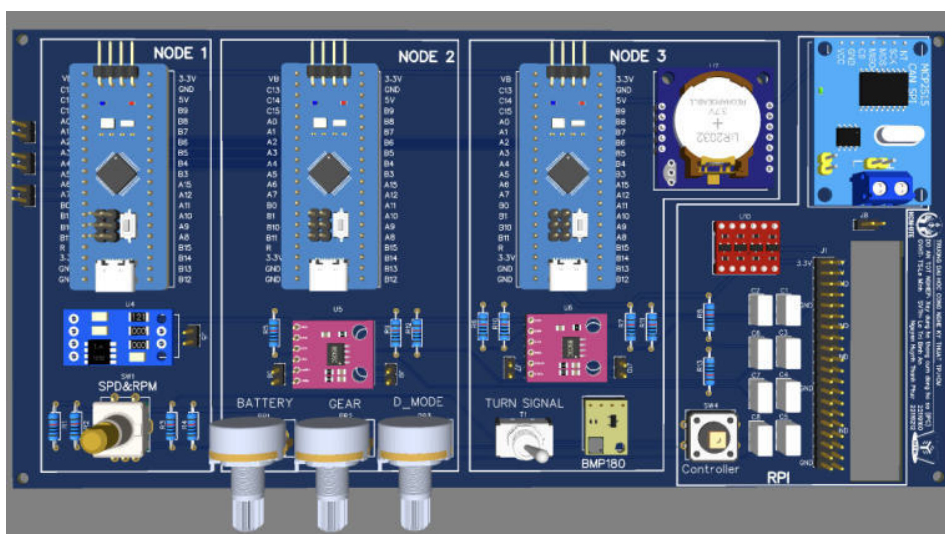
3.2.9 Thi công thiết kế mạch PCB



Hình 3.42 Sơ đồ bố trí linh kiện và đi dây (PCB Layout) của mạch.

Mạch PCB của đề tài IPC được thi công bám sát sơ đồ nguyên lý, đóng vai trò như một hệ thống giả lập mạng lưới ECU

Hình 3.42 và 3.43, đối chiếu với thực tế thi công ở, các linh kiện được hàn ráp và bố trí ba Node STM32, các cụm biến trở/nút nhấn, module CAN MCP2515 và cụm header giao tiếp với Raspberry Pi 5 đều đảm bảo khoảng cách vật lý an toàn, không cản trở nhau khi cắm rút dây, cung cấp mức áp 3.3V và 5V ổn định và mạch hoạt động.

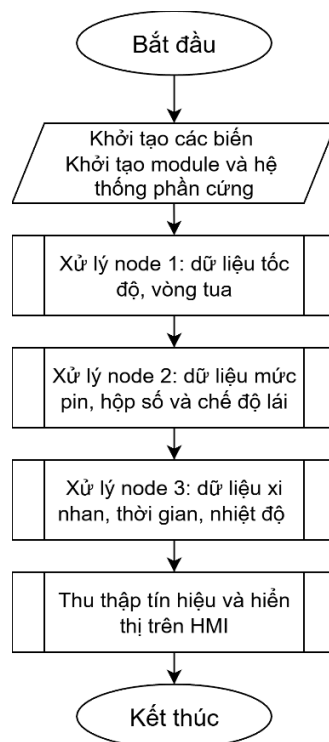


Hình 3.43 Thiết kế 3D của mạch PCB tổng thể

3.3 Thiết kế phần mềm

Dựa vào phần thiết kế phần cứng mà nhóm đã xây dựng lên các lưu đồ giải thuật toàn hệ thống và lưu đồ theo các chức năng của từng Node ECU (Node 1, Node 2, Node 3), truyền nhận CAN, thu thập và xử lý tín hiệu CAN và lưu đồ bộ điều khiển trung tâm.

3.3.1 Lưu đồ giải thuật toàn hệ thống



Hình 3.44 Lưu đồ toàn hệ thống

Lưu đồ tổng quát hệ thống mô tả toàn bộ chu trình khởi tạo và vận hành đồng thời của mô hình cụm đồng hồ số IPC. Quá trình xử lý được phân tách rõ ràng thành hai giai đoạn chính:

- Giai đoạn khởi tạo: Hệ thống thực hiện cấp nguồn, khởi tạo hệ thống phần cứng, thiết lập trạng thái ban đầu cho các ngoại vi và cấu hình giao thức mạng truyền thông CAN Bus với tốc độ bộ định thời 500kbps (giao diện can0), đảm bảo các đường truyền tín hiệu sẵn sàng hoạt động.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

- Giai đoạn vận hành (Vòng lặp hệ thống): Sau khi khởi tạo thành công, hệ thống phân tách thành các luồng xử lý song song và độc lập bao gồm ba nút cấu trúc (Node 1, Node 2, Node 3) đảm nhận nhiệm vụ thu thập dữ liệu, kết nối trực tiếp với trung tâm xử lý dữ liệu:
 - Xử lý node 1: Chịu trách nhiệm đo đặc Vận tốc và Vòng tua máy thông qua việc cấu hình bộ mã hóa vòng quay (Encoder) kết hợp với bộ định thời (TIM).
 - Xử lý node 2: Thực hiện thu thập các thông số về nguồn năng lượng bao gồm Pin, Số xe (Gear), và Chế độ lái (Drive Mode) bằng cách xử lý các tín hiệu tương tự thông qua bộ chuyển đổi dữ liệu (ADC).
 - Xử lý node 3: Đảm nhận việc đo đặc Nhiệt độ, cập nhật Thời gian hệ thống và điều khiển đèn hiệu Xi-nhan thông qua việc giao tiếp qua giao thức (I2C + GPIO).

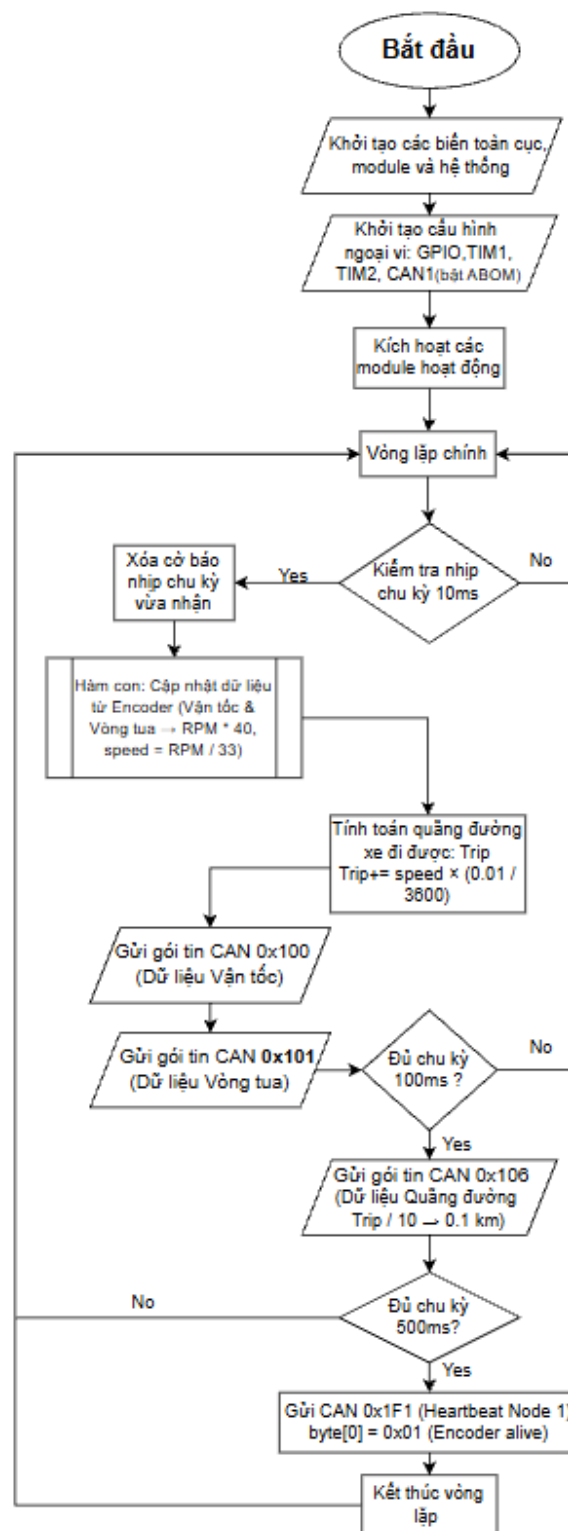
Dữ liệu từ ba Node ECU sau khi được xử lý và đóng gói sẽ liên tục truyền tải trên môi trường CAN Bus (500 kbps) dưới dạng các khung truyền dữ liệu dữ liệu mạng.

Thu thập tín hiệu và hiển thị với VCU Dashboard (Raspberry Pi 5) liên tục tiếp nhận gói tin từ bus CAN qua tiến trình CanReader. Dữ liệu sau đó được đồng bộ hóa vào mô hình VehicleModel và đẩy lên tầng giao diện QML. Quá trình này giúp hiển thị trực quan, theo thời gian thực toàn bộ trạng thái xe lên màn hình HMI (chạy trên nền tảng EGLFS) trước khi kết thúc một chu kỳ hoạt động.

3.3.2 Lưu đồ xử lý dữ liệu tốc độ vòng tua – node 1

Hàm thực hiện việc thu thập dữ liệu vận hành thông qua việc đọc tín hiệu từ bộ cảm biến đo xung (Encoder), kết hợp với thuật toán làm mượt dữ liệu và tích phân toán học theo thời gian thực để quy đổi thành các thông số trạng thái của xe. Sau đó, hệ thống tiến hành đóng gói và gửi các thông tin này lên đường truyền mạng điều khiển vùng (CAN bus) để màn hình hiển thị nhận diện và cập nhật giao diện. Hoạt động chính của hàm thể hiện qua hình 3.45 dưới đây.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG



Hình 3.45 Lưu đồ xử lý dữ liệu tốc độ vòng tua – node 1

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Ngay sau khi hệ thống được cấp nguồn, vi điều khiển thiết lập trạng thái ban đầu bằng việc khởi tạo các biến toàn cục, cấu hình module và hệ thống. Tiến trình này bao gồm:

- Cấu hình chế độ đọc xung tốc độ cao (Encoder Mode) trên bộ định thời TIM1 để đo vận tốc và vòng tua.
- Thiết lập chu kỳ ngắt định thời cho TIM2 để làm nhịp xung chuẩn cho hệ thống.
- Thiết lập tham số truyền thông cho bộ điều khiển mạng CAN 1, đồng thời kích hoạt tính năng tự động phục hồi từ trạng thái lỗi (ABOM - Automatic Bus-Off Management).

Khi các bước thiết lập phần cứng hoàn tất, toàn bộ các module được kích hoạt đồng bộ để bắt đầu tiến vào vòng lặp chính While(1).

Cơ chế Quản lý tiến trình theo Thời gian thực (Chu kỳ 10ms)

Trong cấu trúc vòng lặp vô hạn while(1), tiến trình vận hành của hệ thống được điều phối thông qua cơ chế giám sát thời gian thực dựa trên việc kiểm tra liên tục chu kỳ 10ms (vốn được cập nhật tự động từ ngắt của TIM2). Quy trình xử lý logic điều hướng theo hai nhánh cụ thể:

- Nhánh No (Chu kỳ chưa hoàn tất): Nhằm tối ưu hóa hiệu suất xử lý và tiết kiệm tài nguyên tính toán, vi điều khiển sẽ thiết lập một vòng phản hồi để liên tục kiểm tra nhịp thời gian, đồng thời trì hoãn việc thực thi toàn bộ các khối lệnh phía dưới.
- Nhánh Yes (Chu kỳ hoàn tất): Khối tính toán logic được kích hoạt ngay khi cờ báo nhịp chu kỳ nhận giá trị thỏa mãn. Đồng thời, hệ thống thực hiện hoàn trả cờ báo này về trạng thái mặc định, hoàn tất công tác chuẩn bị để đón nhận chu kỳ kế tiếp.

Xử lý Thuật toán và Giám sát Trạng thái Đường truyền

Sau khi vượt qua chốt chặn 10ms, hệ thống gọi hàm con cập nhật dữ liệu từ Encoder để tính toán Vận tốc và Vòng tua máy theo các công thức thực nghiệm:

$$\text{Vòng tua (RPM)} = \text{Xung đọc được} \times 40 \quad (3.11)$$

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

$$\text{Vận tốc (Speed)} = \text{Speed} \times \frac{0.01}{3600} \quad (3.12)$$

Giá trị sau khi tính toán sẽ được xử lý để làm mượt, tránh hiện tượng nhảy bậc dữ liệu. Ngay sau đó, hệ thống thực hiện thuật toán tích phân cộng dồn để tính toán Quãng đường xe đi được (Trip) dựa trên phân đoạn thời gian chu kỳ 10ms vừa trôi qua theo công thức:

$$\text{Trip} += \text{Speed} \times \frac{0.01}{3600} \quad (3.13)$$

Cơ chế Phân tầng Phân phối Dữ liệu trên mạng CAN

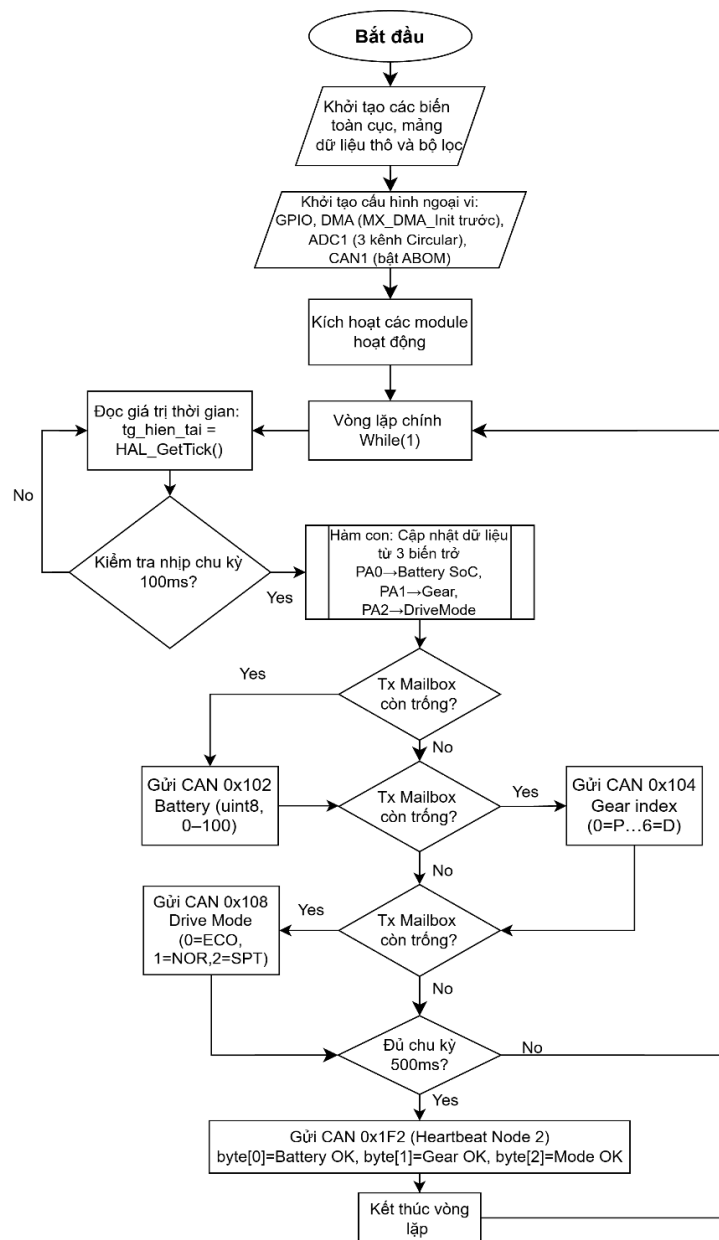
Để tối ưu hóa băng thông đường truyền và tránh nghẽn mạch CAN bus, các gói tin dữ liệu sau khi xử lý xong sẽ được phân tầng và phát đi theo các chu kỳ thời gian khác nhau:

- Chu kỳ 10ms (Mỗi nhịp): Gói tin dữ liệu Vận tốc (ID 0x100) và dữ liệu Vòng tua (ID 0x101) yêu cầu độ nhảy cao nên luôn được đóng gói và gửi đi liên tục tại mỗi chu kỳ 10ms.
- Chu kỳ 100ms: Hệ thống kiểm tra điều kiện tích lũy thời gian. Nếu đủ mốc 100ms (Nhánh Yes), vi điều khiển sẽ chuyển đổi đơn vị dữ liệu Quãng đường ($\text{Trip}/10 \rightarrow 0.1\text{km}$) và đóng gói gửi đi dưới dạng gói tin CAN có ID 0x106. Nếu chưa đủ mốc (Nhánh No), khối này sẽ bị bỏ qua và chuyển trực tiếp xuống bước kế tiếp.
- Chu kỳ 500ms: Cuối cùng, hệ thống kiểm tra điều kiện nhịp 500ms để gửi gói tin kiểm tra trạng thái hoạt động (Heartbeat Node 1) với ID 0x1F1 và dữ liệu $\text{byte}[0] = 0x01$ (Encoder alive) nhằm báo cáo cho khối trung tâm biết Node 1 vẫn đang vận hành ổn định.

Sau khi thực hiện xong tất cả các bước kiểm tra chu kỳ và gửi tin, luồng chương trình kết thúc một vòng lặp và quay ngược về đỉnh vòng lặp *While(1)* để chờ nhịp thời gian 10ms kế tiếp.

3.3.3 Lưu đồ xử lý dữ liệu mức pin, hộp số và chế độ lái - Node 2

Node 2 thực hiện việc thu thập dữ liệu vận hành thông qua việc đọc tín hiệu điện áp từ các biến trở mô phỏng, kết hợp với các bộ lọc kỹ thuật số và thuật toán nội suy tuyến tính để quy đổi thành thông số trạng thái của Pin, vị trí số xe và chế độ lái. Sau đó, vi điều khiển thực hiện đóng gói và truyền tải các thông tin này lên mạng CAN như hình 3.46:



Hình 3.46 Lưu đồ xử lý dữ liệu mức pin, hộp số và chế độ lái - Node 2

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Hệ thống tiến hành cấu hình phần cứng: thiết lập các chân GPIO, cấu hình bộ chuyển đổi ADC1 quét 3 kênh ở chế độ vòng lặp, thiết lập bộ dịch chuyển dữ liệu tự động DMA1 và cấu hình bộ điều khiển mạng CAN1 (bật chế độ ABOM). Sau khi thiết lập, toàn bộ các module được kích hoạt đồng bộ để bắt đầu chu trình xử lý trong vòng lặp chính While(1).

Bước vào vòng lặp vô hạn While(1), vi điều khiển liên tục đọc giá trị thời gian hệ thống bằng hàm HAL_GetTick(). Khối điều kiện tiến hành kiểm tra nhịp chu kỳ 100ms:

- Nhánh No (Chưa đủ thời gian): Luồng chương trình quay ngược lại để tiếp tục kiểm tra, duy trì vòng lặp chờ nhằm đảm bảo tính thời gian thực.
- Nhánh Yes (Đạt mốc 100ms): Hệ thống cho phép mở khóa luồng dữ liệu để tiến vào hàm con cập nhật và xử lý tín hiệu cảm biến.

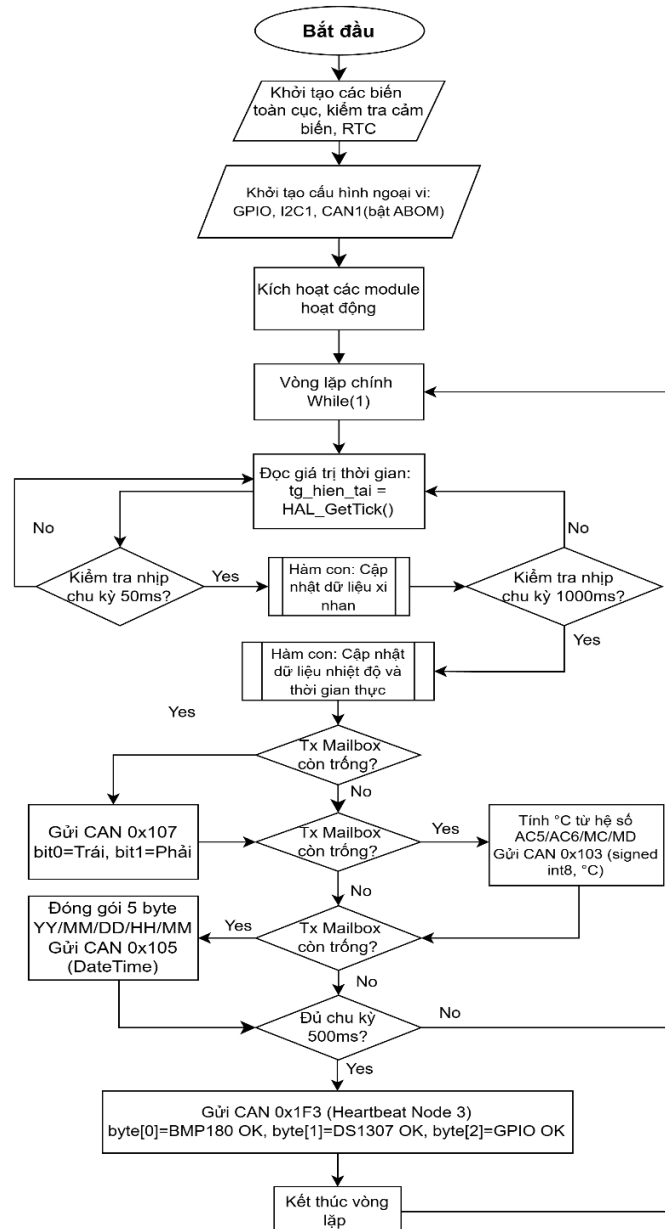
Xử lý Thuật toán và Giám sát Đường truyền. Chương trình sẽ gọi hàm con chức năng để truy xuất dữ liệu từ 3 biến trở mô phỏng: chân PA0 thu thập dung lượng Pin (Battery SoC), chân PA1 xác định vị trí Số xe (Gear), và chân PA2 ghi nhận Chế độ lái (DMode).

Kiểm tra TxMailbox và Phát tín hiệu lên mạng CAN. Để phát các thông số ra Bus mạng mà không gây tràn bộ đệm, hệ thống áp dụng cơ chế kiểm tra tăng nấc trạng thái trống của các hộp thư truyền dữ liệu (Tx Mailbox):

- Gói tin Battery (ID 0x102, giá trị 0 → 100) được kiểm tra; nếu Mailbox trống (Nhánh Yes), gói tin sẽ được phát đi.
- Tiếp theo, hệ thống kiểm tra Mailbox trống để lần lượt đóng gói và phát gói tin Gear index (ID 0x104, các vị trí P R N D) và gói tin Drive Mode (ID 0x108, các chế độ 0=ECO, 1=NOR, 2=SPT). Nếu tại bất kỳ bước nào Mailbox bị bận (Nhánh No), gói tin tương ứng sẽ bị bỏ qua để chống nghẽn mạch.
- Cuối cùng, khối logic kiểm tra chu kỳ 500ms. Nếu đủ mốc (Nhánh Yes), gói tin Heartbeat Node 2 (ID 0x1F2) mang các byte dữ liệu báo trạng thái hoạt động tốt (Battery OK, Gear OK, Mode OK) sẽ được gửi đi trước khi kết thúc một vòng lặp hệ thống.

3.3.4 Lưu đồ dữ liệu xi nhan, thời gian, nhiệt độ - Node 3

Node 3 thực hiện chức năng thu thập dữ liệu nhiệt độ và đồng bộ thời gian thực của hệ thống thông qua việc giao tiếp với các cảm biến, kết hợp tính toán hiệu chuẩn thông số vật lý từ các hệ số định sẵn. Sau khi chuẩn hóa dữ liệu, vi điều khiển tiến hành phân tãg và truyền tải các gói tin này lên mạng CAN bus theo các chu kỳ độc lập như hình 3.47:



Hình 3.47 Lưu đồ dữ liệu xi nhan, thời gian, nhiệt độ - Node 3

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Chu trình bắt đầu ngay khi vi điều khiển được cấp nguồn. Hệ thống thực hiện cấp phát vùng nhớ cho các biến toàn cục, mảng lưu trữ cấu trúc thời gian và cấu hình các khối ngoại vi phần cứng bao gồm: các chân GPIO chức năng, bộ giao tiếp dữ liệu nối tiếp I2C1 để kết nối với các IC ngoại vi, và bộ điều khiển mạng diện rộng CAN1 với tính năng tự động phục hồi lỗi đường truyền (ABOM). Sau khi các thiết lập phần cứng được xác lập thành công, toàn bộ các module được chuyển sang trạng thái hoạt động và chương trình tiến vào vòng lặp điều khiển trung tâm While(1).

Quản lý đa tiến trình và đồng bộ thời gian thực

Bên trong vòng lặp chính While(1), hệ thống thực hiện kỹ thuật chia thời gian (Time-slicing) bằng cách liên tục lấy mốc thời gian hệ thống qua hàm HAL_GetTick(). Tiến trình xử lý tại đây được phân tách thành hai nhánh độc lập nhằm tối ưu hóa tần suất đáp ứng của các loại dữ liệu khác nhau:

- Nhánh kiểm tra chu kỳ 50ms: Thực hiện quét tần suất cao để phục vụ tác vụ kiểm tra trạng thái và điều khiển đèn xi-nhan. Nếu đạt mốc (Nhánh Yes), hàm con cập nhật dữ liệu xi-nhan sẽ được kích hoạt.
- Nhánh kiểm tra chu kỳ 1000ms: Thực hiện quét chu kỳ dài để cập nhật các thông số môi trường và thời gian. Khi điều kiện thời gian thỏa mãn (Nhánh Yes), vi điều khiển gọi hàm con cập nhật dữ liệu nhiệt độ và thời gian thực.
- Nếu chưa đạt các mốc thời gian tương ứng (Nhánh No), luồng dữ liệu của nhánh đó bị bỏ qua để giảm tải cho CPU.

Xử lý dữ liệu cảm biến và thực thi thuật toán hiệu chuẩn

Tại các hàm con chuyên biệt, quá trình thu thập và biến đổi toán học được thực hiện tuần tự:

- Tín hiệu điều hướng: Trạng thái logic từ các chân GPIO kết nối với công tắc được đọc và chuyển đổi trực tiếp thành cấu trúc bit đại diện cho đèn báo rẽ bên trái (bit0) và bên phải (bit1).

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

- Thông số nhiệt độ: Vi điều khiển đọc giá trị điện áp số hóa từ cảm biến BMP180 qua bus I2C. Dữ liệu thô này không được sử dụng trực tiếp mà phải chạy qua thuật toán hiệu chuẩn dựa trên các hệ số cố định của nhà sản xuất bao gồm \$AC5, AC6, MC, MD\$ để nội suy ra giá trị nhiệt độ chính xác với đơn vị độ C (signed int8, °C).
- Thông số thời gian: Dữ liệu ngày, tháng, năm, giờ, phút được truy xuất trực tiếp từ chip thời gian thực DS1307 theo định dạng chuẩn, sau đó được đóng gói thành cấu trúc chuỗi dữ liệu gồm 5 byte (YY/MM/DD/HH/MM).

Kiểm tra trạng thái Mailbox và phân phối dữ liệu lên CAN Bus

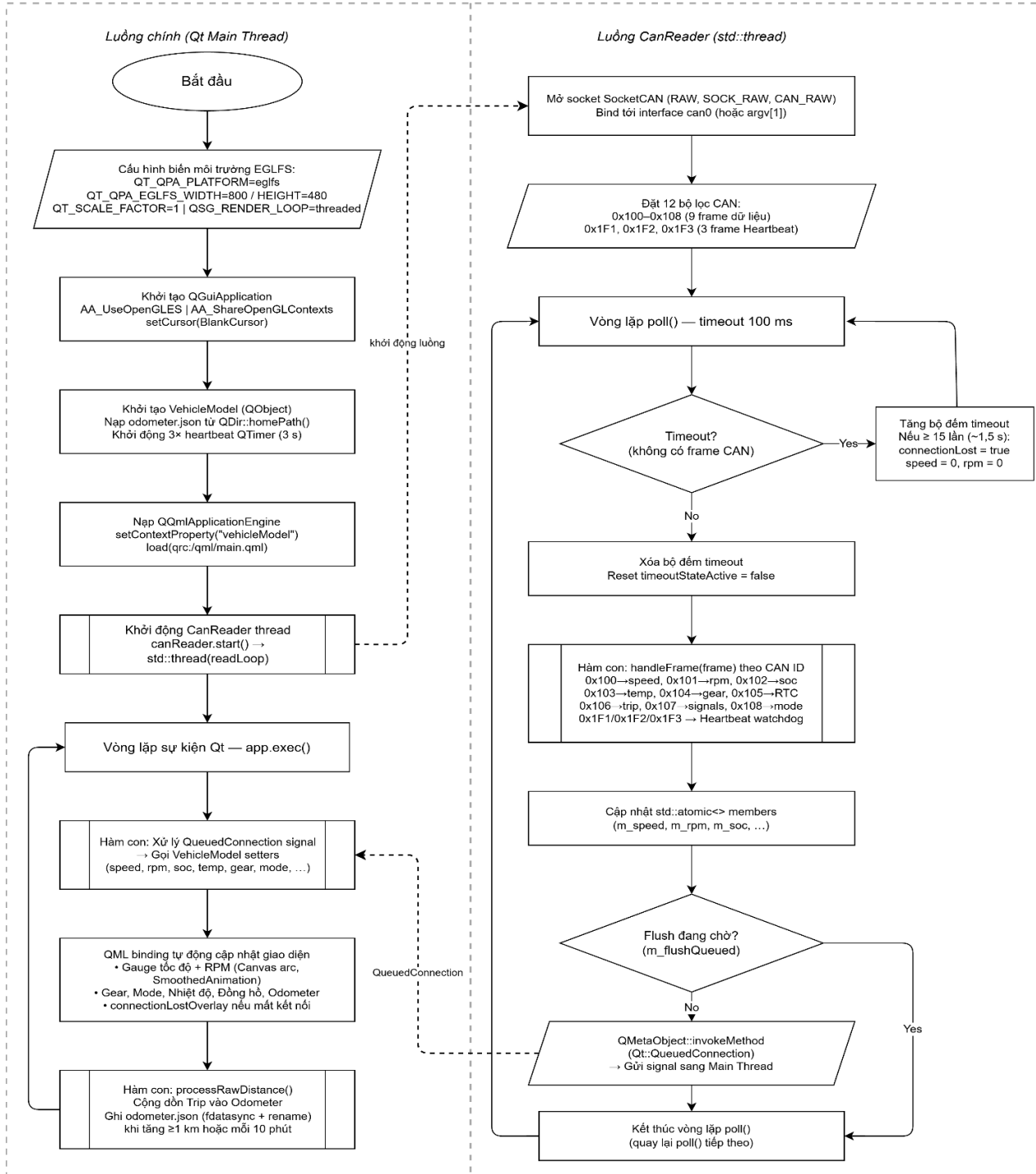
Sau khi dữ liệu được làm sạch và chuẩn hóa, hệ thống sử dụng cấu trúc điều kiện lồng nhau để kiểm tra trạng thái của các hộp thư truyền dữ liệu phần cứng (Tx Mailbox) trước khi đẩy gói tin ra môi trường truyền dẫn, tránh hiện tượng ghi đè hoặc tràn bộ đệm:

- Gói tin Xi-nhan (ID 0x107) được nạp vào Mailbox nếu trống và phát đi lập tức.
- Tiếp theo, nếu Mailbox tiếp theo sẵn sàng (Nhánh Yes), gói tin Nhiệt độ (ID 0x103) và gói tin Thời gian thực (ID 0x105) lần lượt được cấu hình cấu trúc khung và truyền lên trục bus dữ liệu.
- Cuối cùng, khối logic kiểm tra mốc thời gian tổng chu kỳ 500ms. Nếu đủ điều kiện (Nhánh Yes), gói tin Heartbeat Node 3 (ID 0x1F3) tích hợp các byte báo trạng thái phần cứng hoạt động ổn định (BMP180 OK, DS1307 OK, GPIO OK) sẽ được truyền tải trên mạng CAN, kết thúc một chu trình vận hành đồng bộ và quay lại đỉnh vòng lặp While(1).

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.3.5 Lưu đồ thu thập tín hiệu và hiển thị trên HMI

Để đảm bảo hiệu suất hiển thị hình ảnh ở tần suất cao và duy trì tính toàn vẹn của dữ liệu CAN bus, khối trung tâm VCU Dashboard áp dụng kiến trúc song song đa luồng như hình 3.48:



Hình 3.48 Lưu đồ thu thập tín hiệu và hiển thị trên HMI

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Để đạt hiệu suất hiển thị hình ảnh ở tần suất cao và duy trì tính toàn vẹn của dữ liệu CAN bus, khối trung tâm VCU Dashboard áp dụng kiến trúc đa luồng song song. Cơ chế này phân tách hoàn toàn giữa luồng xử lý giao diện đồ họa và luồng truyền thông mạng độc lập nhằm triệt tiêu độ trễ hệ thống.

Hiệu suất hiển thị giao diện đạt tần suất 60 FPS đòi hỏi luồng đồ họa chính phải được giải phóng khỏi các tác vụ nghẽn I/O. Do đó, hệ thống cô lập tiến trình lắng nghe mạng vào một luồng độc lập, thiết lập liên kết thông qua cơ chế an toàn bộ nhớ (Thread-safe).

Ngay khi khởi chạy, hệ thống cấu hình trực tiếp nền tảng **EGLFS** để bỏ qua các lớp quản lý cửa sổ trung gian của hệ điều hành. Quyết định này cho phép ứng dụng vẽ trực tiếp dữ liệu lên bộ đệm khung hình (Framebuffer) của GPU với các tham số tối ưu (HEIGHT=480, WIDTH=800). Lớp dữ liệu trung tâm VehicleModel thừa kế từ QObject thực hiện nạp dữ liệu quãng đường tích lũy từ tệp định dạng JSON odometer.json định vị trong bộ nhớ Flash.

Sau khi kích hoạt 3 bộ định thời giám sát (*Heartbeat QTimer*), luồng chính sử dụng *QQmlApplicationEngine* để ánh xạ mô hình trạng thái vào tầng hiển thị giao diện đồ họa main.qml. Luồng chính ra lệnh khởi động tiến trình đọc dữ liệu thông qua hàm gọi *canReader.start()* và chính thức tiến vào vòng lặp quản lý sự kiện bộ nền *app.exec()*.

Chu trình giám sát mạng của luồng nền (CanReader Thread)

Luồng *CanReader* vận hành hoàn toàn tách biệt dưới dạng một luồng hệ điều hành (*std::thread*). Tại giai đoạn khởi tạo, tiến trình mở một cổng SocketCAN lớp mạng thô với cấu hình thuộc tính: *SocketCAN (RAW, SOCK_RAW, CAN_RAW)*

Socket này liên kết trực tiếp tới giao diện phần cứng *can0* của Linux. Để tối ưu hóa tài nguyên CPU, luồng cấu hình một mảng lọc gồm 12 phần tử để chỉ tiếp nhận nhóm gói tin dữ liệu phương tiện (0x100 → 0x108) và nhóm gói tin giám sát hoạt động (0x1F1 → 0x1F3).

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Tại nhân xử lý, luồng nền vận hành một vòng lặp kiểm tra liên tục dựa trên lệnh hệ thống *poll()* với cơ chế chốt chặn thời gian (*Timeout*) cấu hình cố định ở mức 100ms:

- Trường hợp xảy ra Timeout (Nhánh Yes): Xác nhận không có khung dữ liệu CAN trên Bus, hệ thống lũy tiến giá trị đếm bộ đệm. Nếu tổng số lần Timeout liên tiếp đạt ngưỡng 15 lần (tương đương khoảng thời gian cô lập 1.5s), khối điều khiển kết luận mạng bị gián đoạn, biến cờ hiệu *connectionLost* đẩy lên mức logic true, đồng thời hạ toàn bộ giá trị tốc độ và vòng tua máy về mức 0.
- Trường hợp nhận được gói tin (Nhánh No): Bộ đệm Timeout lập tức reset (*timeoutStateActive = false*). Hệ thống chuyển đổi luồng dữ liệu sang hàm *con handleFrame(frame)* để bóc tách các byte dữ liệu thô từ mạng CAN theo cấu trúc ma trận định danh ID:
 - o ID 0x100 Vận tốc (*speed*), ID 0x101 Vòng tua máy (*rpm*), ID 0x106 Quãng đường ngắn hạn (*trip*), ID 0x1F1 giám sát Node 1 (Heartbeat Node 1).
 - o ID 0x102 Dung lượng Pin (*soc*), ID 0x104 Vị trí số xe (*gear*), ID 0x108 Chế độ lái phương tiện (*mode*), ID 0x1F2 giám sát Node 2 (Heartbeat Node 2).
 - o ID 0x103 Nhiệt độ (*temp*), ID 0x105 Thời gian thực (*rtc*), ID 0x107 xi nhan ID 0x1F3 giám sát Node 3 (Heartbeat Node 3).

Dữ liệu thô sau khi giải mã toán học được lưu trữ trực tiếp vào các biến thành viên có thuộc tính an toàn đơn nguyên *std::atomic* (như *m_speed*, *m_rpm*, *m_soc*) nhằm triệt tiêu hiện tượng tranh chấp dữ liệu (*Race condition*) giữa các luồng.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Sau bước cập nhật biến, luồng nền kiểm tra trạng thái bộ đệm đầy dữ liệu (*m_flushQueued*). Nếu không có yêu cầu chặn (Nhánh No), luồng CanReader thực hiện lệnh gọi hàm bất đồng bộ xuyên luồng *QMetaObject::invokeMethod* với phương thức cấu hình kết nối xếp hàng *Qt::QueuedConnection*. Cơ chế này đóng gói dữ liệu và đẩy một tín hiệu (Signal) vào hàng đợi sự kiện của Luồng chính một cách an toàn.

Tại luồng chính, khi nhận được tín hiệu từ hàng đợi, hàm con xử lý tín hiệu xếp hàng kích hoạt các hàm thiết lập (*Setters*) của *VehicleModel* để cập nhật lại các thuộc tính dữ liệu. Do tầng giao diện đồ họa QML cấu hình cơ chế liên kết động (Binding) trực tiếp với các thuộc tính này, mọi sự thay đổi về giá trị số học lập tức thay đổi trạng thái hình học của các phần tử đồ họa trên màn hình HMI:

- Giá trị vận tốc và vòng tua máy điều khiển các đường cong Canvas Arc và thuật toán làm mượt đồ họa SmoothedAnimation chuyển động tuyến tính.
- Các ký tự hiển thị về Hộp số, Chế độ lái, Nhiệt độ và Đồng hồ thời gian thực thay đổi trạng thái tức thời.
- Nếu cờ lỗi kết nối mạng bật, giao diện tự động phủ một lớp cảnh báo mất tín hiệu (*connectionLostOverlay*).

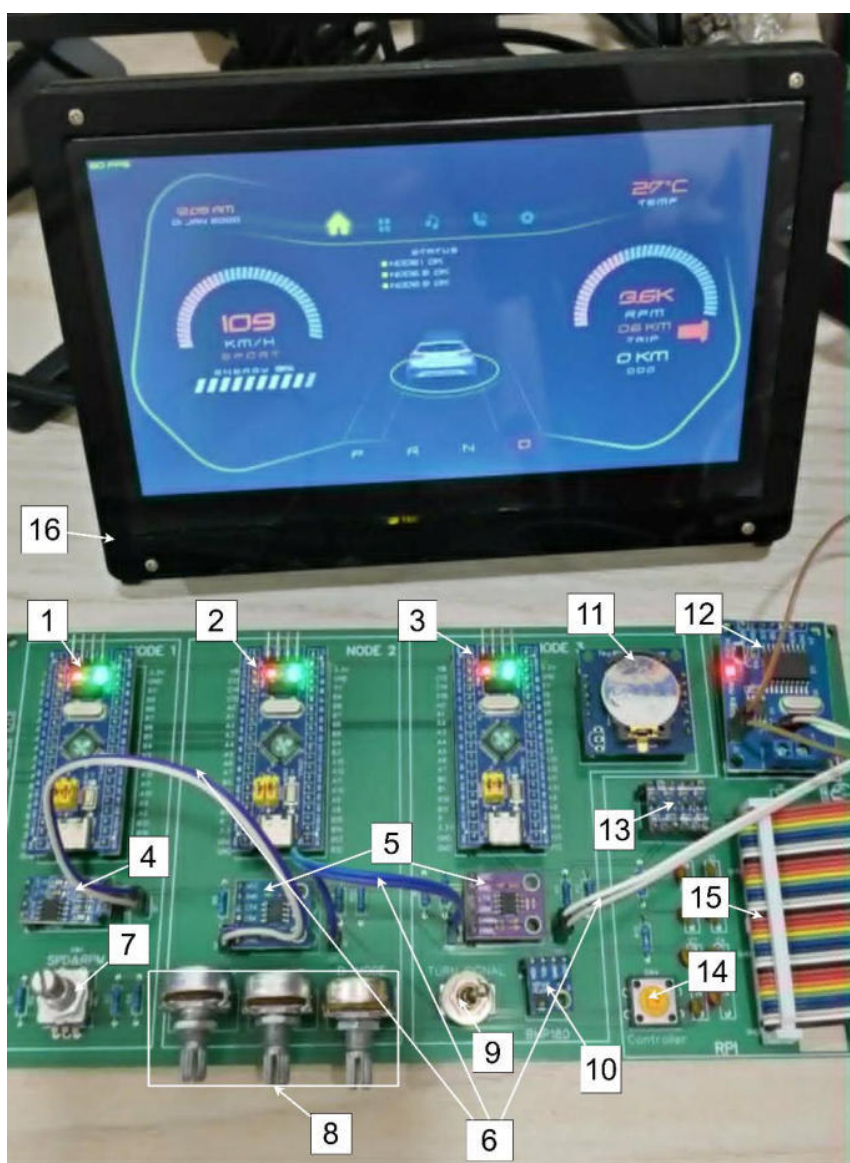
Song song với quá trình kết xuất đồ họa, hàm con tích hợp *processRawDistance()* của luồng chính liên tục thực hiện thuật toán cộng dồn giá trị biến quãng đường ngắn hạn (Trip) vào tổng biến quãng đường di chuyển của phương tiện (Odometer). Định kỳ khi tổng quãng đường tăng thêm ≥ 1 km hoặc sau mỗi phân đoạn thời gian 10 phút, hệ thống thực thi ghi đồng bộ dữ liệu mới đề vào tệp cấu hình *odometer.json* bằng các lệnh gọi an toàn hệ thống (*fdatasync + rename*), đảm bảo dữ liệu hành trình không bị sai lệch ngay cả khi hệ thống trung tâm mất nguồn đột ngột. Vòng lặp kết thúc một chu kỳ hoạt động và quay trở lại trạng thái chờ của lệnh *poll()*.

CHƯƠNG 4: KẾT QUẢ HỆ THỐNG

4.1 Giới thiệu

Trong chương này là quá trình đánh giá kết quả, kiểm tra phần cứng và mạch điện. Thực thi phần mềm để đưa ra những hình ảnh trực quan về hoạt động chức năng của toàn bộ hệ thống xem có đáp ứng đúng yêu cầu của đề tài không

4.2 Kết quả phần cứng và màn hình hiển thị



Hình 4.1 Phần cứng và màn hình hiển thị đồng hồ IPC

CHƯƠNG 4: KẾT QUẢ HỆ THỐNG

Thiết kế cụm đồng hồ số IPC (Hình 4.1) gồm hai lớp độc lập: phần cứng giả lập (phía dưới) và giao diện kết xuất HMI (phía trên). Ba vi điều khiển STM32F103C8T6 đóng vai trò là các Node ECU đảm nhận nhiệm vụ thu thập ngoại vi và đóng gói dữ liệu vận hành. Các gói tin này sau đó được truyền nhận qua mạng CAN Bus để gửi lên bộ xử lý trung tâm Raspberry Pi và hiển thị trực quan trên màn hình. Dưới là mô tả thành phần:

STT	Mô tả
1	Vi điều khiển STM32F103C8T6 – Node 1
2	Vi điều khiển STM32F103C8T6 – Node 2
3	Vi điều khiển STM32F103C8T6 – Node 3
4	Module CAN bus TJA1050 – Node 1
5	2 × Module CAN bus MCP2551 – Node 2 và Node 3
6	Đường truyền CANBus giữa các module CAN
7	Encoder EC11
8	3× Biến trở 10K
9	Công tắc gạt MTS - 103
10	Module nhiệt độ BMP180
11	Module RTC DS1307
12	Module CAN bus MCP2515
13	Logic Level Converter 4 kênh
14	Nút nhấn
15	Đầu header 40 pin nối với Raspberry Pi 5
16	Màn hình LCD 7 inch

Bảng 4.1 Mô tả thành phần phần cứng hệ thống

CHƯƠNG 4: KẾT QUẢ HỆ THỐNG

Ở giao diện HMI, sử dụng màn hình LCD hiển thị theo thời gian thực. Các thông số vận hành như dải vòng tua, hộp số (P-R-N-D), mức pin và nhiệt độ được hiển thị trên màn hình như một đồng hồ số trên xe. Bảng theo dõi trạng thái của các Node tích hợp trên GUI liên tục báo xanh (OK) cho toàn bộ 3 Node, đảm bảo vai trò xác nhận các Node Can ECU giao tiếp mạng CAN có đang duy trì kết nối ổn định và triệt tiêu hoàn toàn rủi ro trên đường truyền hay bị mất tín hiệu. Dưới đây là thành phần trong màn hình giao diện HMI hình 4.2:



Hình 4.2 Giao diện HMI - IPC

STT	Mô tả
1	Vòng cung hiển thị tốc độ
2	Giá trị tốc độ hiện tại của xe – km/h
3	Chế độ lái hiện tại của hệ thống (Eco / Normal / Sport)
4	Dung lượng pin và mức tiêu thụ năng lượng
5	Vị trí hộp số của xe (P - R - N - D)
6	Tổng quãng đường xe di chuyển lũy (ODO)
7	Quãng đường di chuyển hiển thị theo chặng (TRIP)
8	Giá trị vòng tua (RPM)

CHƯƠNG 4: KẾT QUẢ HỆ THỐNG

Hình 4.3

9	Vòng cung hiển thị vòng tua
10	Thời gian thực và ngày tháng hiển thị hệ thống
11	Giá trị hiển thị nhiệt độ môi trường xung quanh (°C)
12	Đèn tín hiệu xi-nhan
13	Khung giám sát trạng thái kết nối hoạt động của các Node mạng CAN, các thiết bị mô phỏng (STATUS NODE 1/2/3)
14	Khung hình trên giây (FPS Counter)

Bảng 4.2 Mô tả thành phần giao diện HMI

4.3 Kết quả mô phỏng và hiển thị Node 1

Dữ liệu Node 1 bao gồm tốc độ vòng tua được mô phỏng từ Encoder EC11 kết nối với STM32F103C8T6. Các ID dữ liệu CAN của Node 1 bao gồm:

CAN ID	DLC	Dữ liệu
0x100	2	Tốc độ (Speed)
0x101	8	Vòng tua máy (RPM)
0x106	2	Quãng đường (Trip)
0x1F3	2	Trạng thái Node 1

Bảng 4.3 Dữ liệu ID Node 1

Các ID được tạo ra để mô phỏng lại với độ dài khác nhau của các dữ liệu CAN trên xe, được nhóm thiết kế và quy ước riêng vì trên thực tế mỗi hãng xe đều bảo mật tệp từ điển CAN (DBC file) của họ và gán ID theo quy tắc riêng.

Các dữ liệu sau khi được tính toán và mô phỏng sẽ được STM32F103C8T6 đóng gói, gán các thông tin như định danh CAN ID và độ dài dữ liệu (DLC) để đảm bảo tuân thủ đúng cấu trúc thông điệp của chuẩn truyền thông CAN, sau đó tiến hành truyền tải lên mạng CAN bus. Hình 4.2 ở dưới là kết quả của câu lệnh *candump can0* trên Raspberry để kiểm tra dữ liệu giá trị của mạng CAN

CHƯƠNG 4: KẾT QUẢ HỆ THỐNG

can0	101	[2]	07	D0						
can0	1F1	[8]	01	00	00	00	00	00	00	00
can0	100	[2]	00	3C						
can0	101	[2]	07	D0						
can0	106	[2]	00	15						

Hình 4.3 Kiểm tra dữ liệu Node 1 bằng lệnh

STT	CAN ID	Thông số	Giá trị HEX	Giá trị sau xử lý
1	0x100	Tốc độ xe (Speed)	00 3C	60 km/h
2	0x101	Vòng tua máy (RPM)	07 D0	2000 vòng/phút
3	0x106	Quãng đường (Trip)	00 15	2.1 km
4	0x1F1	Trạng thái Node (Heartbeat)	01 00 00 00 00 00 00 00	Node 1 OK

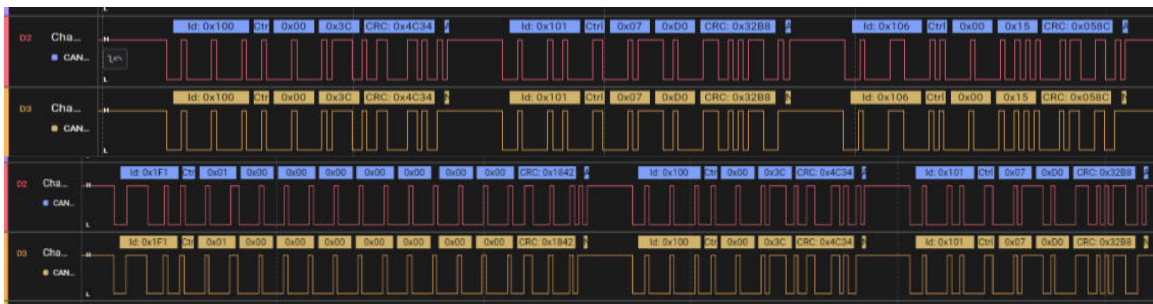
Bảng 4.4 Dữ liệu ID Node 1 sau khi xử lý



Hình 4.4 Màn hình hiển thị tốc độ - vòng tua

- Vòng đồng hồ tốc độ: Đang hiển thị trực quan tốc độ di chuyển của xe, duy trì ổn định ở mức 60 km/h. Từ đó tính ra Trip là 2.1 km sau 126 giây.
- Vòng tua máy: Thể hiện thông số vòng quay động cơ đang hoạt động đạt mức 2.000 vòng/phút (được hiển thị rút gọn trên giao diện là 2.0K RPM).
- Trạng thái Node 1: Hiển thị "NODE 1 OK" (Xác nhận tín hiệu Heartbeat và kết nối mạng CAN ổn định).

CHƯƠNG 4: KẾT QUẢ HỆ THỐNG



Hình 4.5 Đo dữ liệu CAN Node 1 bằng logic analyzer

Hình 4.5 là hình ảnh bắt tín hiệu dữ liệu mạng CAN tại Node 1 bằng Logic Analyzer. Với D2 gắn với chân RX và D3 là chân TX của STM32F1003C8T6, tín hiệu của dữ liệu đóng gói và phát chuẩn xác các khung dữ liệu của CAN (ID 0x100, 0x101, 0x106, 0x1F1)

- Node1 truyền tải liên tục các ID vận hành bao gồm 0x100 (Tốc độ – 0x3C), 0x101 (Vòng tua – 0xD0) và 0x106 (Quãng đường – 0x15).
- Có thêm khung truyền trạng thái 0x1F1 (Heartbeat) với độ dài 8 byte. Byte đầu tiên mang giá trị 0x01 xác nhận phần cứng Node 1 đang hoạt động ổn định.

4.4 Kết quả mô phỏng và hiện thị Node 2

Dữ liệu Node 2 bao gồm tốc độ vòng tua được mô phỏng từ 3 con biến trở kết nối với STM32F103C8T6. Các ID dữ liệu CAN của Node 2 bao gồm:

CAN ID	DLC	Dữ liệu
0x102	1	Mức pin (Battery)
0x104	1	Hộp số (Gear)
0x108	1	Chế độ lái (Dmode)
0x1F2	8	Trạng thái Node 2

Bảng 4.5 Dữ liệu ID Node 2

Tương tự Node 1, các dữ liệu Node 2 sau khi được tính toán và mô phỏng sẽ được STM32F103C8T6 đóng gói, gán các thông tin như định danh CAN ID và độ dài dữ liệu (DLC) để đảm bảo tuân thủ đúng cấu trúc thông điệp của chuẩn truyền thông CAN, sau đó tiến hành truyền tải lên mạng CAN bus.

CHƯƠNG 4: KẾT QUẢ HỆ THỐNG

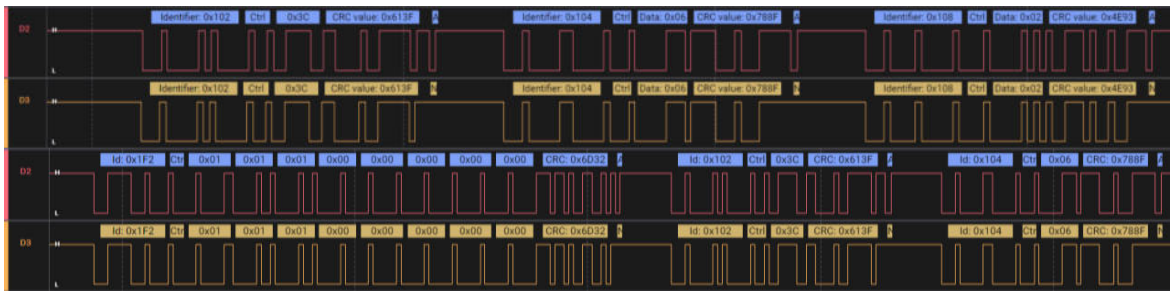
Hình 4.6 ở dưới là kết quả của câu lệnh `candump can0` trên Raspberry để kiểm tra dữ liệu giá trị của mạng CAN Node 2

```
can0 102 [1] 3C
can0 104 [1] 06
can0 108 [1] 02
can0 1F2 [8] 01 01 01 00 00 00 00 00
```

Hình 4.6 Kiểm tra dữ liệu ID Node 2 bằng lệnh

STT	CAN ID	Thông số	Giá trị HEX	Xử lý dữ liệu
1	0x102	Mức Pin (Battery)	3C	Dung lượng pin 60 %
2	0x104	Hộp số (P-R-N-D)	06	Gear D (Drive)
3	0x108	Chế độ lái (DMode)	02	Sport mode (Thể thao)
4	0x1F2	Trạng thái Node 2 (Heartbeat)	01 01 01 00 00 00 00 00	Node 2 OK

Bảng 4.6 Dữ liệu ID Node 2 sau khi xử lý

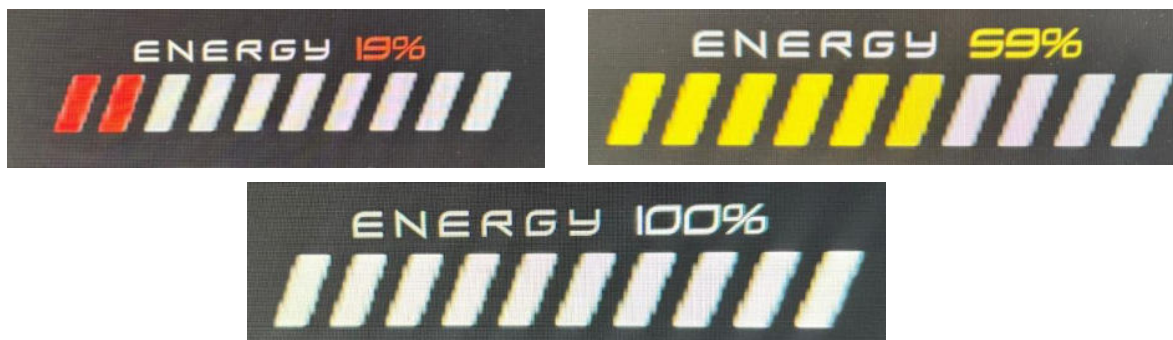


Hình 4.7 Đo dữ liệu CAN Node 2 bằng logic analyzer

Hình 4.7 là hình ảnh bắt tín hiệu dữ liệu mạng CAN tại Node 2 bằng Logic Analyzer. Với D2 gắn với chân RX và D3 là chân TX của STM32F103C8T6, tín hiệu được đóng gói và phát chuẩn xác các khung dữ liệu của CAN (ID 0x102, 0x104, 0x108, 0x1F2).

- Node 2 truyền tải liên tục các ID vận hành bao gồm 0x102 (Mức pin – 0x3C), 0x104 (Hộp số – 0x06) và 0x108 (Chế độ lái – 0x02).
- Có thêm khung truyền trạng thái 0x1F2 (Heartbeat) với độ dài 8 byte. Các byte đầu tiên mang giá trị 0x01 xác nhận phần cứng Node 2 đang kết nối và hoạt động ổn định.

CHƯƠNG 4: KẾT QUẢ HỆ THỐNG



Hình 4.8 Màu sắc của mức Pin – Battery (a b c)

a. Pin dưới 20% b. Pin dưới 60% c. Pin 100%

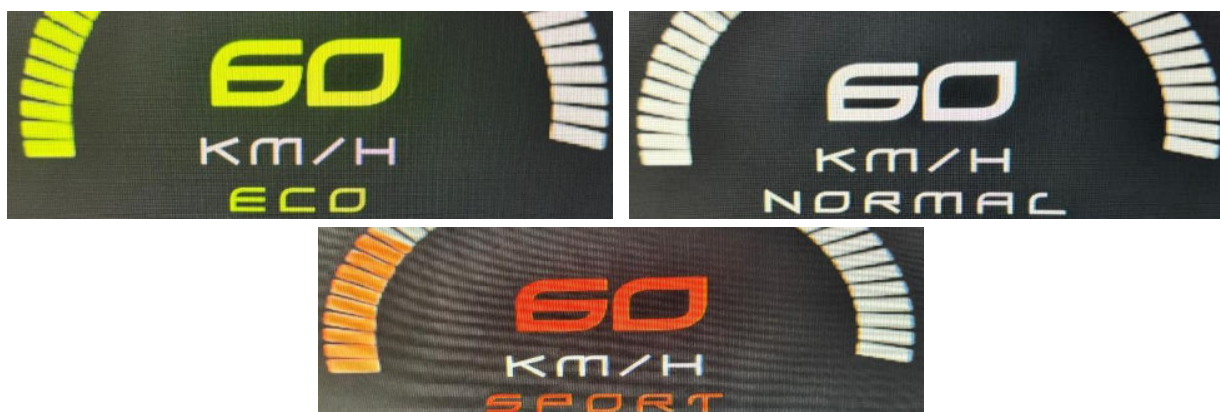
Hình 4.8 minh họa mức pin (ID 0x102) tự động đổi màu trực quan theo dung lượng: cảnh báo đỏ ($<20\%$, Data $< 0x14$), vàng ($<60\%$, Data $< 0x3C$) và trắng (100% , Data $0x64$).



Hình 4.9 Vị trí của các hộp số – Gear (a b c d)

a. Vị trí P b. Vị trí R c. Vị trí N d. Vị trí D

Hình 4.9 minh họa giao diện hiển thị vị trí hộp số (ID 0x104) tự động cập nhật theo dữ liệu CAN. Vị trí – ID data của từng Gear (P – $0x01$, R – $0x02$, N – $0x03$, D – $0x04$).



Hình 4.10 Màu sắc của chế độ lái – Dmode (a b c)

a. ECO b. Normal c. Sport

Hình 4.10 minh họa giao diện hiển thị chế độ lái (ID 0x108) tự động cập nhật theo dữ liệu CAN. Chế độ – ID data của từng Drive Mode (ECO – $0x01$, NORMAL – $0x00$, SPORT – $0x02$).

CHƯƠNG 4: KẾT QUẢ HỆ THỐNG

4.5 Kết quả mô phỏng và hiện thị Node 3

Dữ liệu Node 3 bao gồm dữ liệu xi nhan, nhiệt độ và thời gian được mô phỏng từ công tắc gạt, cảm biến BMP 180 và RTC DS1307 kết nối với STM32F103C8T6. Các ID dữ liệu CAN của Node 3 bao gồm:

CAN ID	DLC	Dữ liệu
0x103	1	Nhiệt độ (Temp)
0x105	5	Thời gian (RTC)
0x107	1	Xi nhan (Turn signal)
0x1F3	8	Trạng thái Node 3

Bảng 4.7 Dữ liệu ID Node 3

Tại Node 3, STM32F103C8T6 tiến hành đọc, đóng gói tín hiệu từ công tắc gạt, cảm biến BMP180 và mạch DS1307. Các thông số thực này lập tức được gán định danh (CAN ID 0x103, 0x105, 0x107, 0x1F3) cùng chỉ số DLC hợp lệ để truyền tải lên mạng CAN bus.

Hình 4.10 ở dưới là kết quả của câu lệnh *candump can0* trên Raspberry để kiểm tra dữ liệu giá trị của mạng CAN Node 3

```
can0 103 [1] 21
can0 105 [5] 1A 06 10 17 12
can0 1F3 [8] 01 01 01 00 00 00 00 00
can0 107 [1] 00
can0 107 [1] 01
can0 107 [1] 02
```

Hình 4.11 Kiểm tra dữ liệu Node 3 bằng lệnh

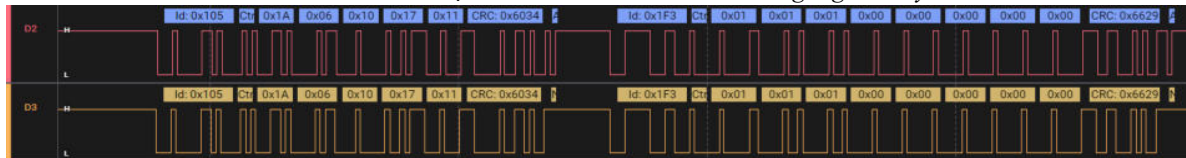
STT	CAN ID	Thông số	Giá trị HEX	Giá trị sau xử lý
1	0x103	Nhiệt độ (Temp)	21	33 °C
2	0x105	Thời gian (RTC)	1A 06 10 17 12	23:18 ngày 16/06/2026
3	0x107	Xi nhan (Turn signal)	00 01 02	Tắt xi nhan Xi nhan trái Xi nhan phải
4	0x1F3	Trạng thái Node 3 (Heartbeat)	01 01 01 00 00 00 00 00	Node 3 OK

Bảng 4.8 Dữ liệu ID Node 3 sau khi xử lý

CHƯƠNG 4: KẾT QUẢ HỆ THỐNG



Hình 4.12 a Đo dữ liệu CAN Node 3 - ID 0x103 bằng logic analyzer



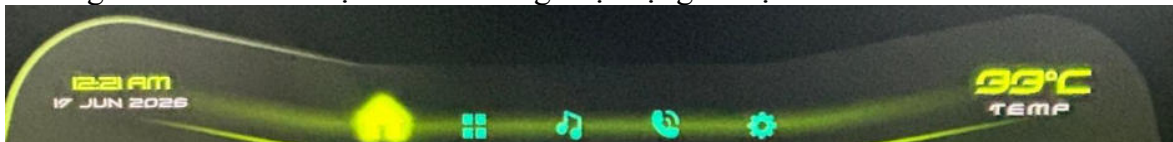
Hình 4.12 b Đo dữ liệu CAN Node 3 - ID 0x105 và ID 0x1F3 bằng logic analyzer



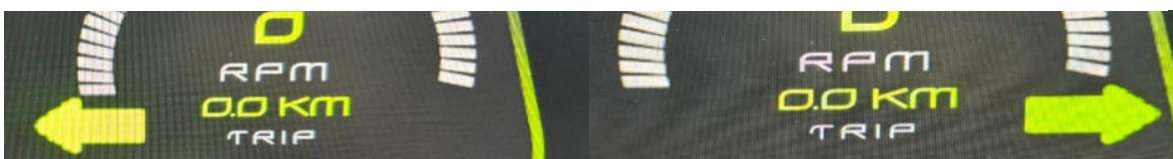
Hình 4.12 c Đo dữ liệu CAN Node 3 - ID 0x107 bằng logic analyzer

Hình 4.12 (a, b, c) ghi nhận tín hiệu mạng CAN tại Node 3 bằng Logic Analyzer qua hai chân RX (D2) và TX (D3) của STM32F103C8T6 đã thu thập và phát chuẩn xác các khung dữ liệu phần cứng:

- Truyền liên tục thông số vật lý: ID 0x103 (Nhiệt độ – 0x21), ID 0x105 (Thời gian RTC – 0x1A 0x06 0x10 0x17 0x11) và ID 0x107 (Xi nhan – 0x00).
- Khung trạng thái ID 0x1F3 (Heartbeat) mang giá trị 0x01 phát định kỳ để hệ thống giám sát và xác nhận Node 3 đang hoạt động ổn định.



Hình 4.13 Vị trí của thời gian và nhiệt độ trên giao diện



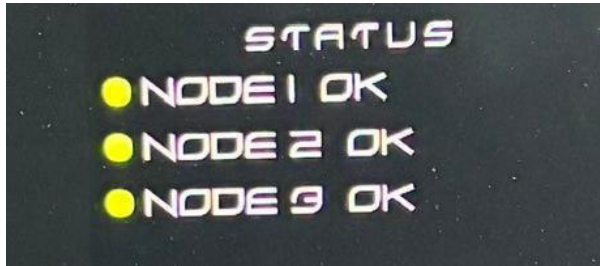
Hình 4.14 Vị trí của xi nhan trái – phải trên giao diện

- Hình 4.13 là cụm thông tin thời gian và nhiệt độ được đặt ở hai góc trên cùng của giao diện: thời gian thực (giờ, ngày tháng) nằm ở góc trên bên trái, thông số nhiệt độ hiển thị ở góc trên bên phải.
- Hình 4.14 là tín hiệu cảnh báo rẽ (xi nhan) sử dụng biểu tượng mũi tên màu vàng, được bố trí trực quan ở khu vực phía dưới của hai vòng đồng hồ hai bên (xi nhan trái ở cụm đồng hồ bên trái, xi nhan phải ở cụm đồng hồ bên phải).

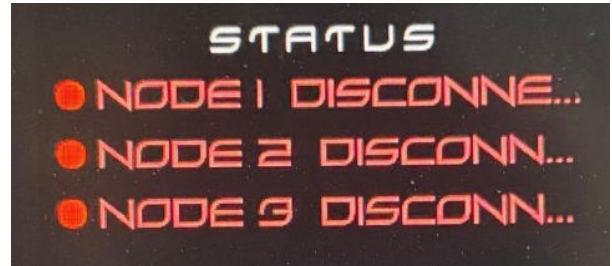
CHƯƠNG 4: KẾT QUẢ HỆ THỐNG

4.6 Kết quả hiển thị của trạng thái các Node

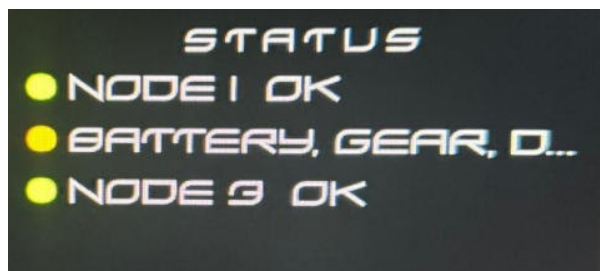
Giải lập các trường hợp các Node CAN bị lỗi:



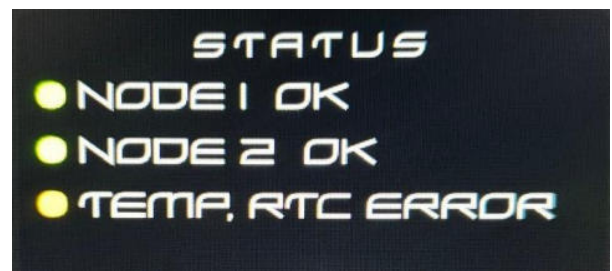
Hình 4.15 a Trạng thái 3 Node hoạt động tốt



Hình 4.15 b Trạng thái bị 3 Node ngắt kết nối



Hình 4.15 c Trạng thái Node 2 bị lỗi



Hình 4.15 d Trạng thái Node 3 bị lỗi

- Hình 4.15a: Hệ thống hoạt động tốt các đường truyền thông suốt, dữ liệu cảm biến gửi về đầy đủ và đạt trạng thái 0x01. Màn hình hiển thị cả 3 Node đều báo OK màu xanh.
- Hình 4.15b: Mất kết nối hoàn toàn, giả lập rút nguồn tổng hoặc các dây CAN. Quá 3 giây không có Heartbeat, Watchdog trên Pi 5 hết thời gian chờ, giao diện chuyển đỏ và báo lỗi DISCONNECTED....
- Node 1 lỗi ngoại vi, giả lập hỏng cảm biến tốc độ (Encoder). STM32 vẫn duy trì khung truyền 0x1F1, nhưng byte dữ liệu báo trạng thái Encoder chuyển sang mã lỗi. Dashboard nhận diện và hiển thị chi tiết lỗi phần cứng ENCODER ERROR
- Hình 4.15c: Node 2 lỗi thành phần, giả lập ngắt đường dây pin (Battery), hộp số (Gear) và chế độ lái (Dmode) . Node 2 đổi byte trạng thái trong gói 0x1F2, màn hình Dashboard lập tức cập nhật tên phân vùng gặp sự cố và báo BATTERY, GEAR, DMODE ERRORS.
- Hình 4.15d: Node 3 lỗi cảm biến giả lập rút module BMP180 và DS1307. Hàm đọc ngoại vi báo lỗi, ép các biến trạng thái về 0xFF. Gói tin 0x1F3 mang mã lỗi truyền về Pi 5, giao diện lập tức đổi LED sang vàng và báo TEMP, RTC ERROR.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết Luận

Sau quá trình nghiên cứu lý thuyết, thiết kế sơ đồ nguyên lý, thi công các bo mạch phần cứng và tối ưu hóa mã nguồn điều khiển thời gian thực, đề tài "Xây dựng hệ thống cụm đồng hồ số trên xe hơi" nhóm nghiên cứu đã hiện thực hóa mô hình giả lập IPC chạy hệ điều hành nhúng Yocto ổn định ở tần suất cập nhật dữ liệu 60 FPS mà không xảy ra hiện tượng mất gói tin mạng. Nhóm sinh viên xin tổng hợp các kết quả thực nghiệm đạt được như sau:

Hệ thống đã triển khai thành công mạng giao tiếp CAN bus đa điểm giữa ba nút mạng độc lập hoạt động đồng bộ. Các khung tin từ Node 1 (thu thập dữ liệu tốc độ, vòng tua), Node 2 (thu thập dữ liệu mô phỏng mức Pin, hộp số xe, chế độ lái) cho đến Node 3 (quét trạng thái công tắc xi-nhan, đọc cảm biến nhiệt độ trong xe và thời gian thực) đều được đóng gói và truyền tải chính xác. Cụm đồng hồ trung tâm sử dụng màn hình Waveshare 7 inch đã thực hiện bóc tách dữ liệu thu nhận từ Bus mạng một cách mượt mà, hiển thị trực quan các thông số vận hành lên giao diện số hóa mà không xảy ra hiện tượng trễ hay mất mát dữ liệu thời gian thực.

Về mặt thuật toán lập trình nhúng, nhóm đã giải quyết tốt bài toán ổn định tín hiệu tương tự từ cảm biến và biến trở thông qua các bộ lọc số kết hợp bảng nội suy tuyến tính. Điểm sáng công nghệ là việc ứng dụng thành công cơ chế tự động bẫy lỗi Bus-Off trên thanh ghi điều khiển CAN, cho phép hệ thống tự phục hồi đường truyền mà không gây treo tiến trình xử lý. Kết quả thực tế đã chứng minh tính đúng đắn của phương pháp thiết kế phần mềm phân tầng chu kỳ lồng và kiến trúc phần cứng chống nhiễu vi sai trên đường truyền CAN bus. Kết quả vận hành thực tế của cụm đồng hồ số không chỉ chứng minh tính đúng đắn của giải pháp thiết kế mà còn giúp nhóm tối ưu hóa quy trình bố trí linh kiện (layout) trên mạch tốc độ cao.

5.2 Hướng phát triển

Để đưa hệ thống cụm đồng hồ số từ mô hình giả lập trong phòng thí nghiệm ứng dụng trực tiếp trên xe thương mại, các nghiên cứu tiếp theo cần tập trung vào các giải pháp sau:

- Nâng cấp giao thức truyền thông nâng cao, chuyển đổi cấu trúc điều khiển sang chuẩn mạng mới có băng thông rộng và tốc độ cao hơn hoặc tích hợp thêm mạng Ethernet chuyên dụng trên xe hơi nhằm mở rộng tối đa dung lượng dữ liệu và tăng tốc độ truyền tải thông tin giữa các hộp điều khiển.
- Tích hợp các tầng thư viện chẩn đoán tiêu chuẩn, cài đặt giao thức chẩn đoán lỗi quốc tế nhằm giúp cụm đồng hồ có khả năng tự động phát hiện, lưu trữ các mã lỗi phần cứng và hỗ trợ truy xuất thông tin chẩn đoán thông qua cổng kết nối tiêu chuẩn hóa.
- Cải tiến thiết kế đồ họa người máy (HMI), ứng dụng các thư viện đồ họa nhúng mã nguồn mở mạnh mẽ trực tiếp lên phần cứng màn hình chuyên dụng nhằm thiết kế giao diện hiển thị đồng hồ không gian ba chiều, tối ưu hóa tốc độ làm tươi đem lại trải nghiệm trực quan và hiện đại hơn cho người dùng.
- Mở rộng mô hình chẩn đoán từ xa qua điện toán đám mây: Tích hợp thêm các module truyền thông không dây băng thông rộng để đồng bộ hóa liên tục dữ liệu vận hành từ mạng giao tiếp lên máy chủ, phục vụ bài toán giám sát hành trình xe và hỗ trợ kỹ thuật từ xa.
- Quy chuẩn hóa và tối ưu thiết kế phần cứng vật lý: Tiến hành thu nhỏ kích thước các mạch, bổ sung thêm mạch cách ly và các tầng bảo vệ chống dòng biến thiên cho đường dây vi sai, đồng thời gia công vỏ hộp bảo vệ đạt tiêu chuẩn kháng bụi nước cao để mô hình chịu được điều kiện rung lắc trên xe thực tế.

PHỤ LỤC

Mã nguồn chương trình và video ghi lại quá trình chạy thực nghiệm của hệ thống được tổng hợp tại các đường link dưới đây để thuận tiện cho việc truy cập, nghiệm thu:

- Mã nguồn chương trình:

https://github.com/zilfrost/IPC_Project

- Video demo sản phẩm:

https://drive.google.com/drive/folders/1yrB5wbKdFbgZqy09yviStC5_ttBO7IHW?usp=sharing

TÀI LIỆU THAM KHẢO

- [1] International Organization for Standardization 11898-1 : Road vehicles — Controller area network (CAN) — Part 1: Data link layer and physical signalling, ISO Standard 11898-1:2015, 2015.
- [2] International Organization for Standardization 11898-2, *Road vehicles — Controller area network (CAN) — Part 2: High-speed medium access unit*, ISO Standard 11898-2:2016, 2016.
- [3] International Organization for Standardization 15765-2: Road vehicles — Diagnostic communication over Controller Area Network (DoCAN) — Part 2: Transport protocol and network layer services (ISO-TP). ISO Standard 15765-2:2016, 2016
- [4] International Organization for Standardization 11783: *Tractors and machinery for agriculture and forestry — Serial control and communications data network*, ISO Standard 11783, 2017 (ISOBUS).
- [5] SAE International, *Serial Control and Communications Heavy Duty Vehicle Network*, SAE Standard J1939, 2018.
- [6] CircuitState, "What is CAN Bus? How to use CAN interface with ESP32 and Arduino." [Online]. Available: <https://www.circuitstate.com/tutorials/what-is-can-bus-how-to-use-can-interface-with-esp32-and-arduino/>.
- [7] SparkFun Electronics. (n.d.). *Serial Peripheral Interface (SPI)* [Online]. Available: <https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi>
- [8] SparkFun Electronics. (n.d.). *I2C* [Online]. Available: <https://learn.sparkfun.com/tutorials/i2c>
- [9] Philips Semiconductors. *The I2C-bus specification, Version 2.1.* (2000). [Online]. Available: https://www.csd.uoc.gr/~hy428/reading/i2c_spec.pdf
- [10] InterData. (2025, Oct. 27). *VS Code là gì? Tính năng, Ưu/Nhược điểm & Các lệnh cơ bản* [Online]. Available: <https://interdata.vn/blog/vs-code-la-gi/>

- [11] ATP Software. (2025, Sep. 12). EasyEDA - Công Cụ Thiết Kế Vi Mạch - 2026 [Online]. Available: <https://atpsoftware.vn/easyeda-cong-cu-thiet-ke-vi-mach>
- [12] TAPIT. (2023, Dec. 5). *Hướng dẫn nhúng mô hình học sâu nhận diện âm thanh lên vi điều khiển STM32* [Online]. Available: <https://tapit.vn/huong-dan-nhung-mo-hinh-hoc-sau-nhan-dien-thanh-len-vi-dieu-khien-stm32/>
- [13] FPT Shop. (2026, May 1). *Cách sử dụng Figma để thiết kế giao diện đẹp, tối ưu trải nghiệm* [Online]. Available: <https://fptshop.com.vn/tin-tuc/giai-tri/cach-su-dung-figma-176061>
- [14] Thế Giới IC. (n.d.). *TJA1050 CAN Bus Module*. [Online]. Available: <https://www.thegioiic.com/tja1050-can-bus-module>
- [15] Thế Giới IC. (n.d.). *EC11 Rotary Encoder 20 Vị Trí Nút Dài 20mm* [Online]. Available: <https://www.thegioiic.com/ec11-rotary-encoder-20-vi-tri-num-dai-20mm>
- [16] Thế Giới IC. (n.d.). *Mạch STM32F103C8T6 Type-C* [Online]. Available: <https://www.thegioiic.com/mach-stm32f103c8t6-type-c>
- [17] Thế Giới IC. (n.d.). *MCP2551 Mạch CAN Giao Tiếp UART* [Online]. Available: <https://www.thegioiic.com/mcp2551-mach-can-giao-tiep-uart>
- [18] Thế Giới IC. (n.d.). *WH148-3P-103 Biến Trở Chiết Áp Volume Đơn 10 kOhm 20% 0.125W 3 Chân* [Online]. Available: <https://www.thegioiic.com/wh148-3p-103-bien-tro-chiet-ap-volume-don-10-kohm-20-0-125w-3-chan>
- [19] Thế Giới IC. (n.d.). *MTS-103 Công Tắc Toggle ON-OFF-ON Gạt Giữ 3 Chân 6A-125VAC* [Online]. Available: <https://www.thegioiic.com/mts-103-cong-tac-toggle-on-off-on-gat-giu-3-chan-6a-125vac>
- [20] Thế Giới IC. (n.d.). *GY-68 BMP180 Cảm Biến Áp Suất* [Online]. Available: <https://www.thegioiic.com/gy-68-bmp180-cam-bien-ap-suât>

- [21] Thế Giới IC. (n.d.). *HW-111 Mạch Thời Gian Thực TinyRTC DS1307 Giao Tiếp I2C* [Online]. Available: <https://www.thegioic.com/hw-111-mach-thoi-gian-thuc-tinyrtc-ds1307-giao-tiep-i2c>
- [22] Thế Giới IC. (n.d.). *MCP2515 Mạch CAN Giao Tiếp SPI* [Online]. Available: <https://www.thegioic.com/mcp2515-mach-can-giao-tiep-spi>
- [23] Thế Giới IC. (n.d.). *I2C UART SPI TTL Mạch Chuyển Mức Tín Hiệu 4 Kênh* [Online]. Available: <https://www.thegioic.com/i2c-uart-spi-ttl-mach-chuyen-muc-tin-hieu-4-kenh>
- [24] Raspberry Pi Ltd., "Raspberry Pi 5 Product Brief", [Online]. Available: <https://pip.raspberrypi.com/documents/RP-008348-DS-raspberry-pi-5-product-brief.pdf>
- [25] Raspberry Pi Ltd., "RP1 Peripherals", [Online]. Available: <https://datasheets.raspberrypi.com/rp1/rp1-peripherals.pdf>
- [26] Waveshare. (n.d.). *7inch HDMI LCD (C)* [Online]. Available: [https://www.waveshare.com/wiki/7inch_HDMI_LCD_\(C\)](https://www.waveshare.com/wiki/7inch_HDMI_LCD_(C))
- [27] Thế Giới IC. (n.d) *Nút Nhấn 12x12mm* [Online]. Available: <https://www.thegioic.com/b3f-4055-nut-nhan-12x12mm-cai-7-3mm-4-chan-xuyen-lo>
- [28] Trần Thu Hà – Trương Thị Bích Nga – Nguyễn Thị Lương – Bùi Thị Tuyết Đan – Phù Thị Ngọc Hiếu – Dương Thị Cẩm Tú, Giáo trình Điện tử cơ bản, Nhà xuất bản đại học quốc gia TP. Hồ Chí Minh, 2013
- [29] Nguyễn Minh Tâm – Trần Tùng Giang – Lê Thị Thanh Hoàng, Giáo trình Mạch điện tập 1, Nhà xuất bản đại học quốc gia TP. Hồ Chí Minh, 2017
- [30] Nguyễn Minh Tâm – Trần Tùng Giang – Lê Thị Thanh Hoàng – Lê Mỹ Hà, Giáo trình Mạch điện tập 2, Nhà xuất bản đại học quốc gia TP. Hồ Chí Minh, 2017

Minh Le

22119160&22119212_Xay dung cum dong ho ky thuat so tren xe hoi

 25-26,hk2,An,Phuc

 check

 Ho Chi Minh City University of Technology and Engineering (HCMUTE)

Document Details

Submission ID**trn:oid::1:3599461656****Submission Date****Jun 22, 2026, 2:07 PM GMT+7****Download Date****Jun 22, 2026, 2:25 PM GMT+7****File Name****19160_22119212_Xay_dung_cum_dong_ho_ky_thuat_so_tren_xe_hoi.docx****File Size****36.2 MB****113 Pages****28,426 Words****104,880 Characters**

19% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Top Sources

- 16%  Internet sources
- 5%  Publications
- 10%  Submitted works (Student Papers)

Integrity Flags

1 Integrity Flag for Review

-  **Replaced Characters**
158 suspect characters on 33 pages
Letters are swapped with similar characters from another alphabet.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Top Sources

- 16% Internet sources
- 5% Publications
- 10% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers		
	Ho Chi Minh University of Technology and Education	6%	
2	Internet		
	www.slideshare.net	1%	
3	Student papers		
	Post and Telecommunications Institute of Technology	<1%	
4	Internet		
	vi.wikitechpro.com	<1%	
5	Internet		
	text.123docz.net	<1%	
6	Student papers		
	Ton Duc Thang University	<1%	
7	Internet		
	tailieu.vn	<1%	
8	Internet		
	www.ctu.edu.vn	<1%	
9	Student papers		
	Thai Nguyen University of Education	<1%	
10	Publication		
	Phenikaa University	<1%	
11	Student papers		
	Vietnam Maritime University	<1%	